

Noviembre 2025



# Guía de Contenido 1er y 2do Parcial

Diseño de Sistemas de Información

# Índice

- Refactoring
- Entity vs Value Objects
- SOLID Principles
- Componentes Stateless vs Stateful
- Arquitectura de Software
- Modelo en Capas
- Cliente – Servidor
- API REST
- Interfaces entrantes y salientes
- Cuestiones de Sincronismo
- Solicitud – Respuesta
- Persistencia
- Modelo relacional
- Diseño y funcionamiento del ORM
- Sesión: Stateless vs Stateful
- Protección de Recursos: Roles, Permisos.
- Tipos de clientes
- Atributos de Calidad Arquitectónicos y sus tradeoffs
- Escalabilidad
- Arquitectura Monolítica y SOA
- Aplicaciones en la Nube vs Tradicionales
- Componentes de una Arquitectura
- Seguridad de la información
  - Autenticación vs Autorización
- Bounded Context
- Documentación de la Arquitectura
- Arquitectura Hexagonal
- Integración de sistemas
- Tipos de APIs
- Colas de Mensajes
- Broker: Apache Kafka
- Peer to Peer
- Trazabilidad: Logs, Métricas, Trazas, Alarmas
- Contenido Extra

# Inyección de Dependencias

>> Incidente

```
public void enviarAvisos(){
    this.interesados.forEach(i -> whatsappSender.enviar(i, "Ha ocurrido un incidente"));
}
```

Suponiendo que existe un objeto “whatsappSender”, ¿cómo llegamos a conocerlo? Algunas opciones:

- Tener una única instancia de su clase accesible globalmente.
- Obtener ese objeto a través de otro que lo provea.
- Que a la clase Incidente le llegue por parámetro whatsappSender

>> Incidente

```
public void enviarAvisos(){
    this.interesados.forEach(i -> WhatsAppSender.getInstance().enviar(i, "Ha ocurrido un incidente"));
}
```

>> WhatsAppSender

```
private static WhatsAppSender instance = null;

public static WhatsAppSender getInstance() {
    if(instance == null)
        instance = new WhatsAppSender(); // más toda la configuración
    return instance;
}
```

>> Incidente

```
public void enviarAvisos(){
    this.interesados.forEach(i -> ServiceLocator.get("whatsappSender").enviar(i, "Ha ocurrido un incidente"));
}
```

>> Incidente

```
public Incidente(WhatsAppSender whatsappSender){...}

public void enviarAvisos(){
    this.interesados.forEach(i -> this.whatsappSender.enviar(i, "Ha ocurrido un incidente"));
}
```

Suponiendo que **A**, instancia de la clase 1, necesita a **B**, instancia de la clase 2, para poder realizar la tarea **X**...

## Singleton

- **A solicita a clase 2 una instancia, la cual devuelve siempre la instancia B.**
- “B” es un objeto global, única instancia para toda la ejecución.
- Fuerte acoplamiento entre Clase 1 y Clase 2.
- Difícil de testear pues es complicado mockear a B

## Service Locator

- **A solicita al Service Locator alguien que sea capaz de realizar la tarea X y éste le devuelve la instancia B o algún otro objeto que cumpla con la misma interface.**
- El Service Locator es un objeto global que permite generar distintas configuraciones.
- Permite el mockeo de objetos

## Inyección de Dependencias

- **A recibe como parámetro, en su constructor, a B o algún otro objeto que cumpla con la misma interface.**
- A no solicita a nadie la instancia B (o algún otro similar), sino que “le llega desde afuera”.
- Es más testeable pues permite el mockeo de objetos.
- Se puede combinar con las anteriores.

# Refactoring

## ¿Qué es no refactorizar?

- Refactorizar no es incorporar funcionalidades nuevas.
- Refactorizar no es optimizar.

## ¿Qué es refactorizar?

“Es cambiar la estructura del código para hacerlo más simple y extensible. Esto involucra muchas acciones posibles, algunas simples como cambiar el nombre de una clase o un método, o cosas más complejas como reemplazar condicionales (ifs) por objetos polimórficos o bien separar una clase en varias.”

## Se busca

1. Métodos cortos y con nombres bien definidos: que revelen exactamente el propósito para el que fueron creados
2. Métodos y clases con responsabilidades claras y bien definidas (mayor cohesión)
3. Respecto a las variables de instancia: hay que evaluar el costo-beneficio de tener en variables valores que puedan calcularse, como el total de deuda de un cliente, o la cantidad de hijos de un empleado
4. No tener "god objects" ni "managers", objetos que roban responsabilidades que les corresponden a otros objetos.
5. Es preferible tener objetos chicos antes que un objeto grande con muchas responsabilidades (sobre todo si los objetos chicos pueden intercambiarse).
6. Evitar ciclos de dependencia entre objetos si no los necesito; esto es relaciones bidireccionales innecesarias. Ej: cada cliente conoce a sus facturas, la factura ¿necesita conocer al cliente?

## Code Smells

- Código duplicado:
  - Don't repeat yourself.
  - Once and only once.
- Métodos largos:
  - Identificar el objetivo que cumple
  - Implementar esa abstracción: ponerle un nombre representativo y encontrar que objeto es responsable de ese objetivo.
  - Los métodos generados deberían poder ser utilizados en otro contexto.
  - Delegar el método original en varios submétodos nos permite entender mejor qué es lo que hace.
- God Class: poseen demasiadas responsabilidades, que están fuertemente acopladas a muchos otros objetos y que, por consecuencia, ante cualquier cambio se ven afectadas. Se puede solucionar...
  - Generando nuevas clases: alta cohesión.
  - Generar subclases: especificar los comportamientos con jerarquías.



- Generar interfaces: brinda una lista de operaciones con diferentes comportamientos que el cliente puede usar.

Si la God Class es responsable de una interfaz gráfica, se debe comenzar a pensar en el patrón MVC, e ir descomponiendo las responsabilidades entre las diferentes capas involucradas.

- Parámetros largos:
  - Puede darse cuando varios algoritmos se combinan en un solo método. O también cuando nos estamos olvidando de alguna abstracción del dominio.
  - Si un objeto va a brindar un servicio a otro, lo mejor sería que la cantidad de parámetros del método en cuestión no varíe.

```
>> CatalogoDePelis
public void getPelicula(String nombrePeli, String autor, int anio, int versión, int
estrellas){...}
```

Este método está totalmente sujeto a esos parámetros, y cualquier cambio en alguno de ellos afectaría a todo nuestro código. Del mismo modo, al querer agregar un parámetro más, estaríamos “rompiendo” con el cliente que está utilizando este servicio.

Una posible solución:

```
public void getPelicula(BusquedaPelicula peliBuscada){...}
```

- Cadena de mensajes
  - Don't talk to strangers: el objeto solo debería enviar mensajes...
    - A sí mismo
    - A objetos que conoce (como variables de instancia)
    - A objetos que recibe como parámetro
    - A objetos que instancia
- Herencia Rechazada: Se puede dar cuando una clase sobrescribe completamente un método base definido por su padre sin importarle lo que éste le haya dicho que tenía que hacer, o cuando una clase hereda métodos que en realidad no debería tener.
  - Si no son lo mismo, ¿para qué forzarlo? Usar composición por sobre herencia.
- Lazy Class: se da en casos de extra-delegación donde se diseñó por las dudas el componente.
  - YAGNI: para no agregar funcionalidad hasta que sea necesario.
  - Eliminar de nuestro diseño las clases PLD (por las dudas)
- Nombres:
  - De variables cortos: deben representar su funcionalidad
  - En métodos: si nos cuesta elegir un nombre en particular es porque hay mucha lógica en el mismo
- Código muerto: métodos o clases PLD que nos marean a la hora de ver la responsabilidad de una clase.
  - Borrar el código muerto y diseñar de forma que sea extensible, cerrado al cambio.
- Primitive Obsesion: se selecciona comportamiento con primitivos. Hay que buscar abstracciones que nos permitan trabajar polimórficamente.
- Data class: no necesariamente se busca separar los atributos de los comportamientos.

- Métodos fuera de lugar e Intermediario: Si no accedemos a ningún atributo ni tampoco utilizamos comportamiento interno, posiblemente estamos ubicando mal el método: ¿por qué no enviar directamente el mensaje a “Comunidad”?

>> `MiembroDeComunidad`

```
public void unirseAComunidad(Comunidad comunidad){
    comunidad.agregarMiembro(this);
}
```

## Entites vs Value Objects

Representan abstracciones para el dominio: los entity objects son para las principales o relevantes al dominio, mientras que los value objects, son para las abstracciones “accesorias” que solo guardan datos. Solo tienen sentido si se contextualiza con alguna entidad.

### Tipos de igualdad

| Igualdad de referencia (identidad)                                                                                                                                      | Igualdad estructural (equivalencia)                                                                                                                              | Igualdad de identificadores                                                                                                                                                                                                       |
|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <ul style="list-style-type: none"> <li>• Dos objetos son iguales si son exactamente el mismo, esto es, si están referenciando a la misma porción de memoria.</li> </ul> | <ul style="list-style-type: none"> <li>• Dos objetos son iguales si todos los valores que tienen sus atributos, en un momento determinado, coinciden.</li> </ul> | <ul style="list-style-type: none"> <li>• Implica que una clase tiene un atributo de identificación, natural o ficticio.</li> <li>• Dos objetos son iguales si tienen exactamente el mismo valor para su identificador.</li> </ul> |

### Vida útil

| Entidades                                                                                                                                                       | Value objects                                                                                                                                                                                               |
|-----------------------------------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <ul style="list-style-type: none"> <li>• Viven “a largo plazo”</li> <li>• Pueden tener un historial.</li> <li>• Tienen sentido de existencia propio.</li> </ul> | <ul style="list-style-type: none"> <li>• Son <b>desechables</b>, es decir que su vida útil es “muy corta”.</li> <li>• No tienen sentido de existencia propio, sino que están atados a entidades.</li> </ul> |

### Inmutabilidad

| Entidades                                                                                                                                                    | Value objects                                                                                                                                                                                                          |
|--------------------------------------------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <ul style="list-style-type: none"> <li>• Generalmente son <b>mutables</b>, o sea, sus atributos pueden cambiar sus valores a lo largo del tiempo.</li> </ul> | <ul style="list-style-type: none"> <li>• Son <b>inmutables</b>, no puede cambiar los valores de los atributos.</li> <li>• Cambiar un atributo, implica una nueva instancia (funcional como Scala y Haskell)</li> </ul> |

## ¿Entidad o Value Object?

- Una entidad puede que no tenga comportamiento y eso no la convierte en un value object pues tiene sentido de existencia propio y es algo “muy relevante” para nuestro dominio.
- Un value object es un objeto inmutable, desechable, accesorio de algo principal, que no tiene sentido de existencia propio, sino que debe ser contextualizado.
- Obviamente una entidad puede tener comportamiento, es decir, lógica propia del negocio; y eso no le quita su condición de “entidad”.

## Bibliotecas vs Frameworks

| Biblioteca                                                                                                                                                                                                                                                                                                                                                                                            | Framework                                                                                                                                                                                                                                                                                                                                                                                                    |
|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <ul style="list-style-type: none"><li>• Es responsabilidad del desarrollador decidir cómo y cuándo utilizar los componentes.</li><li>• Las decisiones de diseño que se toman suelen tener bajo impacto en el diseño del código que la utiliza.</li><li>• Utilizan control directo: El usuario llama funciones de la biblioteca e instancia las estructuras que la biblioteca pueda definir.</li></ul> | <ul style="list-style-type: none"><li>• Este define la forma en la que se estructurará el código y el desarrollador se ve obligado a seguir esa forma</li><li>• Las decisiones de diseño que se toman pueden condicionar fuertemente el diseño del código cliente.</li><li>• Utilizan <b>control inverso</b>: Es el framework el que llama funciones abstractas que el usuario define en concreto.</li></ul> |

## SOLID Principles

### Single Responsibility

- Cada clase debe tener responsabilidad sobre una sola parte de la funcionalidad del software - > Clases Cohesivas
- Esta responsabilidad debe estar encapsulada por la clase, y todos sus servicios deben estar estrechamente alineados con esa responsabilidad.

**Evitar la clase “Dios” propiciando la alta Cohesión**

### Open Closed

- Las entidades deben estar abiertas para la expansión, pero cerradas para su modificación.
- Se basa en la implementación de herencias y el uso de interfaces para resolver el problema.

**Se sugiere evitar la utilización excesiva de los “switchs” y propiciar el polimorfismo entre objetos**

### Liskov Substitution

“Cada clase que hereda de otra puede usarse como su superclase sin necesidad de conocer las diferencias entre las clases derivadas. Lo mismo vale para las realizaciones de Interfaces”

**Se debería utilizar correctamente la herencia y las realizaciones de Interfaces**

## Interface Segregation

- Los clientes de un componente sólo deberían conocer de éste aquellos métodos que realmente usan y no aquellos que no necesitan usar.
- Muchas interfaces cliente específicas son mejores que una interfaz de propósito general.

**Se debería propiciar un diseño orientado a interfaces, para mantener el acoplamiento entre clases al mínimo posible, y también evitar generar interfaces extensas (con muchos métodos)**

## Dependency Inversion

- Los módulos de alto nivel no deben depender de módulos de bajo nivel. Ambos deben depender de abstracciones. Es una forma de desacoplar módulos.”

**Se sugiere utilizar inyectores de dependencias**

## Componentes Stateless y Stateful

“El estado de una aplicación (o de cualquier otra cosa) hace referencia a su condición o cualidad de existir en un momento determinado. Es su capacidad de estar. Que un sistema tenga estado depende del tiempo durante el cual se registra interacción con él y de la forma en que se debe almacenar esa información. “

### Componentes Stateless

- Un proceso, una aplicación o, genéricamente, un componente sin estado se refiere a los casos en que éstos están aislados.
- No se almacena información sobre las operaciones anteriores ni se hace referencia a ellas.

Cada operación se lleva a cabo desde cero, como si fuera la primera vez.

Los objetos Stateless no deberían tener estado interno; o si lo tienen, éste no debería condicionar el funcionamiento frente a las distintas peticiones que podría recibir el objeto en cuestión.

Si pensamos a nuestros objetos Stateless, podríamos llegar a la conclusión de que ellos son reutilizables; es decir, que no dependen de otros objetos para sobrevivir, sino que tienen sentido de existencia propio.

### Componentes Stateful

- Las aplicaciones, los procesos o, genéricamente, los componentes con estado son aquellos a los que se puede volver una y otra vez y éstos recuerdan “quiénes somos” y “qué hicimos anteriormente”.
- Se realizan con el contexto de las operaciones anteriores, y la operación actual puede verse afectada por lo que ocurrió previamente.

Si pensamos a nuestros objetos Stateful, podríamos llegar a la conclusión de que generalmente ellos no serían reutilizables; es decir, que por estar – al menos - levemente acoplados a algún otro objeto, no podrían servir para que otro objeto realice operaciones con ellos.

# Arquitectura de Software

Representa la estructura o estructuras del sistema que consiste en componentes de software, las propiedades externas visibles de esos componentes y las relaciones entre ellos.

## ¿Qué es un componente?

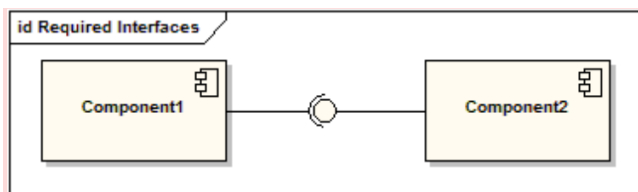
Es una pieza de software que puede ser código fuente, código binario, un ejecutable, o una biblioteca con una interfaz definida.

## ¿Qué es la interfaz de un Componente?

Establece las operaciones externas de un componente, las cuales determinan una parte del comportamiento del mismo.

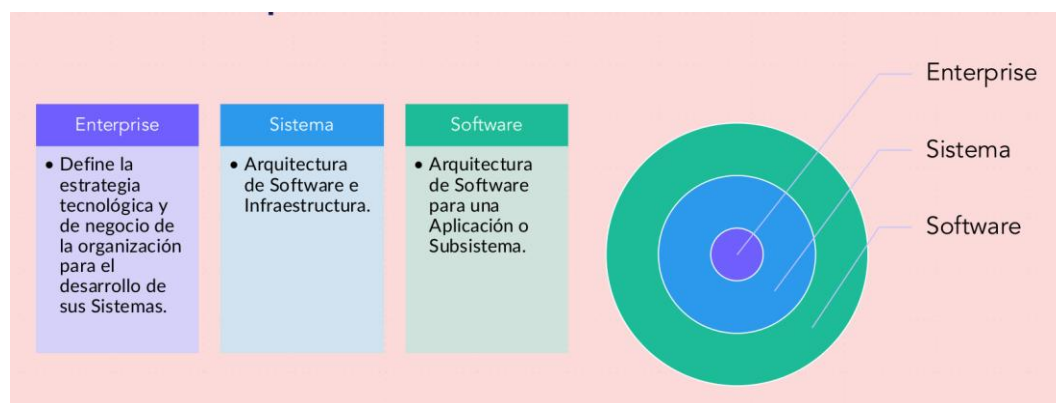
## ¿Cómo se relacionan con otros componentes? (en UML)

Se representan las dependencias entre componentes o entre un componente y la interfaz de otro, es decir uno de ellos usa los servicios o funcionalidades del otro.



El componente 1 consume la interfaz que disponibilidad el componente 2. (Como PacMan)

## Niveles de la Arquitectura de Software



## Características

- Diseño de más alto nivel, es diseño de nivel estratégico.
- No es solo lógico sino también físico y organizacional.
- Define el sistema en términos de elementos y de interacción entre ellos.
- Contiene las estrategias para resolver los atributos de calidad.

## Decisiones Arquitectónicas

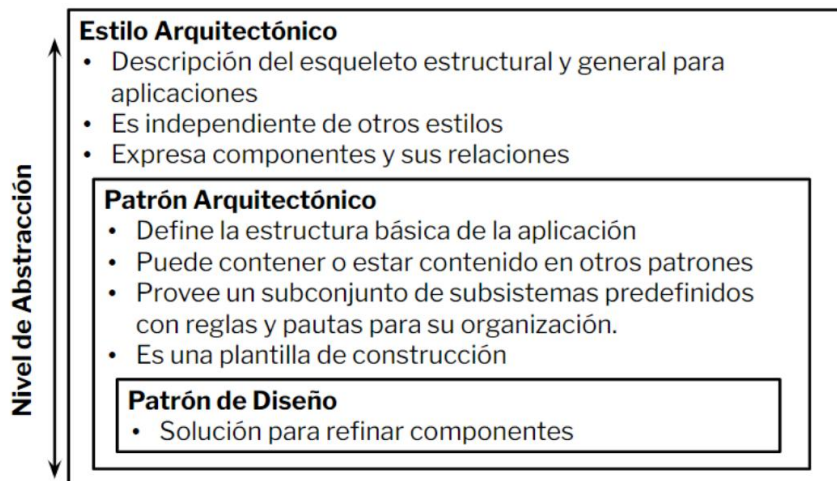
- Afectan a muchas partes del sistema.
- Le dan estructura al Sistema.

- Muchas de ellas deben tomarse en forma temprana.
- Suelen ser difíciles de cambiar.
- Identifican y mitigan Riesgos.

## Entradas

- Restricciones técnicas y de negocio.
- Experiencia del arquitecto.
- Estilos, patrones Arquitectónicos y Modelo en capas
- Requerimientos funcionales, no funcionales. Futuros requerimientos y atributos de calidad.

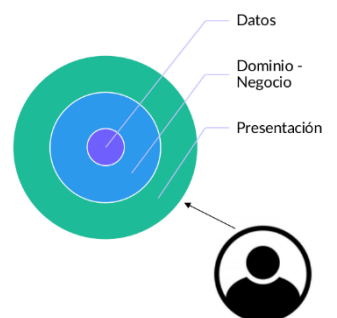
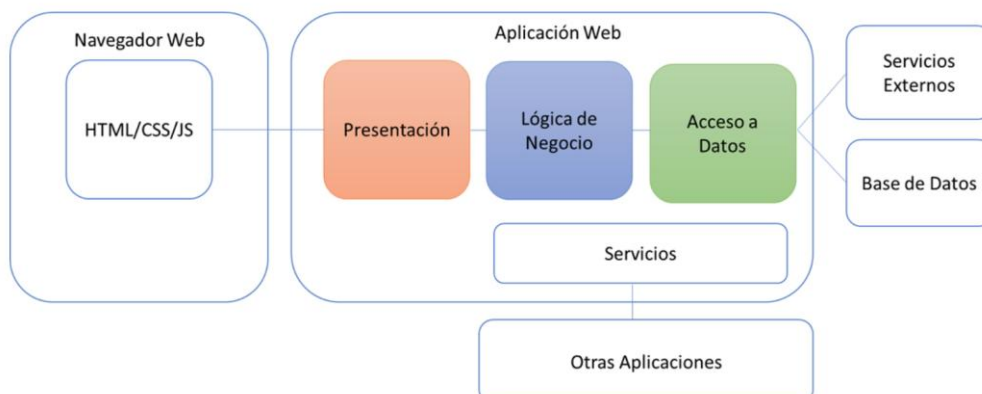
## Niveles de Abstracción



## Modelo en Capas

Dividir en capas (layering) es una de las técnicas más comunes para resolver un problema de diseño complejo. Donde cada capa es un todo coherente (cohesión), con un rol único en el sistema.

Las capas superiores usan servicios de las inferiores, pero no así de forma contraria o saltando niveles.



## Capa de presentación

Encargada de presentar los datos al usuario, con la cual éste interactuará. No es necesariamente es visual puede dividirse en al menos en dos formas:

- Presentación de datos mediante **interfaz gráfica** (interfaz desktop, interfaz Web, aplicación móvil, etc.)
- Presentación de datos mediante **APIs** (ejemplo, mediante API REST).

## Capa de Dominio/Negocio

Es la capa encargada de modelar las reglas de negocio, las entidades del dominio.

Contiene la parte estructural y de comportamiento que brinda sostén a todos los posibles Casos de Uso del Sistema.

## Capa de Datos

Es la capa encargada de la manipulación de la Persistencia de los datos del Sistema (“guardado de los datos”). Si existe persistencia en una base de datos, esta capa es la encargada de comunicarse con ella.

## Capa de controladores

Se encuentra entre el dominio y la presentación. En esta capa se suelen agregar métodos que “orquestan” las funcionalidades; además de verificar si el usuario que está intentando realizar dicha acción posee los permisos necesarios.

En algunos Sistemas se toma la decisión de delegar en otra capa la verificación de Permisos (una capa llamada “Middleware”).

### + Ventajas

- Nos mantiene enfocados en el problema a resolver.
- Esconde el detalle de cómo se llevan a cabo los servicios que expone.
- Puede ser reemplazada la implementación de los servicios siendo transparente para los consumidores.
- Minimiza la dependencia entre componentes.
- Facilita las pruebas.

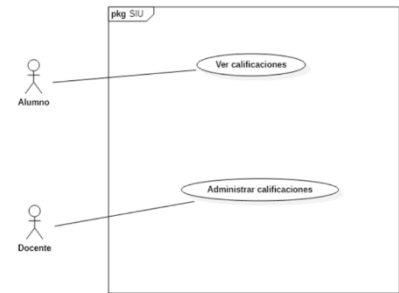
### - Desventajas

- Los cambios pueden generar efecto cascada.
- Demasiadas capas agregan complejidad y afectan negativamente al rendimiento.

## Orquestación de CU

Existe un diseño incorrecto al mezclar la lógica de negocio con los hipotéticos permisos que tiene un actor/rol en particular sobre nuestro Sistema. Basados en los requerimientos:

- El Sistema debe permitir que los alumnos visualicen sus calificaciones
- El Sistema debe permitir la administración (alta, baja y modificación) de calificaciones por parte de los docentes.



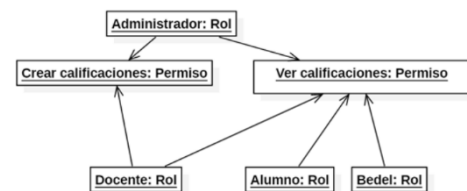
Sería incorrecto plantear:

1. Una clase Alumno con un método “verCalificaciones”: ¿qué haría ese método?
2. Una clase Docente con un método “administrarCalificaciones”: nuevamente, ¿qué haría ese método?

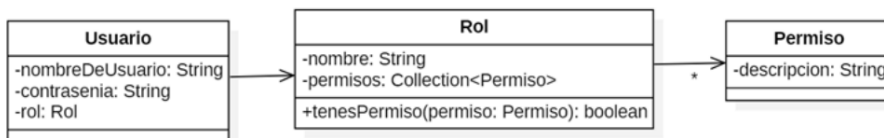
Llamaremos “Usuario” a aquella entidad que contiene los datos de Acceso para que una persona física u otro Sistema pueda identificarse en nuestro Sistema al usarlo.

Una misma persona física o Sistema podría cumplir varios roles dentro de nuestro Sistema y ejecutar diferentes casos de uso.

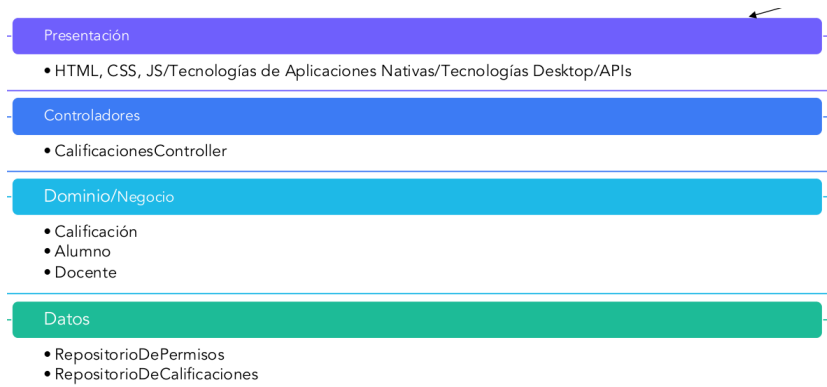
Distintos roles podrían tener la facultad de ejecutar el mismo caso de uso:



- Cada rol podría ejecutar muchos casos de uso.
- Para ejecutar un caso de uso se necesita tener uno o varios permisos.
- Un mismo permiso podría ser adjudicado a varios roles distintos.



- Los permisos podrían estar modelados con enumerados, entendiendo que esto trae aparejado ciertas limitaciones.
- Los usuarios podrían tener múltiples roles (colección de roles).

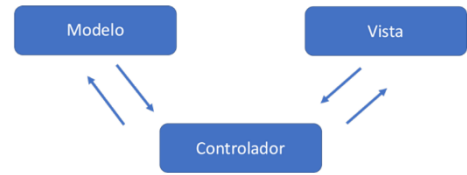




# MVC

Es un patrón de arquitectura de software de interacción que separa la arquitectura en tres componentes: el modelo, la vista y el controlador.

Vinculado con la representación de los datos con los cuales el Sistema opera. Cada capa valida la lógica y las reglas del negocio para enviar los datos solicitados a la vista.



## + Ventajas:

- Encapsula el estado del sistema y los concerns.
- Gestiona todos los accesos a dichos datos (consultas, actualizaciones, etc.)
- Flexibilidad y reusabilidad de vistas y controladores.

## - Desventajas:

- Mayor complejidad de los sistemas.
- No siempre útil en aplicaciones con “vistas simples” o con “poca interactividad”.
- Difícil de testear “como un todo”

## Controlador

Responde a eventos (usualmente acciones del usuario) e invoca peticiones al modelo cuando se hace alguna solicitud sobre los datos.

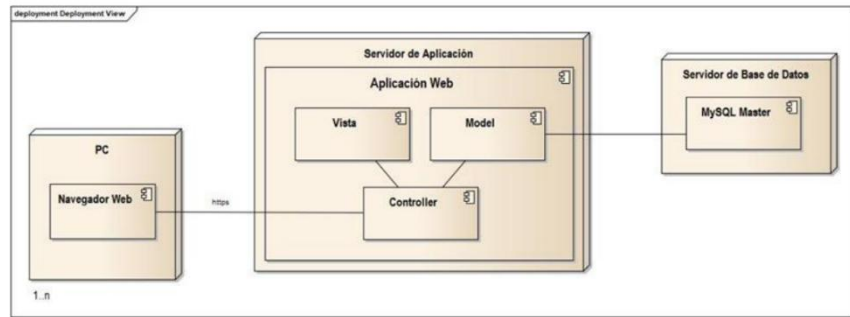
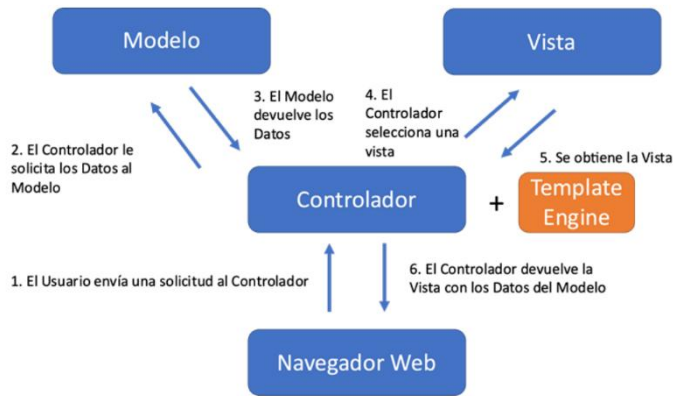
- Es el intermediario entre el modelo y la vista
- Define el comportamiento del sistema
- Traduce acciones del usuario a actualizaciones del modelo
- Es el gestor del ciclo de vida del sistema
- Responde a eventos
- Invoca peticiones al modelo

## Vista

Presenta los datos y su forma de interactuar en un formato adecuado para el usuario.

- Posee lógica para poder representar los datos de la forma más “amigable” para el usuario
- Envía acciones del usuario al controlador
- Solicita actualizaciones al modelo a través del controller
- La comunicación entre la vista y el controlador se realiza mediante objetos de transferencia (DTO – Data Transfer Objects).

## Web

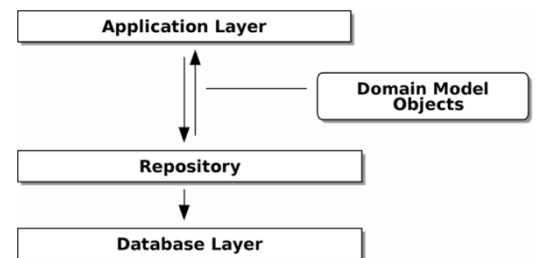


## Validación de Datos

- Hay una validación en profundidad, o sea, se validan en cada capa.
- En la vista se puede validar campos requeridos y consistencia (números y/o letras donde corresponda, entre otras cosas).
- En el controlador se debería volver a validar la consistencia de los datos y los campos requeridos. También se puede realizar una validación adicional contra un servicio externo.
- En el modelo se deberían realizar las validaciones explícitas del negocio.

## Patrón repository

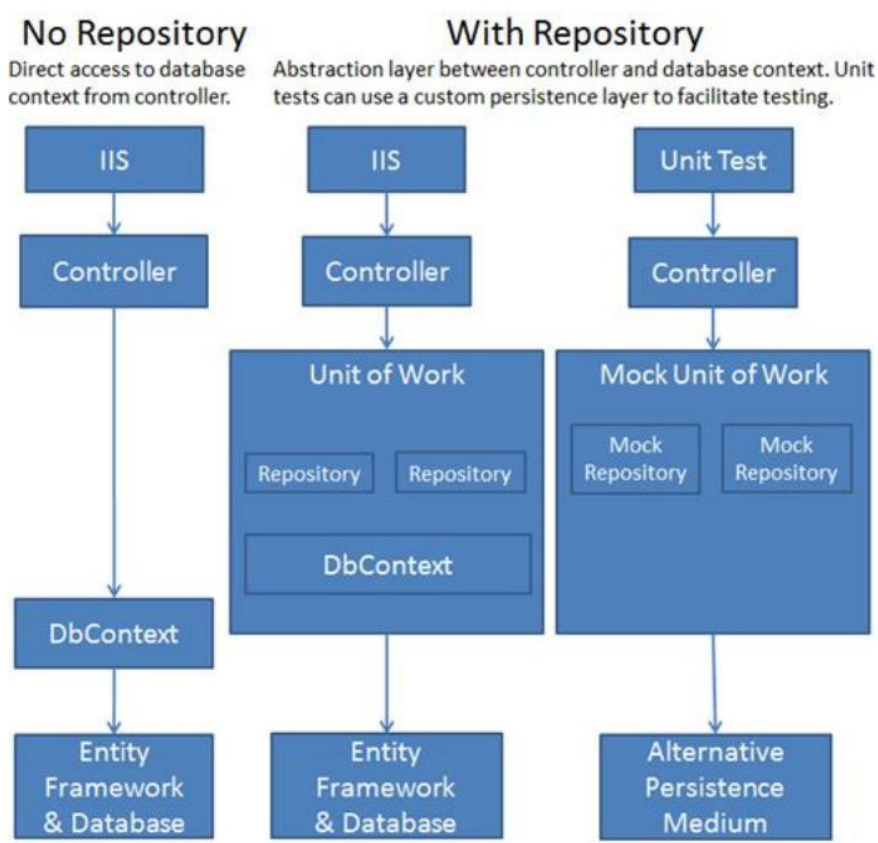
Es un **patrón arquitectónico** que nos ayuda a estructurar el aplicativo para lograr una buena separación de concerns y se apoya sobre el estilo en capas para estructurar el aplicativo. Esta propone **una capa de persistencia** encargada del **acceso a los datos**.



Propone que en la capa de persistencia existan objetos “Repository”.

Donde cada repositorio debería poder:

- agregar(unObjeto)
- modificar(unObjeto)
- eliminar(unObjeto)
- buscar(unObjeto)
- buscarTodos(objetos)

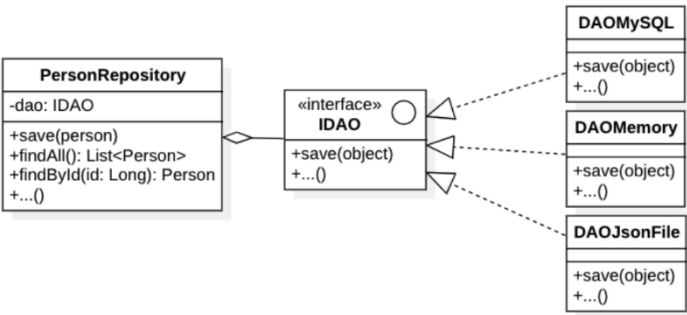


### DAO (Data Access Object)

Son los responsables reales de acceder a los datos. Puede acceder a los datos necesarios y se debe usar en combinación del patrón repository.

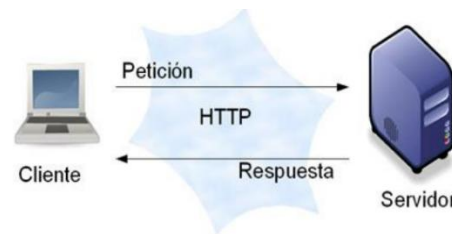
Donde cada repositorio puede delegar su responsabilidad en un DAO concreto.

Los clientes de los repositorios no deberían enterarse cual es el medio persistente, es indistinto el DAO concreto usado.



# Cliente – Servidor

Modelo de comunicación en el que un cliente recursos o servicios y un servidor los provee a una red (HTTP/HTTPS). En el que participan 2 componentes:



solicita  
través de

- Servidor: provee uno o más servicios a través de una interfaz.
  - Procesa las peticiones y devuelve la respuesta.
  - Se enfoca en la lógica y datos.
- Cliente: usa esos servicios como parte de su operación en el acceso al servidor.
  - Puede ser una aplicación web, app móvil, navegador, etc.
  - Se enfoca en la interfaz.

## Clasificación según responsabilidades

| Tipo cliente | Tipo servidor | Responsabilidad Cliente y Servidor                                                               |
|--------------|---------------|--------------------------------------------------------------------------------------------------|
| Activo       | Pasivo        | C: Mayor lógica de negocio.<br>S: Limita su funcionalidad a la persistencia.                     |
| Pasivo       | Pasivo        | Baja lógica de negocio en ambos. Son componentes intermedios de algo más grande.                 |
| Pasivo       | Activo        | C: presenta los datos.<br>S: mayor lógica de negocio.                                            |
| Activo       | Activo        | Lógica de negocio entre ambos componentes. El cliente posee lógica de presentación de los datos. |

### *Analizando Cliente Pasivo – Servidor Activo...*

#### *+ Ventajas*

- Mantenibilidad: Cambios de funcionalidad Centralizados
- Seguridad: Centralización de Control de Accesos a recursos

#### *- Desventajas (considerando único servidor)*

- Eficiencia (tiempo de respuesta): El servidor puede ser un cuello de botella
- Disponibilidad: Único punto de falla

## Protocolo HTTP

HyperText Transfer Protocol es un protocolo de comunicación basado en el modelo Cliente-Servidor que define cómo se deben intercambiar datos en la web. Es un protocolo de aplicación dentro de la pila de protocolos de Internet y funciona sobre TCP.

- HTTP: transmite datos en texto plano (Por el puerto 80).
- HTTPS: utiliza TLS/SSL para cifrar los datos, proporcionando confidencialidad e integridad en la comunicación. (Por el puerto 443)

## Uso básico

1. Cuando un cliente (normalmente un navegador) necesita acceder a un recurso en la web, envía una solicitud HTTP (HTTP request) a un servidor web.
2. Este servidor procesa la solicitud y responde con una respuesta HTTP que contiene los datos solicitados o un código de estado que indica el resultado de la solicitud.

## Operaciones CRUD

- GET → Solicita un recurso sin modificarlo.
- POST → Envía datos al servidor para que sean procesados.
- PUT → Actualiza o crea un recurso en el servidor.
- DELETE → Elimina un recurso del servidor.
- PATCH/UPDATE → Modifica parcialmente un recurso.

## HTTP – Código de Estado

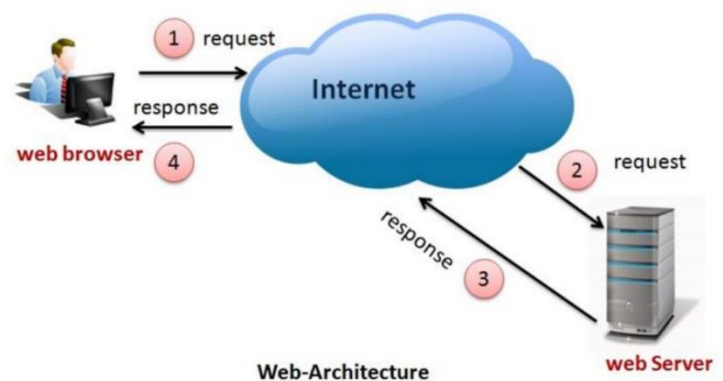
- **1xx:** Respuestas informativas. Indica que la petición ha sido recibida y se está procesando.
- **2xx:** Respuestas correctas. Indica que la petición ha sido procesada correctamente.
- **3xx:** Respuestas de redirección. Indica que el cliente necesita realizar más acciones para finalizar la petición.
- **4xx:** Errores causados por el cliente. Indica que ha habido un error en el procesamiento de la petición a causa de que el cliente ha hecho algo mal.
- **5xx:** Errores causados por el servidor. Indica que ha habido un error en el procesamiento de la petición a causa de un fallo en el servidor.

## Arquitectura Web

La Arquitectura Web es el conjunto de principios, tecnologías y estructuras que definen cómo se organizan y comunican los componentes de una aplicación web.

También lo podemos definir como el conjunto de componentes que forman una aplicación accesible mediante un navegador web.

Se basa en la interacción entre clientes y servidores a través de protocolos como HTTP/HTTPS, permitiendo la distribución de información y la ejecución de aplicaciones en la web.



## API REST

### ¿Qué es una API?

Una interfaz de programación de aplicaciones (API) es un conjunto de herramientas, definiciones y protocolos que se usan para diseñar e integrar aplicaciones.

Permite que un producto o servicio se comuniquen con otros productos y servicios, sin la necesidad de saber cómo se implementan internamente.

## Tipos de API

- **Arquitectura**
  - **Local:** Proporciona Servicios de Sistema Operativo, Bibliotecas o Middleware.
  - **Web:** Proporciona Servicios entre Componentes Distribuidos y en general sobre el protocolo HTTP.
  - **Programa:** proporciona la posibilidad de Ejecución de Código de forma Remota entre Componentes Distribuidos (RPC).
- **Alcance**
  - Pública
  - Partner (para terceros)
  - Private (interna)

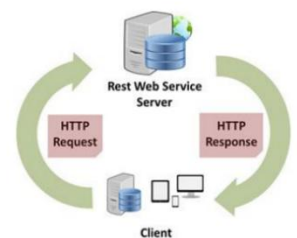
## Ejemplos

- **API Stream** de Java, que brinda un conjunto de métodos para poder manipular un Stream de Datos.
- **JPA** (Java Persistence API), que brinda un estándar para el Mapeo Objeto - Relacional
- **JDBC** (Java Database Connectivity), proporciona clases e interfaces para interactuar con bases de datos relacionales.
- **Win32 API (Windows)**, proporciona funciones y estructuras de datos para interactuar con los recursos del Sistema, como ventanas, archivos, procesos, memoria, I/O, etc.
- **POSIX API (Linux)**, proporciona funciones para realizar operaciones de bajo nivel, como la gestión de archivos, manejo de procesos, concurrencia, etc.

## API REST

Representational State Transfer es un enfoque de diseño de comunicación de Arquitectura a la hora de realizar una comunicación entre cliente y servidor.

- Todos los objetos se manipulan mediante URI (identificador uniforme de recursos)
- Utiliza como formato de intercambio JSON / XML



## Recursos Web REST

Cuando diseñamos un Sistema con arquitectura Web debemos pensar en qué recursos vamos a permitir manipular. Para poder identificar qué recursos debe exponer un Sistema se deben tener en cuenta los casos de uso.

Para la manipulación de datos consideramos las operaciones CRUD: GET, POST, PUT, UPDATE, DELETE

Las rutas deben llevar nombres de recursos, usando solo sustantivos y no verbos.

## Patrones de Ruta

| Característica | Path Param                  | Query Param                         |
|----------------|-----------------------------|-------------------------------------|
| Propósito      | Identificar recursos únicos | Modificar una consulta              |
| Forma          | /api/usuarios/123           | /api/usuarios?page=2&país=Argentina |
| Usos típicos   | ID de un recurso            | Filtros, orden paginación           |

## Query Params

### Paginación

La paginación de una API REST es una técnica que permite dividir grandes cantidades de datos en partes más pequeñas (llamadas páginas), para evitar sobrecargar al servidor o al cliente con demasiada información en una sola respuesta.

#### + Ventajas:

- Mejora el rendimiento y reduce el uso de memoria.
- Acelera el tiempo de respuesta.
- Facilita la navegación entre conjuntos grandes de datos (usuarios, productos, etc.)

#### Ejemplo:

- 🚦 Base: GET /api/usuarios
- 🚦 Modificada: GET /api/usuarios?page=2&limite=5

### Filtros

| Operador | Descripción         | Ejemplo                         |
|----------|---------------------|---------------------------------|
| =        | Igual               | ?edad=30                        |
| >        | Mayor que           | ?edad_gt=30                     |
| <        | Menor que           | ?edad_lt=30                     |
| in       | Dentro de una lista | ?pais_in=Argentina,Brasil,Chile |
| like     | Búsqueda parcial    | ?nombre_like=Luc                |

#### Ejemplo:

Base: GET /api/usuarios

Modificada: GET /api/usuarios?edad=30&pais =Argentina

## Versionado de APIs

El versionado de APIs REST es una práctica clave para mantener la compatibilidad a largo plazo, especialmente cuando introduces cambios que pueden afectar a los consumidores existentes de la API (como modificar respuestas, renombrar campos, eliminar Endpoint, etc.).

- Permite evolucionar la API sin romper a los clientes que usan versiones anteriores.
- Facilita el desarrollo paralelo entre equipos.
- Da a los consumidores control sobre cuándo migrar a una nueva versión.

## Tipos de versionados

- Versión en la URL (la más común)
  - GET /api/v1/usuarios
- Versión en el encabezado (header-based)
  - GET /api/usuarios
  - Accept: application/vnd.miapi.v1+json
- Versión como parámetro de consulta (menos recomendado)
  - GET /api/usuarios?version=1

## ¿Qué es el Rate Limiting?

Rate Limiting es una técnica que restringe la cantidad de peticiones que un cliente puede hacer a una API en un determinado período de tiempo.

### + Ventajas:

- Proteger tu servidor de sobrecargas (ya sea por mal uso o ataques).
- Garantizar equidad entre los usuarios.
- Controlar costos en APIs comerciales o servicios en la nube.

## ¿Qué es Throttling?

Throttling es la acción de imponer el rate limit cuando este se supera.

| Tipo de Throttling | Descripción                                                       |
|--------------------|-------------------------------------------------------------------|
| Soft               | Ralentiza la respuesta (delay artificial)                         |
| Hard               | Rechaza directamente con error (429 Too many requests)            |
| Dynamic            | Cambia el límite según la carga del sistema o el plan del usuario |

## Interfaces entrantes y salientes

Sabiendo que un Sistema es un conjunto de Componentes que se relacionan para cumplir un objetivo en común; y teniendo en cuenta que el componente A utiliza al componente B para cumplir ese objetivo...

¿Cómo se relacionan dichos componentes?



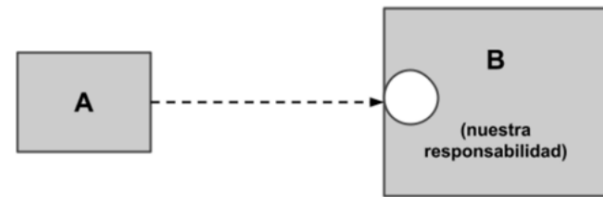
### Consideraciones al momento de Diseñar:

- Dirección y sentido de la comunicación
  - A llama a B
  - B llama a A
  - La comunicación es bidireccional
- Ambiente en el que residen los componentes
  - Los componentes están en el mismo ambiente
  - Los componentes están en ambientes separados
- Sincronismo/Asincronismo en peticiones



## Interface Entrante

La interface entrante es aquel contrato que define qué **datos** necesita **nuestro componente** (componente B en el gráfico) para poder realizar determinada tarea/ejecutar cierta funcionalidad. Se debe tener en cuenta cómo utilizarán los terceros nuestro componente.



## Interface Saliente

La interface saliente es aquel contrato que define qué datos necesita el componente al cual nos queremos integrar (componente B en el gráfico) para poder realizar determinada tarea/ejecutar cierta funcionalidad. En este caso, se considera que nuestro componente A está “consumiendo” al componente B.



## Entorno de ejecución

“Es un contexto que refiere a hardware y software donde se ejecuta una aplicación”.

Es importante destacar que no necesariamente todos los componentes de un aplicativo/Sistema ejecutan en el mismo entorno.

### Posibles entornos:

- En el mismo Sistema – mismo entorno
  - Se debe pensar en qué mecanismo de integración es el adecuado para la comunicación entre ambos componentes.
  - Se debe prestar atención al grado de acoplamiento que se podría generar entre ambos componentes, reduciendo el mismo al máximo posible.
  - Se debe evitar la ruptura del encapsulamiento del componente integrado.
  - Se debe diseñar una integración Testeable.
- En el mismo Sistema – diferente entorno
  - Debemos pensar cuál será el mecanismo para que ambos componentes se comuniquen, en el caso que no estén definidas las interfaces.
  - Si la interfaz del componente ya está definida, debemos integrarnos de la forma en que lo exige.
- En Sistemas diferentes – diferentes entornos

## Cuestiones de Sincronismo

### Integraciones síncronas

Cuando un componente A hace uso de alguna funcionalidad expuesta por el componente B y se queda esperando la respuesta, entonces A está integrándose de forma sincrónica con B para la utilización de dicha funcionalidad.

#### + Ventajas:

- ✓ Es la forma más simple de trabajar porque es fácil de razonar.
- ✓ Es la forma en la que trabaja el envío de mensajes entre objetos por defecto.

#### - Desventajas:

- Estamos penalizando a todo el resto del Sistema por tan solo esperar una respuesta

## Integraciones asíncronas

Cuando un componente A hace uso de alguna funcionalidad expuesta por el componente B y no se queda esperando la respuesta, entonces A está integrándose de forma asincrónica con B para la utilización de dicha funcionalidad.

#### + Ventajas:

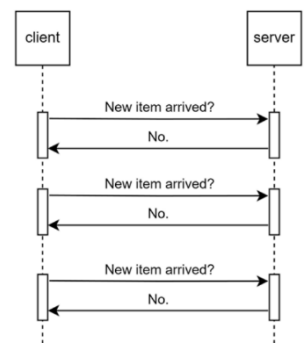
- ✓ No es la forma en la que estamos acostumbrados a pensar el común de las cosas.
- ✓ Es una buena manera de no penalizar toda la ejecución del Sistema.

El componente A puede no quedarse esperando la respuesta de B por alguno de los siguientes motivos:

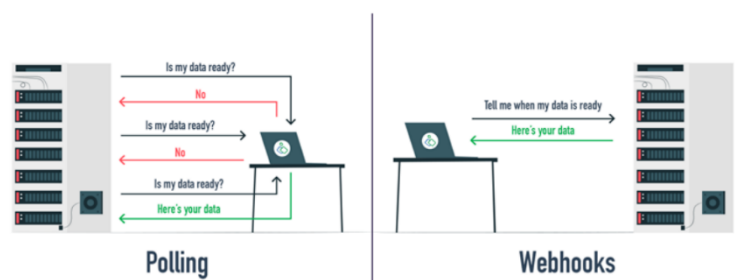
- No necesita el resultado/la respuesta en ese momento para continuar con la ejecución.
- No le interesa el resultado/la respuesta.
- No le estaba pidiendo nada, sino que le estaba “avisando de algo”.

## Tipos de respuestas

1. Preguntarle a B “cuál es el estado de la ejecución” o “si ya terminó”
  - a. Ante una mala utilización de este mecanismo podríamos generar una espera activa en A.
  - b. A esta forma se la conoce como “pull based”
2. Leer periódicamente algún espacio de memoria compartido con B, en el cual B depositará el resultado.
  - a. Nuevamente, ante una mala utilización de este mecanismo podríamos generar una espera activa en A.
3. Pasarse como parámetro para que B le avise cuando haya terminado (si A y B están en el mismo ambiente): “push based”.
4. Enviarle a B un callback para que ejecute luego de haber terminado la ejecución
  - a. Conocido como “Webhook”, en el caso de que ambos componentes estén bajo una Arquitectura Web, también: “push based”



## Polling vs Webhooks



## Cron Task

Un CronJob es una tarea programada que se ejecuta de manera periódica y automática en Sistemas Unix-like (como Linux o MacOS). Se utiliza para automatizar comandos o scripts en intervalos de tiempo definidos por el usuario.

| Requerimiento                                                                                                 | Acción a realizar | Frecuencia       | Evento disparador  |
|---------------------------------------------------------------------------------------------------------------|-------------------|------------------|--------------------|
| <i>"Todas las noches el Sistema debe enviar un email a los clientes para (...)"</i>                           | Enviar email      | Todas las noches | El paso del tiempo |
| <i>"Cada semana el Sistema debe analizar los gastos del cliente y generar un reporte (...)"</i>               | Generar reporte   | 1 vez por semana | El paso del tiempo |
| <i>"Todos los jueves el Sistema debe enviar un voucher de descuento en hamburguesas a los empleados(...)"</i> | Enviar voucher    | 1 vez por semana | El paso del tiempo |

### Elementos de un CronJob

Un CronJob se define en una línea dentro de un archivo llamado "crontab" y tiene la siguiente estructura:

```
* * * * * comando
| | | | |
| | | | +----- Día de La semana (0 - 7) (Domingo puede ser 0 o 7)
| | | +----- Mes (1 - 12)
| | +----- Día del mes (1 - 31)
| +----- Hora (0 - 23)
+----- Minuto (0 - 59)
```

## Solicitud – Respuesta

Es un estilo de interacción sincrónica clásico en la arquitectura de software, donde un cliente envía una solicitud a un servidor y espera una respuesta.

Tiene limitaciones con recibir notificaciones automáticas o manejar cientos de clientes a la vez.

*Cliente Servidor es una implementación concreta con sus limitaciones:*

- **Acoplamiento fuerte:** Cliente y servidor deben conocerse mutuamente y estar activos al mismo tiempo.
- **Dificultad con flujos asíncronos:** El modelo está pensado para peticiones y respuestas inmediatas, no para eventos espontáneos.
- **No orientado a eventos:** El servidor no "emite" eventos espontáneamente. El cliente siempre tiene que preguntar (polling).
- **No ideal para arquitecturas reactivas o distribuidas modernas:** Como IoT, microservicios o sistemas orientados a eventos.

## Subscripción de eventos

Patrón donde un sujeto (emisor) notifica a múltiples observadores (suscriptores) cuando ocurre un evento. Los observadores se registran recibir notificaciones. Se puede utilizar en interfaces gráficas, flujos de datos, y programación reactiva.



para

### + Características:

- Desacoplamiento leve: el emisor no necesita saber quién lo escucha.
- Notificación automática: sin necesidad de "polling".
- Uno a muchos: un evento puede activar múltiples respuestas.

### - Limitaciones:

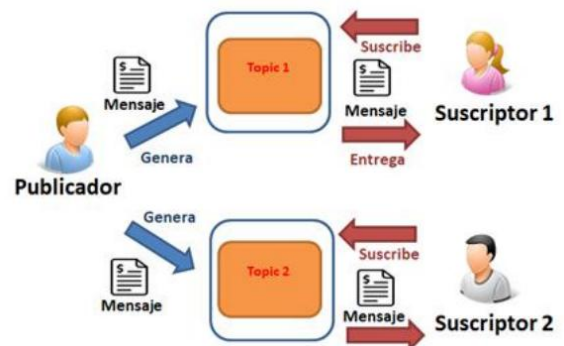
- Aún existe acoplamiento entre el emisor y sus suscriptores (in execution time).
- No escala bien si se necesita persistencia o distribución entre procesos o redes.

## Patrón Publicación – Subscripción

Es un Patrón de Mensajería que define un Subscriptor y un Publicador con diferentes tópicos, minimizando el acoplamiento: al no conocerse entre sí.

### Características:

- Desacoplamiento total: los productores y consumidores no necesitan conocerse.
- Escalabilidad: múltiples publishers/subscribers por tópico.
- Distribución natural: ideal para sistemas distribuidos y microservicios.
- Asíncronía: los mensajes se manejan sin bloquear al emisor.
- Extensibilidad: fácil agregar nuevos consumidores sin modificar el productor.



### Limitaciones:

- Sin persistencia de mensajes (si un sub no está presente, puede perder datos).
- Sin control de calidad de servicio, orden, o seguridad.

### Casos de uso

- Notificaciones push móviles.
- Procesamiento de eventos de usuarios.
- Log de auditoría.
- Comunicación entre microservicios.

### ¿Cómo se puede implementar?

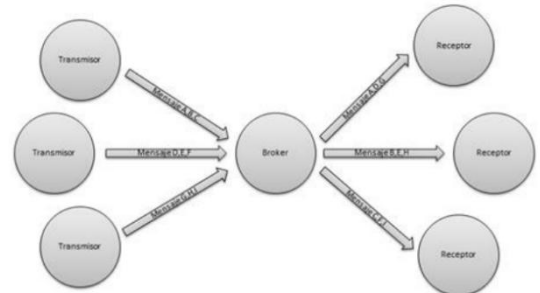
- A nivel objetos: patrón observer
- A nivel arquitectura: con un broker

## Bonus: Websockets

- Es un protocolo (RFC 6455) que permite mantener una conexión persistente bidireccional entre cliente y servidor.
- A diferencia de HTTP, que es request-response, WebSocket permite que el servidor envíe datos al cliente en cualquier momento.
- Ideal para Sistemas de chat, Juegos en línea, Dashboards de tiempo real, Alertas/notificaciones en tiempo real

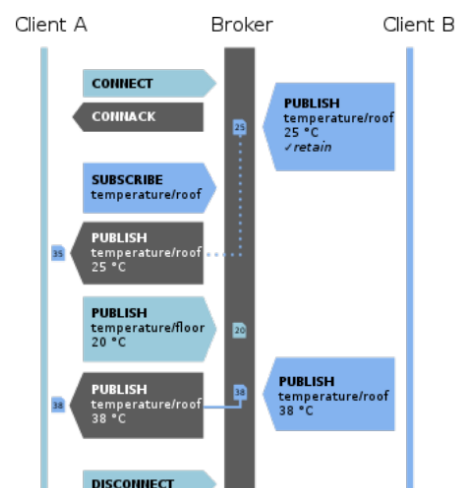
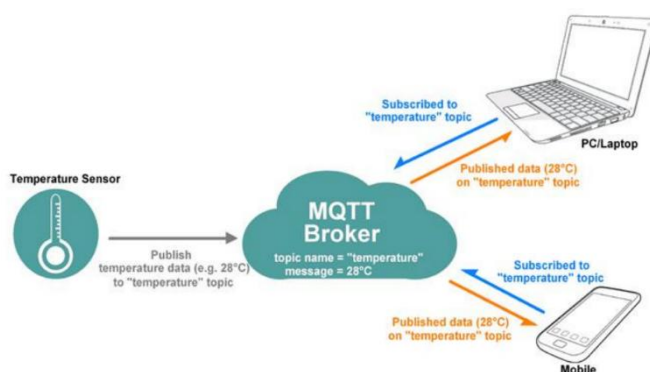
## Patrón Broker

Es un intermediario que implementa Publicación / Suscripción con servicios de autenticación que resuelve problemas de firewall e iguala a los participantes.



| Característica            | Descripción                                                          |
|---------------------------|----------------------------------------------------------------------|
| Desacoplamiento total     | Los publishers y subscribers ni siquiera se conocen.                 |
| Persistencia              | El Broker puede guardar mensajes si el subscriber no está conectado. |
| Escalabilidad             | Maneja miles o millones de mensajes y nodos.                         |
| Enrutamiento inteligente  | Distribuye mensajes según tópicos, prioridades o filtros.            |
| QoS (Calidad de servicio) | Reintentos, entregas garantizadas, control de duplicados.            |
| Seguridad                 | Autenticación, autorización, cifrado.                                |
| Monitorización            | Logs, métricas, herramientas de análisis en tiempo real.             |

| Broker        | Persistencia     | Escalabilidad | Casos típicos                      |
|---------------|------------------|---------------|------------------------------------|
| MQTT          | No (por default) | Muy alta      | IoT, sensores, sistemas embebidos. |
| Kafka         | Sí               | Altísima      | Microservicios, análisis de logs.  |
| RabbitMQ      | Opcional         | Media-alta    | Sistemas de colas, Jobs delayed.   |
| Redis Pub/Sub | No               | Alta          | Sistemas simples o en memoria      |



# Persistencia

Refiere a lo que sobrevive más allá de una única ejecución. En la práctica, almacenando dicho estado en una base de datos o archivos.

## Estrategias de Persistencia

### Persistencia en Memoria

Es la capacidad de un dato u objeto de seguir existiendo luego de ser utilizado en distintas operaciones. Usado para los tests en los sistemas.

### Persistencia en Archivos

Involucra guardar el estado de un Sistema en uno o varios Archivos para que luego éste pueda ser recuperado, y el sistema pueda continuar funcionando/ejecutando desde el mismo punto, es decir, como “si nada hubiera pasado”

Existen tipos de formatos de intercambio: XML, CSB, JSON, Ancho fijo, etc.

### Prevalencia

Es una técnica que almacena el Estado de un Sistema en la MP, pero que regularmente genera “snapshots” a disco para evitar la pérdida completa de los datos.

Las implementaciones de esta técnica tienen la capacidad de manejar transacciones.

+ **Ventajas:** no existen transformaciones de datos -> alta transparencia y performance.

- **Desventajas:** falta de interoperabilidad de los datos y una alta capacidad de memoria.

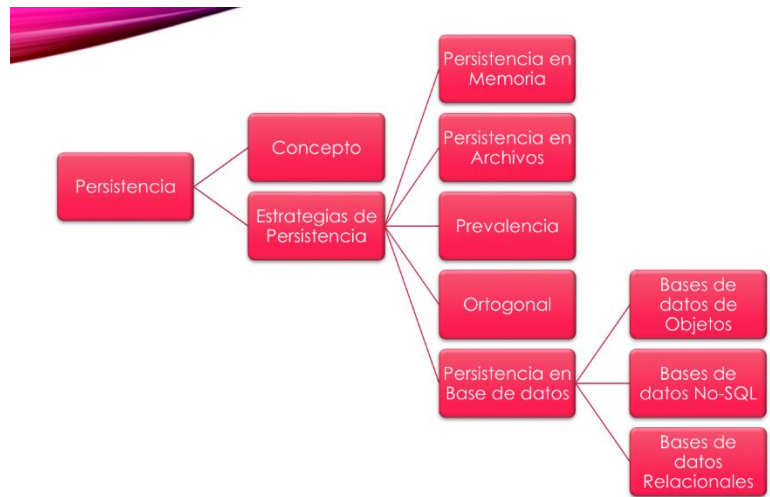
Implementación: [Prevayler](#) en java

### Ortogonal

Refiere a que la Persistencia del Estado de un Sistema se implementa como una propiedad intrínseca de su entorno de ejecución.

Por este motivo, no se requieren acciones específicas para poder guardar o recuperar datos, ya que esto ocurre de forma “transparente”

Implementaciones: hibernate mode y virtualización en SOs.



## Persistencia en Base de datos

Requieren de una transformación previa para ser persistidos.

### + Ventajas:

- Datos se persisten de forma independiente a los sistemas que los manipulan (interoperabilidad)
- Hay mecanismos de recuperación (integridad de datos)

## Base de datos de Objetos

- La principal desventaja radica en la interoperabilidad de los datos.

## Base de datos de relacionales (SQL)

Cumple con el Modelo relacional donde todos los datos se almacenan y se acceden por medio de las relaciones previamente establecidas.

Clave Primaria (PK): atributo que identifica a cada fila de la tabla

- Naturales: CUIT, celular, DNI, ID
- Subrogadas: clave ficticia
- Compuestas

Clave foránea (FK): un grupo de una o más columnas en una tabla que referencian a la PK de otra tabla. Una FK puede ser parte de una PK.

Índices: estructura que mejora la velocidad de las operaciones con un identificador único de cada fila de una tabla. Tanto PK como FK son índices

Integridad Referencial: es una propiedad que establece que la clave externa de una tabla de referencia siempre debe corresponder a una fila válida de la tabla a la que se haga referencia.

Garantiza que la relación entre dos tablas permanezca sincronizada durante las operaciones de actualización y eliminación.

Tipos de datos más usados:

- VARCHAR: cadenas de chars.
- INTEGER: números enteros naturales
- DOUBLE: valores numéricos con punto flotante.
- DATETIME: formato para manejo de fechas.

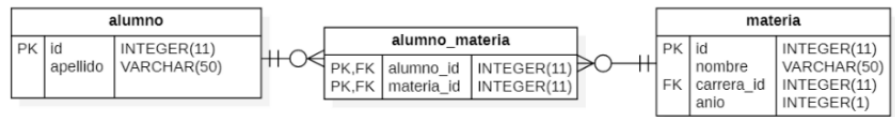
Constraints: restricciones al modelo que se usan para limitar el tipo de dato que puede ingresarse en una tabla. Se especifican al crear o actualizar la tabla.

- Not null: no acepta valores nulos.
- Unique: no acepta valores repetidos.
- Check (o enum): verifica que los valores cumplan determinada condición.
- Primary Key



- Foreign Key

Relaciones: asociaciones entre tablas que describen la estructura de la BD y como se vinculan.



- Cardinalidad: cantidad máxima de relaciones que tiene un registro de una tabla con otra.
- Modalidad: cantidad mínima de relaciones que tiene un registro de una tabla con otra.
- One to One: entidad A tiene una entidad B
- One to Many: entidad A tiene muchas entidades B
- Many to One: entidad A pertenece a muchas entidades B
- Many to Many: A tiene muchos B, y B tiene muchos A
  - Requiere de tabla intermedia

## Modelo Relacional

Relación: es la “entidad” donde se almacenan los datos.

Tupla: es el registro.

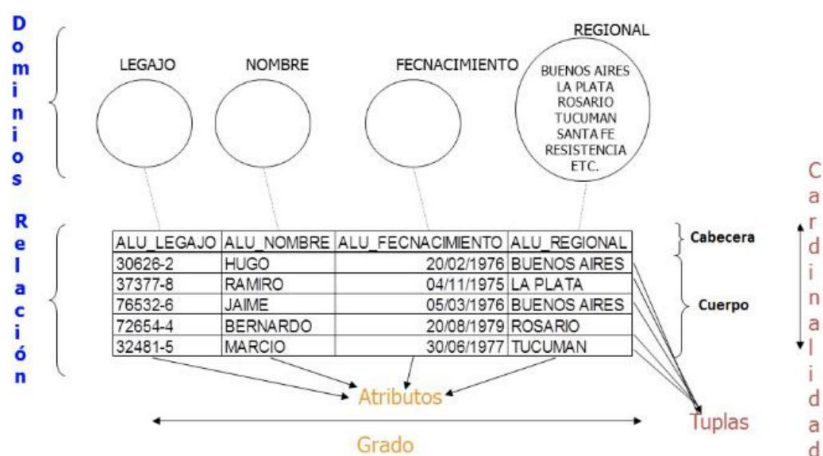
Atributo: es un campo dentro de la tupla.

Dominio: es el conjunto de valores posibles que puede tomar un Atributo. Es la menor unidad semántica de información.

Cardinalidad: número de tuplas.

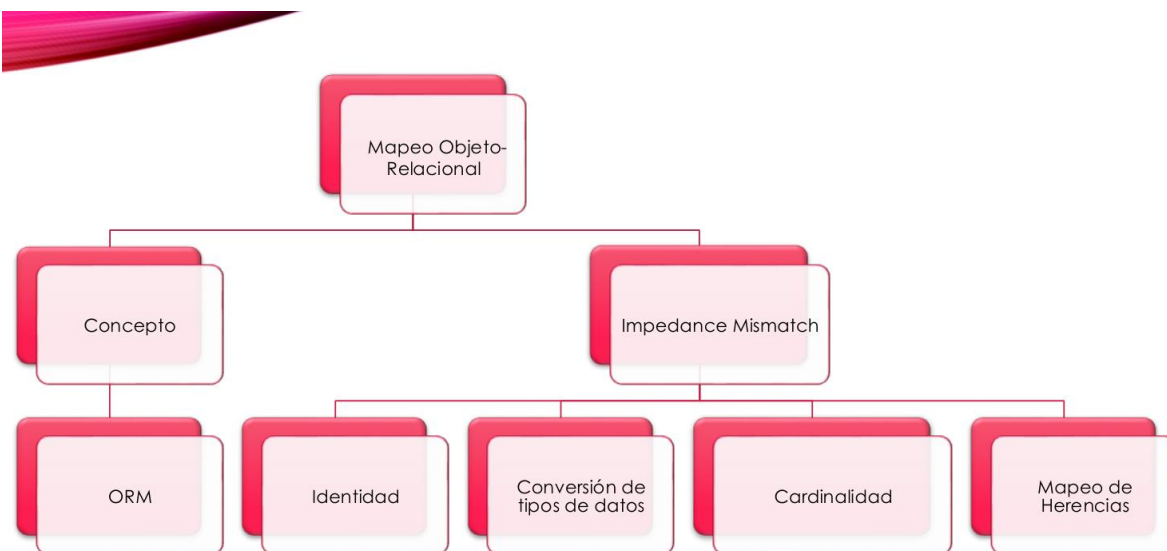
Grado: número de atributos.

Clave primaria: es el identificador único de cada tupla.





## Mapeo Objeto – Relacional



El Mapeo Objeto-Relacional es una técnica usada para convertir los tipos de datos con los que trabaja un lenguaje orientado a objetos a tipos de dato con los que trabaja un sistema de base de datos relacional.

Mientras que ORM es el nombre de la técnica, también se suele conocer como ORMs a los frameworks que la implementan.

Al tratar de encontrar una correspondencia entre estos “mundos”, sucede que existen características que son difíciles de imitar.

A este “desajuste” se lo conoce como **Impedance Mismatch**

### Impedance Mismatch – Identidad

- Un objeto cumple con la característica de Unicidad y un registro necesita una identificación unívoca.

### Impedance Mismatch – Conversión de tipos de datos

- String: sin limitaciones de tamaño vs. El VARCHAR/CHAR con longitud definida.
- Campos numéricos respecto a la precisión de decimales, o el rango de valores máximos y mínimos; incluso cuando el dominio es “un número de 0 a 3000”
- Las fechas tiene tipos no siempre compatibles entre sí.
- Booleanos
- Enumerados

### Impedance Mismatch – Manejo de cardinalidad

- En el POO, las relaciones pueden ser unidireccionales y bidireccionales; con navegabilidad bidireccional.
- Las relaciones entre las entidades de una base de datos relacional no suelen ser bidireccionales

## Impedance Mismatch – Mapeo de la herencia

- En el POO existe el mecanismo de herencia, el cual nos permite reutilizar lógica de una clase y/o extender su comportamiento; pero este concepto no existe en el mundo relacional.

Existen, al menos, cuatro estrategias de mapeo de herencia:

- **SINGLE TABLE:** Única tabla.
- **JOINED:** Una tabla por la superclase y una tabla por cada una de las clases hijas.
- **TABLE PER CLASS:** Una tabla por cada clase concreta.
- **MAPPED SUPERCLASS:** Los atributos de la superclase son persistidos en las tablas de las clases hijas.

### Mapeo de la herencia – Single Table

Tabla única que va a tener columnas por todos los atributos de la superclase y las clases hijas. Además, tendrá una columna de “campo discriminador” que indica a qué clase pertenece la fila en cuestión.

Si consideramos una superclase con N atributos persistentes; una clase hija A con M atributos persistentes; y otra clase hija B con P atributos persistentes; entonces:

- Si se persiste una instancia de la clase A (con todos sus atributos seteados, inclusive los pertenecientes a la superclase), quedarán P columnas nulas.
- Si se persiste una instancia de la clase B (con todos sus atributos seteados, inclusive los pertenecientes a la superclase), quedarán M columnas nulas.

+ **Ventajas:** puede haber muchas columnas nulas.

- **Desventajas:** buena performance for única tabla.

### Mapeo de la herencia – Joined

Genera una tabla para la super clase y N tablas más por cada clase hija.

La tabla de la superclass tendrá únicamente sus propios atributos y la opción de agregarle un campo discriminador. Las tablas de las clases hijas se les agregaran columnas por sus atributos correspondientes.

- Cada una de las PKs de las tablas que representan a las clases hijas, a su vez serán una FK a la tabla que representa a la superclase.
- Para recuperar un objeto con todos sus atributos (los propios + los que están en la superclase), el ORM debe joinear las tablas.

### Mapeo de la herencia – Tabla per Class

Suponiendo que existe una única superclase y N clases hijas concretas, el resultado de mapear la herencia con la estrategia Table per Class generará, en la base de datos, N tablas: una por cada clase hija.

Todos los atributos persistentes de la superclase serán persistidos en cada una de las tablas que mapean contra las clases hijas.

Para recuperar polimórficamente todos los objetos, el ORM debe realizar unions entre todas las tablas que mapean contra las clases hijas.

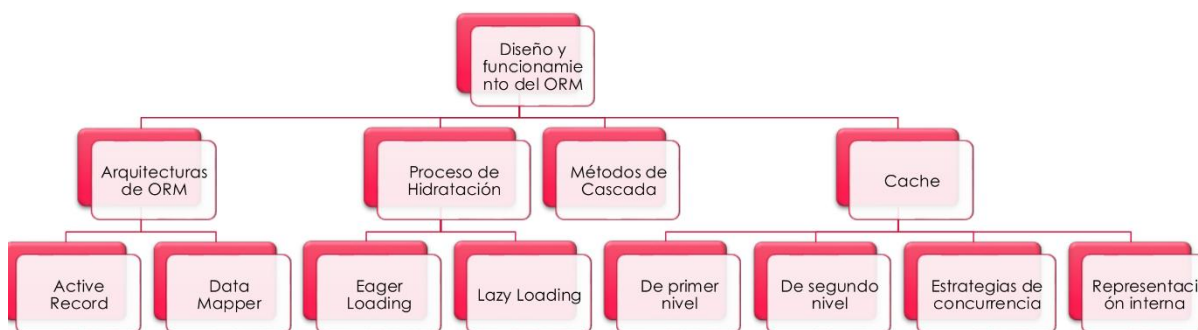
## Maapeo de la herencia – Mapped Superclass

Los atributos de la superclase son persistidos en las tablas de las clases hijas; y la superclase no es considerada una Entidad.

Generalmente utilizado cuando la superclase exhibe, únicamente, comportamiento y/o uno o “pocos” atributos en común.

Uno de los usos más comunes suele ser cuando todas las clases persistentes tienen una PK subrogada y/o un campo “activo” (booleano); pero entre ellas no existe nada más en común.

## Diseño y Funcionamiento del ORM



## Proceso de hidratación

Cuando el ORM recupera un objeto desde la base de datos ejecuta el proceso de Hidratación. Consiste en:

1. Instanciar la clase del objeto que se quiere recuperar (previamente habiendo recuperado los datos mínimos mediante la ejecución de una sentencia SQL).
2. Popular el objeto, es decir, asignarle a cada uno de sus atributos (aquellos no marcados como “lazy loading”) los valores recuperados desde la base de datos.

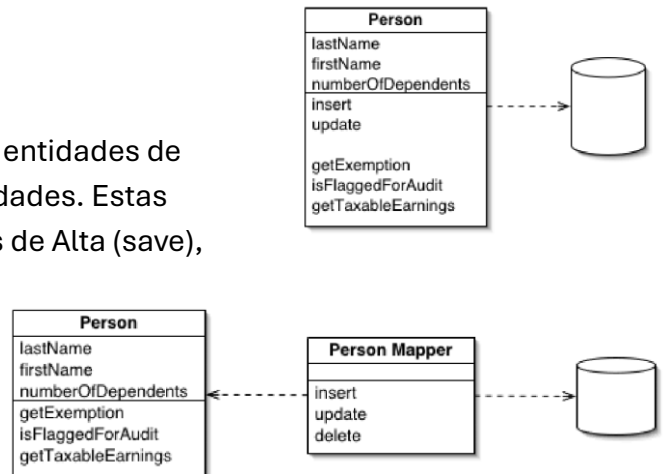
## Atributo marcado como LAZY vs EAGER

1. **Lazy Loading:** será populado por el ORM cuando sea llamado explícitamente.
2. **Eager Loading:** realiza la carga en memoria de los objetos independientemente de si van a ser utilizados o no.

## Arquitectura de ORM

### Active Record

En esta arquitectura los ORMs decoran las clases de entidades de dominio agregándoles funcionalidades/responsabilidades. Estas responsabilidades están relacionadas a las acciones de Alta (save), Baja (delete) y Modificación (update) de la entidad, así como también las funcionalidades para permitir la búsqueda de dicha entidad (find, findBy, findAll, entre otras).



### Data Mapper

Se agrega un Entity manager intermedio entre la Base de Datos y las entidades de dominio. Tiene la responsabilidad de buscar, agregar, modificar y eliminar.

## Métodos de Cascada

La cascada implica que, cuando realicemos alguna acción en la entidad objetivo, la misma acción se aplicará a la entidad asociada.

Tipos de Cascadas (todas no están disponibles en todas las ORMs):

- ALL: propagará todas las operaciones desde la entidad principal a la entidad secundaria.
- PERSIST: propagará la operación de persistencia de una entidad principal a una entidad secundaria. Se guardarán ambos.
- MERGE: propaga la operación de actualización. Ambos se actualizarán.
- REMOVE: propaga la operación de eliminación. Ambos se eliminarán.
- REFRESH: propaga la operación de refrescar todos los atributos de un objeto, repopula la entidad. Ambos se refrescarán.
- DETACH: propaga la operación “detach”, esta quita la entidad del contexto persistente. Ambos se quitarán del contexto persistente.

## CACHÉ

Los ORM tienen la capacidad de almacenar en caché. Lo que ayuda a reducir los costos de acceso: tiempo y recursos, al medio persistente.

### Caché de Primer Nivel

Permite que las sesiones concurrentes trabajen con instancias de forma aislada. Dura hasta que termine la sesión.

### Caché de Segundo Nivel

Aunque no disponible en todos los ORMs, tiene un alcance de SessionFactory (en Hibernate), lo que significa que es compartida por todas las sesiones creadas con la misma factory.

Cuando se busca una instancia de una entidad por su id y la cache de segundo nivel está habilitada, sucede alguno de los siguientes escenarios:

- Si ya está presente en la cache de primer nivel, es devuelta desde allí.
- Si no está presente en el primer nivel, pero sí en el segundo nivel, entonces se obtienen los datos desde allí, se hidrata el objeto y se devuelve la instancia.
- Si no se encuentra en ninguno de los anteriores casos, entonces se realiza el proceso de búsqueda e hidratación común y corriente.

## Estrategias de concurrencia

Existen estrategias de concurrencia de caché de segundo nivel:

- **READ\_ONLY:** se debería utilizar con entidades que nunca cambian. Muy adecuado
- **NONSTRICT\_READ\_WRITE:** la cache se actualiza después de que se haya confirmado una transacción que cambió los datos afectados. Existe una pequeña ventana de tiempo en la que se pueden obtener datos obsoletos desde la cache. Esta estrategia es adecuada si se puede tolerar una mínima inconsistencia eventual.
- **READ\_WRITE:** esta estrategia garantiza una fuerte consistencia que se logra mediante la utilización de bloqueos “suaves”. Cuando se actualiza una entidad en la cache, también se almacena un bloqueo suave para esa entidad en la cache, que se libera después de que se confirma la transacción. Todas las transacciones concurrentes que acceden a registros con bloqueo suave obtendrán los datos correspondientes directamente desde el medio persistente.

## Representación interna

Las entidades no se almacenan en la cache como objetos, sino que solamente se guarda su estado (valores de los atributos).

- Los atributos transitorios no se guardan.
- Las colecciones no se guardan (a menos que se haya explicitado lo contrario)
- En las relaciones xToOne solamente se almacena el id de la entidad externa.

# Sesión

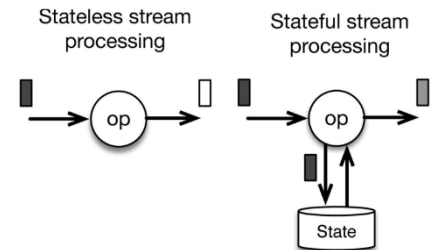
Sirven para identificar al usuario que está intentando acceder a un recurso o ejecutar determinada funcionalidad sobre Sistema.

Puede almacenar temporalmente información relacionada con las actividades del usuario mientras éste está conectado.

No siempre es necesario contar con el concepto de sesión ya que depende del tipo de arquitectura que se haya establecido para el aplicativo.

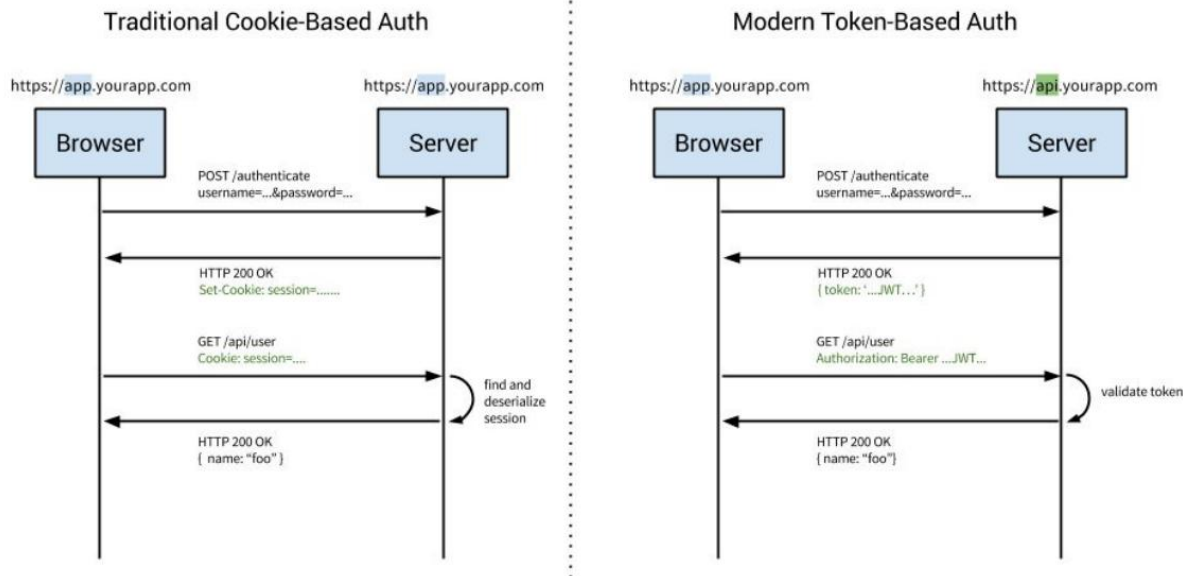
## Cookies

Un pequeño archivo de datos creado por el Servidor, almacenado en el cliente. En cada request del cliente, se envían al servidor para identificar y reconocer el usuario que está realizando la request.



## Stateless vs Stateful (web)

| Stateless                                                                                                                                                                                                                                                                                                                                | Stateful                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                             |
|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <ul style="list-style-type: none"><li>• Sin estado se refiere a los casos en que éstos están aislados.</li><li>• No se almacena ni referencia información sobre las operaciones anteriores.</li><li>• Cada operación opera como si fuera la primera vez.</li></ul>                                                                       | <ul style="list-style-type: none"><li>• Con estado son aquellos a los que se puede volver una y otra vez y éstos recuerdan “quiénes somos” y “qué hicimos anteriormente”.</li><li>• Se realizan con el contexto de las operaciones anteriores, y la operación actual puede verse afectada por lo que ocurrió previamente.</li></ul>                                                                                                                                                                                                                  |
| <ul style="list-style-type: none"><li>• No se almacena sesión del lado del servidor.</li><li>• El usuario se identifica en cada solicitud que realiza.</li><li>• Para identificarse, el usuario envía un token al servidor. El servidor decodifica este token y descubre quién es el usuario que está realizando la solicitud.</li></ul> | <ul style="list-style-type: none"><li>• Se almacena sesión en el servidor.</li><li>• El usuario se identifica una sola vez, no en cada solicitud realizada al servidor.</li><li>• En la solicitud donde el usuario inicia sesión (se validan sus credenciales), el servidor genera un espacio para él (en memoria/archivos/etc.), otorgándole un id. Este id es seteado en la cookie del usuario.</li><li>• En posteriores solicitudes, el servidor revisa la cookie del usuario (que llega en la request) y recupera la sesión por el id.</li></ul> |



## Sticky Sessions

También conocidos como persistencia de sesiones es un feature en load balancers que asegura que las requests de los usuarios siempre sean enviadas a un mismo servidor durante una sesión. Un método de direccionamiento similar o casi igual al hashing. Esta estrategia es útil cuando usamos una sesión stateful y ayuda a mantener la consistencia.

### + Ventajas:

- Consistencia de sesiones: nos aseguramos de que todas las interacciones en la sesión sean consistentes y haya una UX fluida.
- Lógica de aplicación simplificada: se puede usar sesiones stateful sin necesidad de encargarse en los detalles de las sesiones distribuidas en el sistema.
- Optimización de performance y latencia: se evita el overhead de buscar la sesión de un almacenamiento centralizado.
- Implementación más simple: hay load balancers que lo traen incorporado con mínima configuración adicional al implementar.

### - Desventajas:

- Baja eficiencia del Load Balancer: distribuye las sesiones de una forma más desorganizada y puede traer problemas en ciertos casos.
- Single Point of Failure: se cae un servidor y se cae la sesión del usuario si no hay un replicador.
- Problemas al escalar: escalar no sirve si un servidor está siendo sobrecargado, ya que solo manejaría nuevas sesiones y no las existentes en los servidores sobrecargados.
- Imposible de implementar en una arquitectura con sesiones stateless.

## Problemas de seguridad

- Riesgo de hijacking de sesiones: en caso de interceptar los identificadores de una sesión es posible que se pueda tomar control de esta. Se mitiga usando conexiones encriptadas con atributos de cookies seguras para proteger los datos de la sesión (`httpOnly`, `Secure`, `SameSite`)

- Ataques de fijación de sesiones: un atacante puede setear una identificación a un valor conocido de sesión antes de establecer la conexión y usarla para atacarla. Se mitiga al regenerar identificadores de sesiones después de eventos críticos como la autenticación.
- Single Point of Failure: ataques DDoS.

## Recursos en Sistemas Web

Al diseñar un Sistema con arquitectura Web debemos pensar en qué recursos vamos a permitir manipular: dar de alta, modificar, eliminar o buscar. Para esto se debe considerar los casos de uso.

| Recursos independientes/autónomos                                         | Recursos dependientes/anidados                                                                                                                                                                                                                                                |
|---------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Recursos que no depende de nadie, que tienen sentido de existencia propio | Recursos no autónomos: dependen de la existencia de otros. <ul style="list-style-type: none"> <li>• No tiene sentido listar todos estos recursos sin que sean condicionados por su recurso contenedor.</li> <li>• A nivel objetos, es una relación de Composición.</li> </ul> |

## Protección de Recursos

Debemos proteger los recursos que expone un Sistema Web para que éstos no sean accedidos libremente por todos los tipos de usuarios existentes.

Niveles de protección:

- Autenticación
- Autorización basada en roles
- Autorización basada en roles y permisos

Autenticación: Exigir identificación antes de acceder a ellos: pantalla de login / 401 (no autorizado)

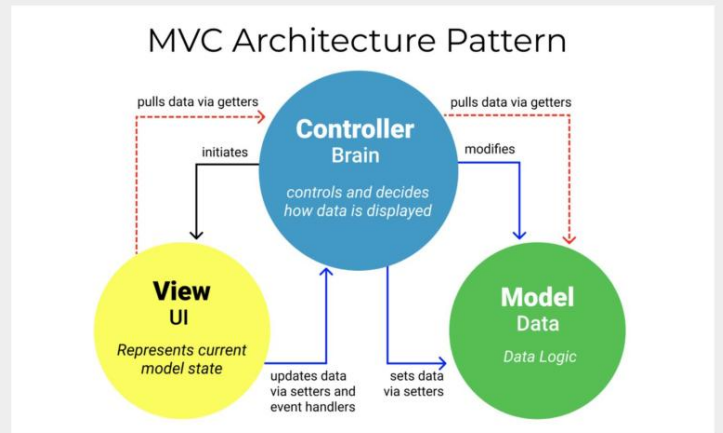
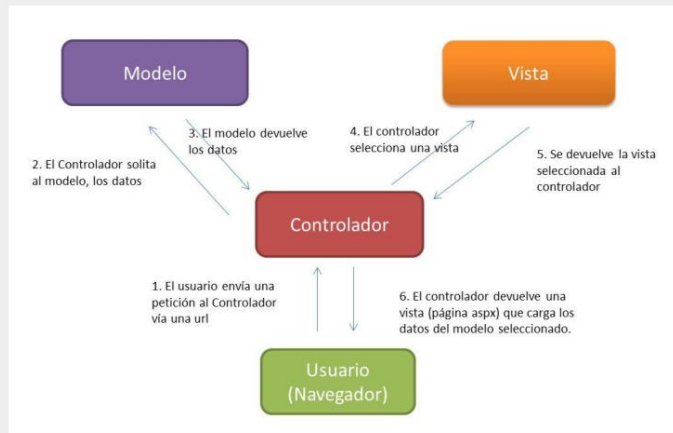
Roles: Requiere de identificación, y que cuente con cierto rol para acceder a ellos, no todos podrán manipular los mismos recursos.

Roles y permisos: Igual que antes, pero con diferentes alcances: propios, propios, todos.



# Patrones Arquitectónicos de Interacción (Web)

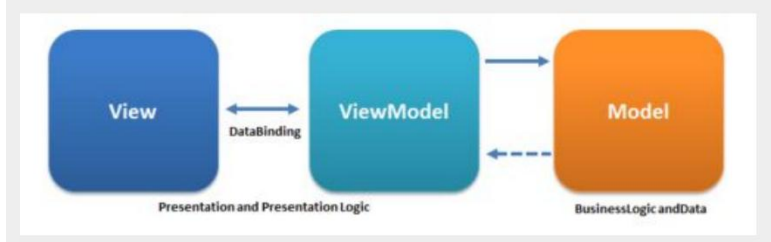
## MVC: Modelo Vista Controlador



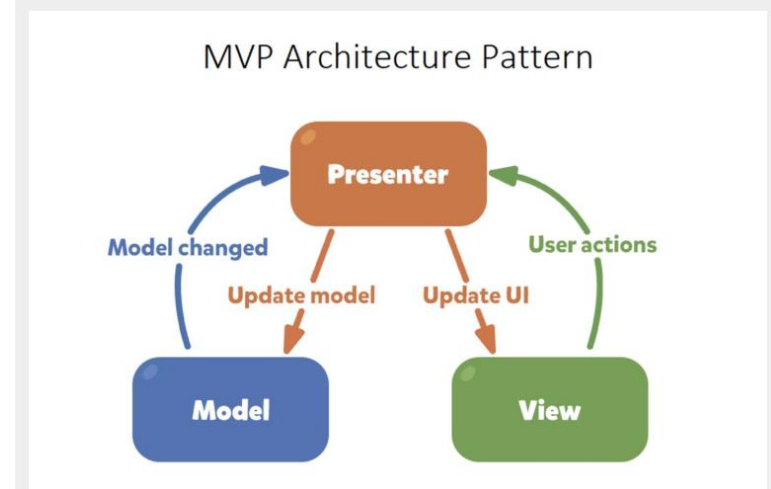
+ **Ventajas**: Buena separación de intereses – Reusabilidad de vistas y Controladores – Flexibilidad.

- **Desventajas**: Mayor complejidad de las aplicaciones – No siempre útil en aplicaciones con vistas simples - Más complejo de testear

## MVVM: Modelo Vista VistaModelo

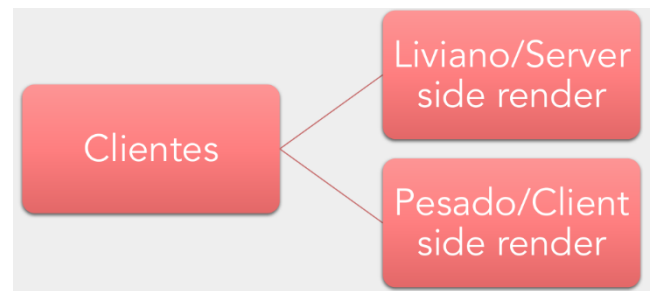


## MVP: Modelo Vista Presentador



## Tipos de Clientes

Cuando se tiene un Sistema diseñado bajo una Arquitectura Web y se necesita contar con capa de presentación visual, surge la necesidad de definir quién tendrá la responsabilidad de generar las vistas.



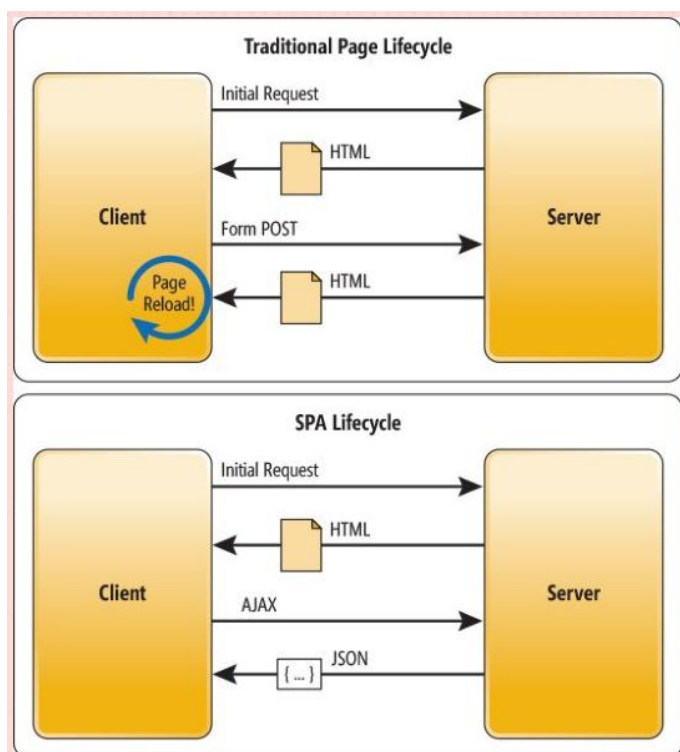
| Liviano/Server side render                                                                                                                                                                                                                                                                        | Pesado/Client side render                                                                                                                                                                                                                                                      |
|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <ul style="list-style-type: none"><li>• Las vistas son generadas del lado del Servidor.</li><li>• Las respuestas a las solicitudes son casi siempre HTML.</li><li>• El navegador se encarga solamente de mostrar el contenido HTML. Con casos de cierto código JS del lado del Cliente.</li></ul> | <ul style="list-style-type: none"><li>• Las vistas son generadas del lado del Cliente.</li><li>• Las respuestas a las solicitudes son JSON/XML...</li><li>• El Cliente posee toda la lógica necesaria para presentar los datos recibidos, mediante código JavaScript</li></ul> |

|                                | Cliente Liviano                                                                                | Cliente Pesado                                                                                 |
|--------------------------------|------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------|
| Generación de la Vista         | Servidor                                                                                       | Cliente                                                                                        |
| Stream (dato que se transmite) | HTML                                                                                           | JSON                                                                                           |
| Procesamiento                  | <ul style="list-style-type: none"><li>• Mayor en Servidor</li><li>• Menor en Cliente</li></ul> | <ul style="list-style-type: none"><li>• Mayor en Cliente</li><li>• Menor en Servidor</li></ul> |
| Uso de Memoria                 | <ul style="list-style-type: none"><li>• Mayor en Servidor</li><li>• Menor en Cliente</li></ul> | <ul style="list-style-type: none"><li>• Mayor en Cliente</li><li>• Menor en Servidor</li></ul> |
| Lógica de Interacción          | Servidor                                                                                       | Cliente                                                                                        |
| Usabilidad (*)                 | -                                                                                              | +                                                                                              |
| Escalabilidad (*)              | -                                                                                              | +                                                                                              |
| Mantenibilidad (*)             | -                                                                                              | +                                                                                              |

| Characteristics               | Web app                                                                            | Hybrid app                                                          | Native app                                                                                                |
|-------------------------------|------------------------------------------------------------------------------------|---------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------|
| <b>Usage</b>                  | Users can access directly from a browser                                           | Users must install the app on their device of choice                | Idem                                                                                                      |
| <b>Internal working</b>       | Client code in the browser communicates with remote server-side code and databases | Client code and browser code wrapped in a native shell or container | Client code written in technology and language specific to the device or platform it will be installed on |
| <b>Native device features</b> | No                                                                                 | Accessible                                                          | Accessible                                                                                                |
| <b>User experience</b>        | Inconsistent and dependent on the browser being used                               | Consistent and engaging                                             | Idem                                                                                                      |
| <b>Access</b>                 | Limited by browser and network connectivity                                        | One-step access with offline features                               | Idem                                                                                                      |
| <b>Performance</b>            | Slower and less responsive                                                         | Faster, but may consume more battery power                          | Performance can be optimized to device                                                                    |
| <b>Development</b>            | Cost-efficient, faster time to market                                              | Cost-efficient, fast time to market                                 | Expensive, slower time to market                                                                          |

## SPA: Single Page Application

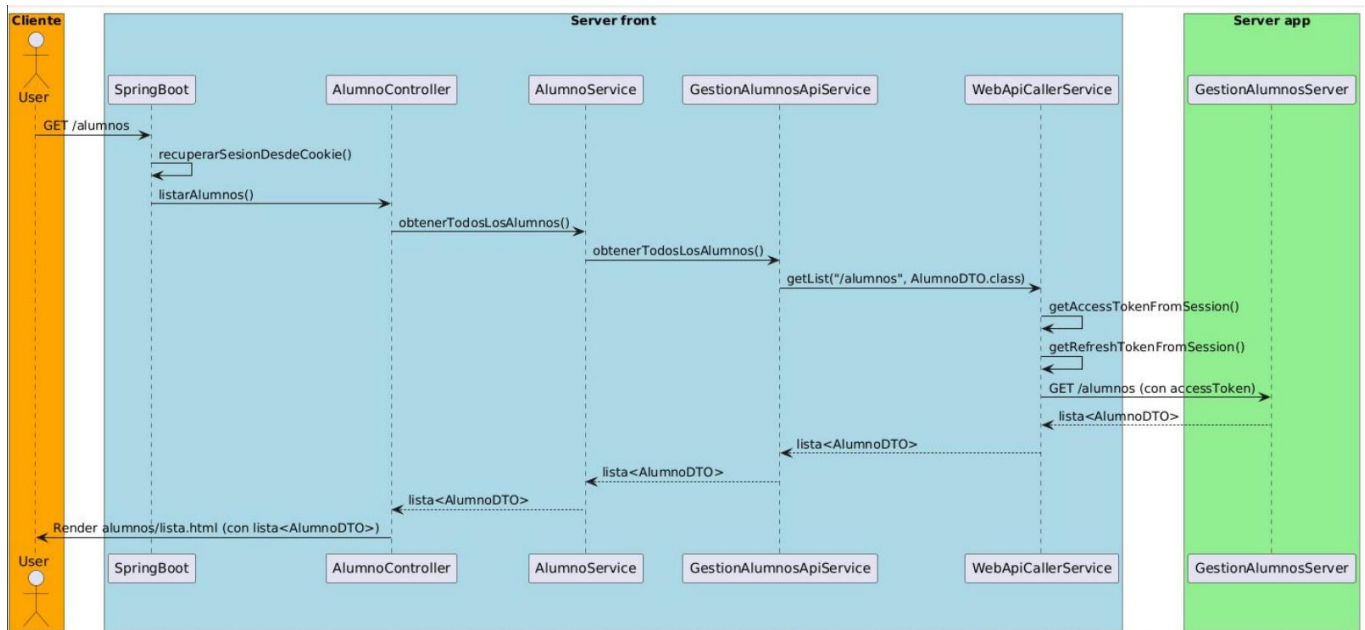
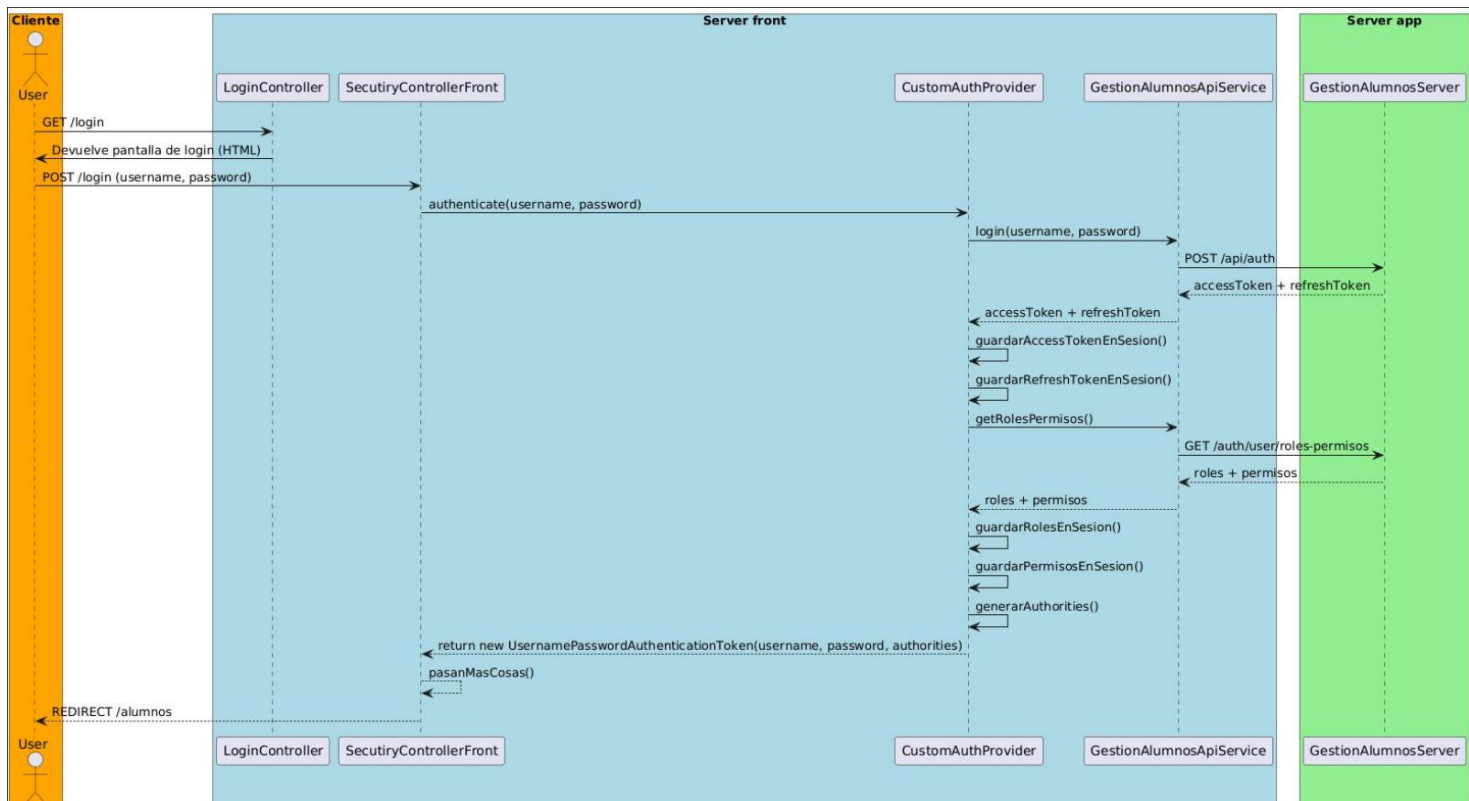
Un tipo de aplicación web donde todas las pantallas las muestra en la misma página, sin recargar el navegador. Toda acción se produce dentro de un mismo HTML con varias vistas: produciendo el efecto de tener varias páginas.



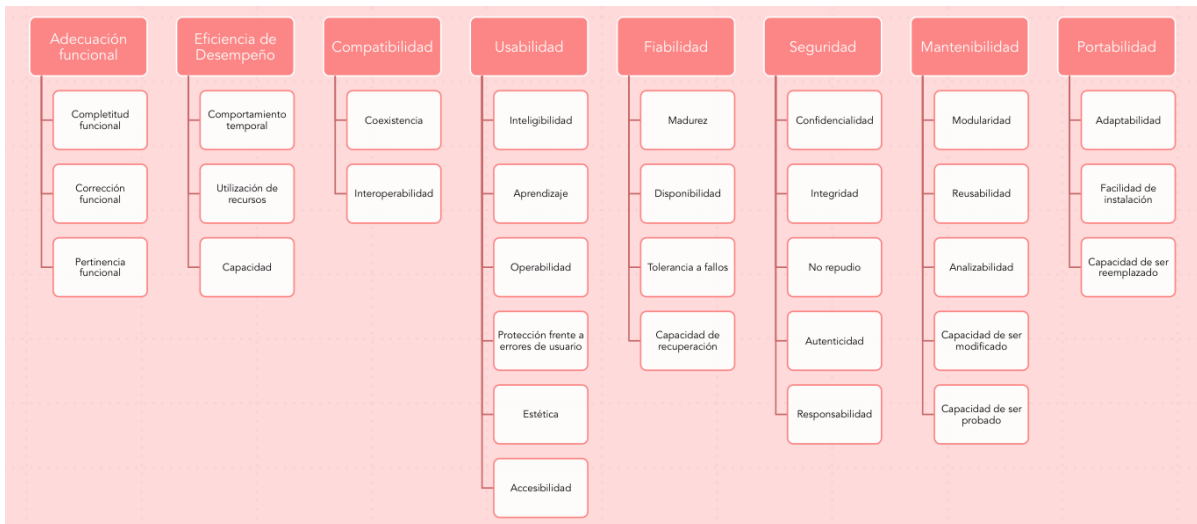
## Sesiones Mixtas

Definimos como “Sesiones mixtas” a la implementación que realizamos cuando nuestro frontend está implementado con un Cliente Liviano Desacoplado y, por lo tanto, manejamos:

- Sesiones en cookies desde el usuario final hacia nuestro Servidor Frontend
- Tokens de autorización desde nuestro Servidor Frontend hacia nuestro Servidor de Aplicación (donde reside la lógica de negocio).



# Atributos de Calidad (Arquitectónicos)



Los más evaluados son:

- Eficiencia de Desempeño – Utilización de recursos (“Performance”)
- Fiabilidad – Disponibilidad
- Fiabilidad – Tolerancia a fallos
- Seguridad (en general)

## Atributos de Calidad en conflicto

- Performance vs. Seguridad
- Seguridad vs Disponibilidad (aunque están vinculados)
- Performance vs Modificabilidad

Se deben evaluar los múltiples atributos de calidad con el objetivo de Diseñar un Sistema “Good Enough” para los Stakeholders.

## Escalabilidad

¿Qué sucedería si un único Servidor tuviera que atender solicitudes de un número muy elevado de clientes a la vez? ¿Podría hacerlo solo?

- La performance podría degradarse sustancialmente (tiempos de respuesta muy elevados)
- La disponibilidad podría verse afectada ya que, si dicho Servidor se “cae”, todos los clientes quedarían sin poder realizar sus solicitudes.

## Escalabilidad Vertical:

- Agregarle más recursos al Servidor físicamente (memoria, disco, procesamiento) para que éste pueda atender más solicitudes de forma concurrente.
- Existe una limitación física
- No soluciona la disponibilidad, solo el performance.

## Escalabilidad Horizontal:

- Implica agregar más equipos Servidores para atender las solicitudes de los clientes, distribuyendo la carga entre ellos.
- No hay limitación física, pero trae complejidad al tener que distribuir las solicitudes.
- Se debe contar con un balanceador de cargas.
- Resuelve la disponibilidad y performance

La solución a implementar una escalabilidad horizontal es el uso de un load Balancer.

## Escalando la capa de datos

Escalar verticalmente es sencillo para una base de datos, el problema está en seguir dándole carga a un mismo punto generando un posible: Single Point of Failure. Hasta cierto punto suele ser más caro escalar de forma vertical que de forma horizontal.

Escalar horizontalmente trae la complejidad de la consistencia de datos entre nuestros sistemas.

Técnicas para escalar de forma horizontal

## Replicación de base de datos

Puede ser usada con sistemas con base de datos ya existentes, donde se deben tener mínimo dos:

- Master: soporta operaciones de escritura (insert, update y delete)
- Slaves: soporta operaciones de lectura (select)



En caso de los read podemos tener varias BD disponibles para la misma, cuando haya que realizar una escritura en el sistema recurrimos a la master y realizamos el cambio. Después, dependiendo de la política de replicación debemos replicar estos cambios en las tres bases de datos esclavas.

### + Ventajas:

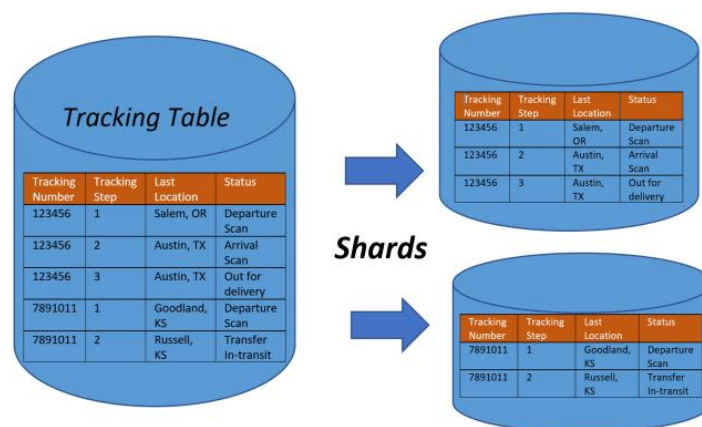
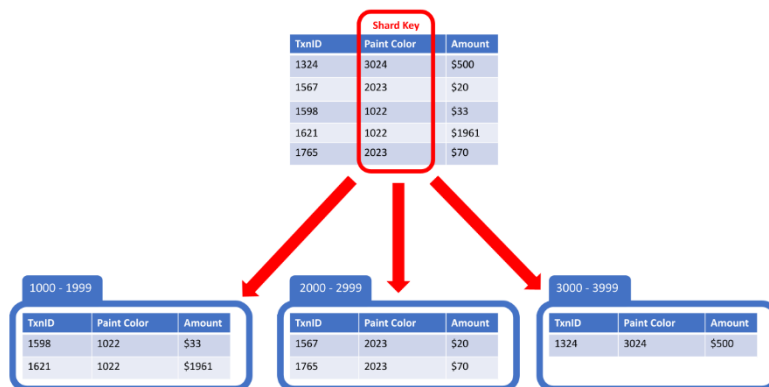
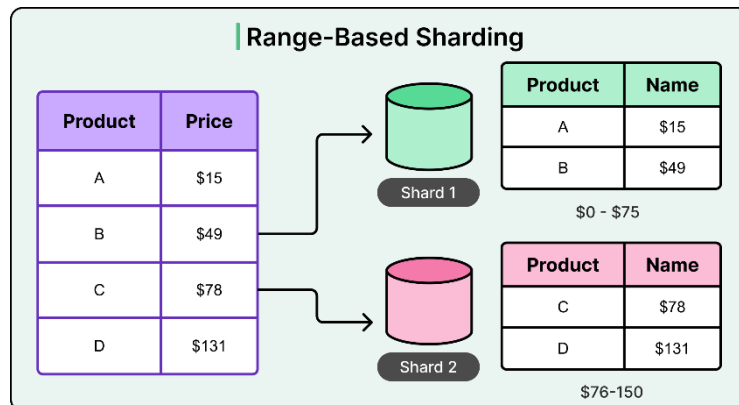
- ✓ Rendimiento: se distribuye la cantidad de consultas que se procesan en simultáneo.
- ✓ Fiabilidad: anulamos el SPOF para las bases de datos y sabemos que otra BD tiene los datos necesarios.
- ✓ Alta disponibilidad: en caso de fallar el sistema puede seguir en funcionamiento.
  - No hay slaves disponibles: la master pasa a atender las consultas de lectura.
  - Caen los masters: se toma un slave para reemplazar, paso previo se aplican operaciones pendientes de escritura para que no esté como master y desactualizada.

## Sharding (fragmentación)

Consiste en la separación de una gran base de datos en múltiples de tamaño más chico. El punto clave para su funcionamiento es en la clave de fragmentación que se encarga dividir los datos.

### - Complejidades:

- Resharding de datos: puede ser necesario volver a ordenar los datos en su formato original cuando un Shard ya está lleno o algunos crecieron de forma desigual al resto. Es un proceso que puede llevar tiempo y se busca evitar.
- Problema de celebridades: ocurre cuando se usan repetidas veces varios datos dentro de un mismo Shard. Puede generar una sobrecarga sobre la misma y causar problemas de performance y hasta disponibilidad.
- Join y desnormalización: puede haber problemas de performance al realizar muchas operaciones JOIN entre los diferentes shards de una base de datos. Para solucionar esto se desnormalizaría la BD para realizar consultas sobre una misma tabla.



## Aplicación Monolítica

Caracterizados por tener toda la lógica de negocio en un único proyecto de código fuente.

Las aplicaciones Desktop tradicionales o aplicaciones Mobile sin conexión a un servidor siguen este tipo de arquitectura.

Si pensamos en un Sistema bajo una Arquitectura Web Monolítica, con capa de presentación visual y un cliente liviano, podemos notar que todos sus componentes están en un único código fuente ejecutándose del lado del Servidor

### + Ventajas:

- Fáciles de desarrollar: al menos que haya un gran número de gente trabajando a la vez.
- Cualquier cambio obliga al aplicativo a reiniciarse por completo.

### Posibles drivers de calidad:

- Escalabilidad: agregar más nodos para soportar mayor carga.
- Disponibilidad: tolerar fallas parciales sin interrumpir el servicio.
- Flexibilidad: permitir actualizaciones y despliegues independientes

## SOA – Service Oriented Architecture

Estilo de arquitectura que promueve descomponer la lógica funcional de una aplicación en unidades autónomas denominadas servicios

### Principios:

- El Valor del Negocio por encima de la estrategia técnica.
- Las Metas Estratégicas por encima de los beneficios específicos de los proyectos.
- La Interoperabilidad Intrínseca por encima de la integración personalizada.
- Los Servicios Compartidos por encima de las implementaciones de propósito específico.
- La Flexibilidad por encima de la optimización.
- El Refinamiento Evolutivo encima de la búsqueda de la perfección inicial.



Actores:



**Desafíos:** Comunicación y sincronización entre nodos, Consistencia de datos, Seguridad y autenticación centralizada.

Componentes:

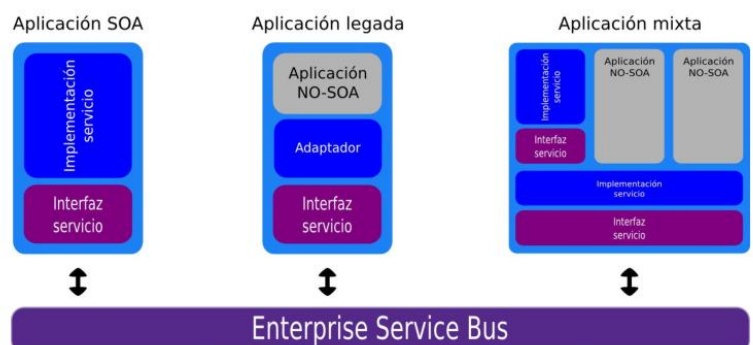
- Consumidor de servicios: demanda la funcionalidad proporcionada de un servicio
- Proveedor de servicios: acepta y ejecuta consultas de consumidores y con una interfaz que el consumidor pueda acceder.
- Registro de servicios: repositorio de servicios disponibles, permite ver sus interfaces.
- Bus de Servicio Empresarial (ESB): gestiona la comunicación entre múltiples servicios web. Se enfoca en resolver el problema la multiplicidad de servicios y apps y el conector entre ellas.
  - Virtualización de los servicios: comunicación a través de un conector.
  - Ubicación e identidad: el ESB identifica y establece las rutas de los mensajes entre servicios.
  - Mediante HTTP, FTP, SMTP.

Implementaciones de ESBs:

- OpenESB
- Oracle ESB
- Azure Service Bus
- IBM WebSphere ESB
- IBM WebSphere Integration Bus (IBM WebSphere Message Broker)
- JBoss Fuse
- Apache ServiceMix
- [Spring Integration](#)



## Tipos de aplicaciones SOA



# Aplicaciones en la Nube vs Tradicionales

## Aplicativos en la nube – **desafíos**:

- Cambia la forma en que se diseñan las aplicaciones.
- En lugar de ser “monolitos”, las aplicaciones se descomponen en servicios menores y descentralizados.
- Los servicios se comunican a través de API o mediante el uso de eventos o de mensajería asincrónica.
- Las aplicaciones se escalan horizontalmente, agregando nuevas instancias, tal y como exigen las necesidades.
- El estado de las aplicaciones se distribuye.
- Las operaciones se realizan en paralelo y de forma asincrónica.
- Las aplicaciones deben ser resistentes cuando se produzcan errores.
- Las implementaciones deben estar automatizadas y ser predecibles.
- La supervisión y la telemetría son fundamentales para obtener una visión general del sistema.

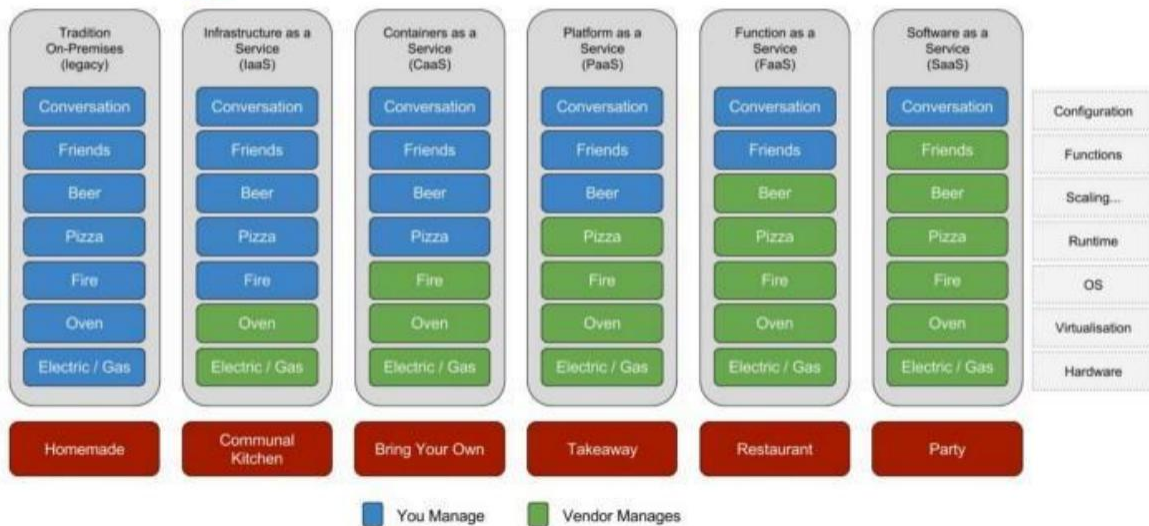
| Local – Tradicional                                                                                                                                                                                                                                                                                                                                                           | Nube – “Moderna”                                                                                                                                                                                                                                                                                                                                                                                                                       |
|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <ul style="list-style-type: none"><li>• Monolítica</li><li>• Diseñada para una escalabilidad predecible</li><li>• Base de datos relacional</li><li>• Procesamiento sincronizado</li><li>• Diseño para evitar errores</li><li>• Actualizaciones grandes ocasionales</li><li>• Administración manual (Infraestructura)</li><li>• Servidores Personalizados (Mutables)</li></ul> | <ul style="list-style-type: none"><li>• Descompuesto</li><li>• Diseñado para un escalado elástico</li><li>• Persistencia Políglota (combinación de tecnologías de almacenamiento)</li><li>• Procesamiento asincrónico</li><li>• Diseñado para soportar errores</li><li>• Pequeñas actualizaciones frecuentes</li><li>• Administración automatizada (Infraestructura)</li><li>• Infraestructura inmutable (Nuevas Instancias)</li></ul> |





## Pizza as a Service 2.0

<http://www.paulkernison.co.uk>



# Componentes de una Arquitectura

Una arquitectura distribuida se construye combinando distintos componentes especializados que trabajan de forma coordinada donde cada uno cumple un rol clave para garantizar el rendimiento, la disponibilidad, la seguridad y la escalabilidad del sistema.

Compuesta de capas funcionales:

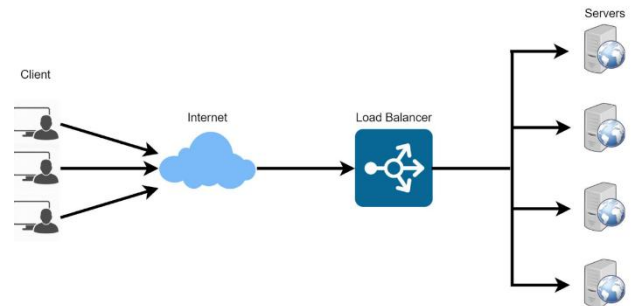
- Entrega y balanceo de tráfico.
- Almacenamiento y rendimiento.
- Gestión de identidad.
- Seguridad perimetral.

## Balanceador de Cargas (Load Balancer)

Distribuye las solicitudes entrantes entre múltiples servidores o instancias de aplicación para optimizar el rendimiento y evitar sobrecargas.

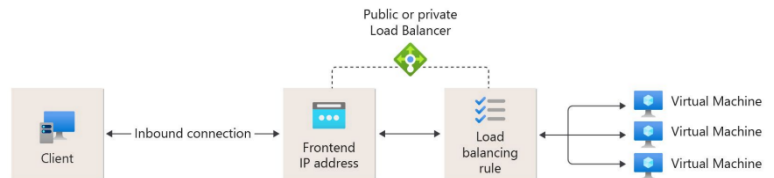
+ Beneficios:

- Si un servidor falla, redirige el tráfico automáticamente. (Disponibilidad)
- Permite agregar o quitar nodos sin afectar al usuario. (Escalabilidad - Horizontal)
- Mantiene el uso de recursos equilibrado. (Performance - Uso de Recursos)



Algoritmos:

- Round Robin: distribuye solicitudes secuencialmente.
- Least Connections: prioriza servidores con menos conexiones activas.
- Weighted Load: asigna pesos según la capacidad del servidor.



Tipos: (importante che)

- L4 (Transporte): balancea tráfico TCP/UDP.
- L7 (Aplicación): inspecciona contenido HTTP/HTTPS y puede tomar decisiones basadas en encabezados, cookies o URL.

## Cache

Almacena resultados de operaciones o datos de uso frecuente para reducir el tiempo de acceso y la carga sobre los servicios principales o las bases de datos.

Algunos Tipos de caché:

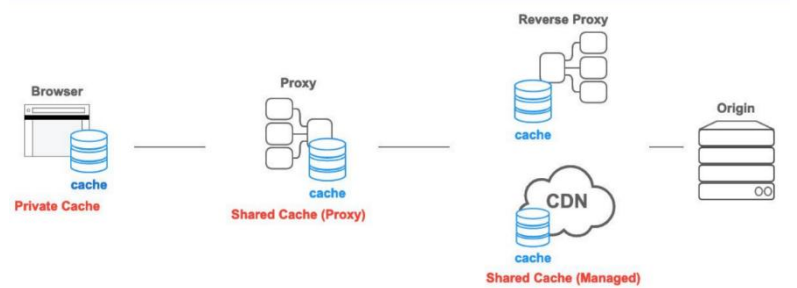
- de aplicación: guarda datos calculados (ej. consultas SQL costosas).
- de contenido: almacena recursos estáticos como imágenes, CSS o JS.
- de navegador: guarda recursos en el cliente final. (Usado casi siempre no usen para justificar)

### + Ventajas:

- Reducción del tiempo de respuesta. (Performance - Tiempo de Respuesta)
- Menor consumo de ancho de banda. (Performance - Uso de Recursos)
- Disminución de carga sobre los servidores de origen. (Performance - Uso de Recursos)

### Características:

- Privada (Usuario)
- Compartida
  - Proxy Reverso
  - CDN
- Tiempo de Expiración



| URL                            | Response               |
|--------------------------------|------------------------|
| https://example.com/index.html | <!doctype html>...     |
| https://example.com/style.css  | body { ...             |
| https://example.com/script.js  | function main () { ... |



| URL                            | Accept-Language | Response               |
|--------------------------------|-----------------|------------------------|
| https://example.com/index.html | ja-JP           | <!doctype html>...     |
| https://example.com/index.html | en-US           | <!doctype html>...     |
| https://example.com/style.css  | ja-JP           | body { ...             |
| https://example.com/script.js  | ja-JP           | function main () { ... |

## Cache HTTP

La caché HTTP almacena una respuesta asociada con una solicitud y reutiliza la respuesta almacenada para solicitudes posteriores.

Hay varias ventajas de reusabilidad. En primer lugar, dado que no es necesario enviar la solicitud al servidor origen, cuanto más cerca están el cliente y la caché, más rápida será la respuesta. El ejemplo más típico es cuando el navegador almacena una caché para las solicitudes.

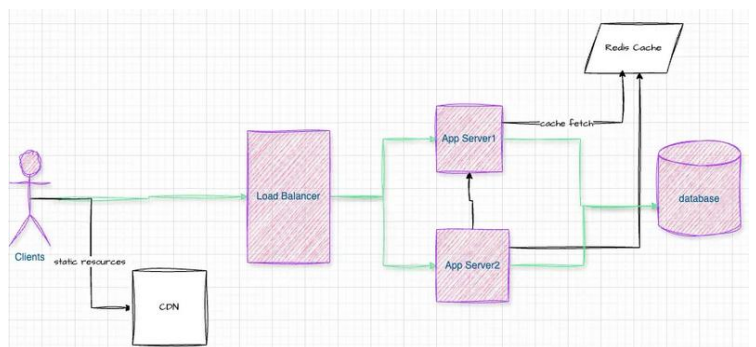
El servidor de origen no necesita procesar la solicitud, por lo que no necesita parsear y enrutar la solicitud, restaurar la sesión en función de la cookie, consultar los resultados de la base de datos o renderizar la plantilla. Eso reduce la carga en el servidor.

El funcionamiento adecuado de la memoria caché es fundamental para la salud del sistema.

## Componentes: CDN Arquitectura

Una red de entrega de contenido (CDN) está formada por un grupo de servidores distribuidos geográficamente que trabajan juntos para ofrecer una entrega rápida de contenido de Internet.

- ✓ Permite la transferencia rápida de activos necesarios para cargar contenido de Internet, incluidas páginas HTML, archivos JS, CSS, imágenes y vídeos.
- ✓ Puede ayudar a proteger sitios web contra algunos ataques maliciosos comunes, como los ataques de denegación de servicio distribuido (DDOS).



### + Ventajas:

- Mejora de los tiempos de carga del sitio web
- Reducción de los costos de ancho de banda
- Aumento de la disponibilidad de contenido y la redundancia
- Mayor seguridad del sitio web

### ¿Qué se puede almacenar en una CDN?

1. Contenido estático: JS, CSS, HTML, fonts.
2. Respuestas de API con headers.
3. Archivos de imágenes y video.
4. Paquetes de software o contenido descargable.

### Proveedores de CDN: Ejemplos

- Cloudflare, Akamai, Amazon CloudFront, Google Cloud CDN.

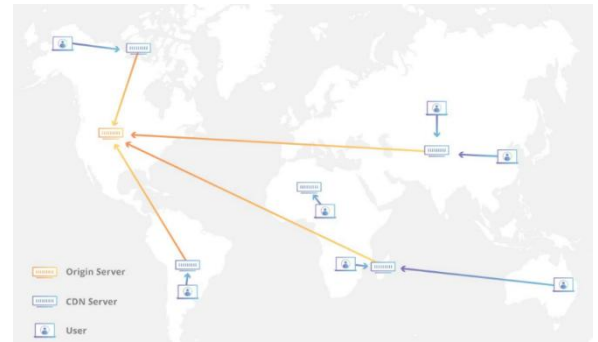
### Métodos de invalidación

- Time-to-live (TTL): setear una expiración para el contenido.
- Invalidación manual: purge de URLs específicas.
- URLs versionadas: cambiar el nombre de archivos fuerza a las CDNs refetchear.

Usado en: páginas web fuertemente estáticas o apps globales

### ¿Dónde se puede cachear datos y cuáles?

| Capa                     | Ejemplo                                    |
|--------------------------|--------------------------------------------|
| <b>Client-side</b>       | Browser caches CSS, JS, images             |
| <b>CDN (edge)</b>        | Cloudflare, Akamai serving static assets   |
| <b>Application-Level</b> | In-memory cache (guava, caffeine in Java)  |
| <b>Database-level</b>    | Query results cached in Redis or Memcached |





## Estrategias de Caching

| Strategy                          | Description                                                        | Example Use Case                 |
|-----------------------------------|--------------------------------------------------------------------|----------------------------------|
| <i>Read-Through</i>               | App queries cache first; if miss, fetch from DB and populate cache | Product details                  |
| <i>Write-Through</i>              | Write to cache and DB at the same time                             | Inventory updates                |
| <i>Write-Behind (Write-Back)</i>  | Write to cache first, then synchronize when a cache miss occurs    | Logging, analytics               |
| <i>Cache-Aside (Lazy Loading)</i> | App loads data into cache when a cache miss occurs                 | User profile info                |
| <i>TTL (Time to live)</i>         | Set expiry time for cached data                                    | Weather info, stock prices       |
| <i>LRU</i>                        | Eviction based on usage patterns                                   | Browser history, recommendations |

## Herramientas de caching comunes

In-Memory Caches:

- Redis: Fast, persistent, soporta tipos de datos complejos.
- Memcached: lightweight, high-performance, simple key-value store.
- Caffeine: Local in-process cache for microservices.

## Errores típicos de la Caché

- Stale Data: sin la invalidación apropiada, los usuarios podrían ver data antigua.
- Caché Stampede: varios usuarios acceden la caché a la vez justo cuando expira.
- Over-Caching: puede llevar a inconsistencias entre la caché y la DB.

## Mejores Prácticas

1. Siempre usar TTL, pero también invalidar con actualizaciones.
2. Usar versionado para claves caché.
3. Monitorear caché hit/miss ratios y latencia.
4. Shard o particionar caché con BD grandes.

## Componentes: IAM/IDM

Es un componente que gestiona las identidades y los accesos.

- IAM: Acceso
  - Estrategias para el acceso a través de contraseñas, biometría, autenticación multifactor...
- IDM: Identidad
  - Se centra en la identidad, roles, permisos, y los grupos a los que pertenece un usuario

## Vínculo de Componentes y Atributos de Calidad

| Componente                     | Disponibilidad                                                                    | Escalabilidad                                                    | Performance                                                 | Seguridad                                                      | Usabilidad                                                |
|--------------------------------|-----------------------------------------------------------------------------------|------------------------------------------------------------------|-------------------------------------------------------------|----------------------------------------------------------------|-----------------------------------------------------------|
| Load Balancer                  | Mejora la disponibilidad al distribuir tráfico y eliminar puntos únicos de falla. | Permite agregar o quitar servidores sin interrumpir el servicio. | Reduce sobrecarga en nodos individuales.                    | Puede integrarse con certificados y WAF.                       | —                                                         |
| Cache                          | Reduce dependencia del backend ante fallas temporales.                            | Puede escalar mediante servidores distribuidos.                  | Mejora tiempos de respuesta y disminuye latencia.           | Requiere control de acceso y coherencia (integridad) de datos. | Mejora la experiencia del usuario al acelerar respuestas. |
| CDN                            | Alta disponibilidad mediante nodos distribuidos globalmente.                      | Escala automáticamente frente a picos de tráfico.                | Entrega contenido desde el servidor más cercano al usuario. | Incluye mitigación DDoS y cifrado TLS.                         | Mejora tiempos de carga percibidos por el usuario.        |
| SSO                            | —                                                                                 | —                                                                | Disminuye autenticaciones redundantes.                      | Centraliza autenticación y reduce exposición de credenciales.  | Unifica la experiencia de inicio de sesión.               |
| WAF (Web Application Firewall) | Evita caídas causadas por ataques.                                                | —                                                                | Añade latencia mínima si se configura correctamente.        | Protege contra amenazas OWASP Top 10.                          | —                                                         |

## Seguridad de la información

La seguridad de la información es el conjunto de procesos, políticas, prácticas y tecnologías orientadas a proteger la confidencialidad, integridad y disponibilidad de la información, ya sea en forma digital o física, frente a accesos no autorizados, alteraciones, pérdidas o destrucción.

¿Qué puede generar un mal diseño a nivel seguridad?

- Vulnerabilidades explotables
  - Fallas estructurales que los atacantes pueden aprovechar.
  - Ej: inyección SQL, XSS, escalamiento de privilegios.
- Accesos no autorizados
  - Usuarios que acceden a datos o funciones que no les corresponden.
  - Falta de controles de autenticación/autorización.
- Fuga o pérdida de información sensible
  - Datos personales, financieros o confidenciales expuestos.
  - Riesgo de sanciones legales (Ley 25.326, GDPR).
- Indisponibilidad del sistema
  - Ataques de denegación de servicio (DoS/DDoS).
  - Faltan mecanismos de recuperación, redundancia o escalabilidad.
- Dificultad para detectar y responder a incidentes
  - Sin logs ni trazabilidad: no sabés qué pasó, ni cuándo, ni cómo.
  - Falta de alertas y monitoreo en tiempo real.
- Costos de corrección elevados
  - Cuesta hasta 100 veces más corregir una falla en producción que en diseño.
  - “La seguridad mal diseñada se paga dos veces: antes y después del ataque.”
- Consecuencias legales y reputacionales
  - Multas, demandas, pérdida de clientes, daño a la marca.
  - Riesgo de que el sistema sea retirado del mercado o discontinuado.



## Tríada de la seguridad

- Confidencialidad: proteger la información de accesos no autorizados: Encriptación de emails.
- Integridad: asegurar que la información no sea modificada: Control de versiones y hash
- Disponibilidad: garantizar que este accesible cuando se necesite: Backups y redundancia

## ¿Qué es el Desarrollo Seguro?

El desarrollo seguro es la práctica de **diseñar, codificar, probar y mantener software** con el objetivo de minimizar vulnerabilidades y prevenir ataques, aplicando principios de seguridad desde las primeras etapas del ciclo de vida del desarrollo.

### Principios de Desarrollo Seguro

- Security by Design: pensar la seguridad desde la etapa de diseño.
- Validación de entradas y salidas en profundidad: previene inyecciones y errores lógicos.
- Gestión segura de errores y excepciones: sin exponer información interna.
- Uso seguro de criptografía: evitar algoritmos débiles o mal implementados.
- Control de versiones y dependencias: evitar bibliotecas vulnerables.
- Autenticación y autorización robusta: con políticas de acceso claras.
- Auditoría y trazabilidad: registrar eventos y cambios críticos.
- Principio de menor privilegio: dar solo los accesos necesarios.
- Revisión de código y pruebas de seguridad (estáticas y dinámicas).

## Mecanismo de seguridad

Un mecanismo de seguridad informática es una técnica usada para fortalecer la confidencialidad, integridad y/o la disponibilidad de un sistema informático.

### Tipos de mecanismos de seguridad

- Preventivos: actúan antes de que un hecho ocurra y su función es detener agentes no deseados.
- Detectivos: actúan antes de que un hecho ocurra y su función es revelar la presencia de agentes no deseados en algún componente del sistema. Se caracterizan por enviar un aviso y registrar la incidencia.
- Correctivos: actúan luego de ocurrido el hecho y su función es corregir las consecuencias.

## Cifrado

El **cifrado** es el proceso de transformar información legible (texto plano) en un formato codificado (texto cifrado) mediante algoritmos matemáticos, de manera que solo personas autorizadas con la clave correcta puedan acceder a su contenido original.

### Características

- Protege la confidencialidad de la información para datos en reposo y en tránsito.

### Tipos de cifrados:

- Cifrado simétrico: usa la misma clave para cifrar y descifrar (ej: AES).

- Cifrado asimétrico: usa un par de claves pública y privada (ej: RSA).

## Hashing

El hashing es un proceso que toma una entrada (o mensaje) y la transforma en una cadena de caracteres de longitud fija, llamada hash o valor hash, mediante una función matemática. Esta transformación es unidireccional, es decir, no es posible reconstruir la entrada original a partir del hash.

## Características

- Usado para verificar la integridad de datos y contraseñas. Donde cada uno es totalmente diferente con un mínimo cambio.
- No se puede recuperar la información original.
- Se combina con salting para proteger contraseñas
- Clave en firmas digitales, blockchain y almacenamiento seguro de contraseñas.
- Útil para comparar grandes volúmenes de datos.

| Característica           | Hashing                                                        | Cifrado                                     |
|--------------------------|----------------------------------------------------------------|---------------------------------------------|
| Propósito principal      | Verificar integridad                                           | Proteger la confidencialidad de los datos   |
| Direccionalidad          | Unidireccional (no reversible)                                 | Bidireccional (cifrar y descifrar)          |
| Resultado                | Valor hash de longitud fija                                    | Texto cifrado, variable según algoritmo     |
| Uso de claves            | No usa claves (aunque se puede usar salting)                   | Usa claves para cifrar y descifrar          |
| Ejemplos comunes         | SHA-256, SHA-3, MD5 (obsoleto)                                 | AES, RSA, Blowfish                          |
| Aplicaciones típicas     | Validación de integridad, almacenamiento seguro de contraseñas | Comunicación segura, almacenamiento cifrado |
| Seguridad ante reversión | Muy difícil de revertir (diseñado así)                         | Puede ser revertido con la clave correcta   |

# Autenticación vs Autorización

| Autenticación                                                                       | Autorización                                                                 |
|-------------------------------------------------------------------------------------|------------------------------------------------------------------------------|
| Confirma que los usuarios son quienes dicen ser, validando la identidad del usuario | Otorga a los usuarios autenticados permiso para obtener acceso a un recurso. |

| Concepto      | Autenticación                                         | Autorización                                                             |
|---------------|-------------------------------------------------------|--------------------------------------------------------------------------|
| ¿Qué es?      | Verificar la identidad del usuario.                   | Determinar qué recursos o acciones puede acceder o realizar ese usuario. |
| Propósito     | Confirmar <i>quién eres</i> .                         | Definir <i>qué puedes hacer</i> .                                        |
| Cuándo ocurre | Antes de la autorización, al iniciar sesión o acceso. | Después de autenticarse, al solicitar recursos.                          |
| Ejemplos      | Contraseñas, biometría, tokens, certificados.         | Roles, permisos, políticas de acceso.                                    |
| Resultado     | Usuario validado o rechazado.                         | Acceso concedido o denegado a funciones o datos.                         |

## Autenticación multifactor

Un método de seguridad que requiere que el usuario presente dos o más factores de verificación independientes para probar su identidad antes de obtener acceso a un sistema o recurso.

Beneficios:

- Reduce significativamente el riesgo de accesos no autorizados.
- Protege incluso si la contraseña es robada o comprometida.
- Es un requisito creciente en regulaciones y buenas prácticas de seguridad.

Tipos de autenticación multifactor

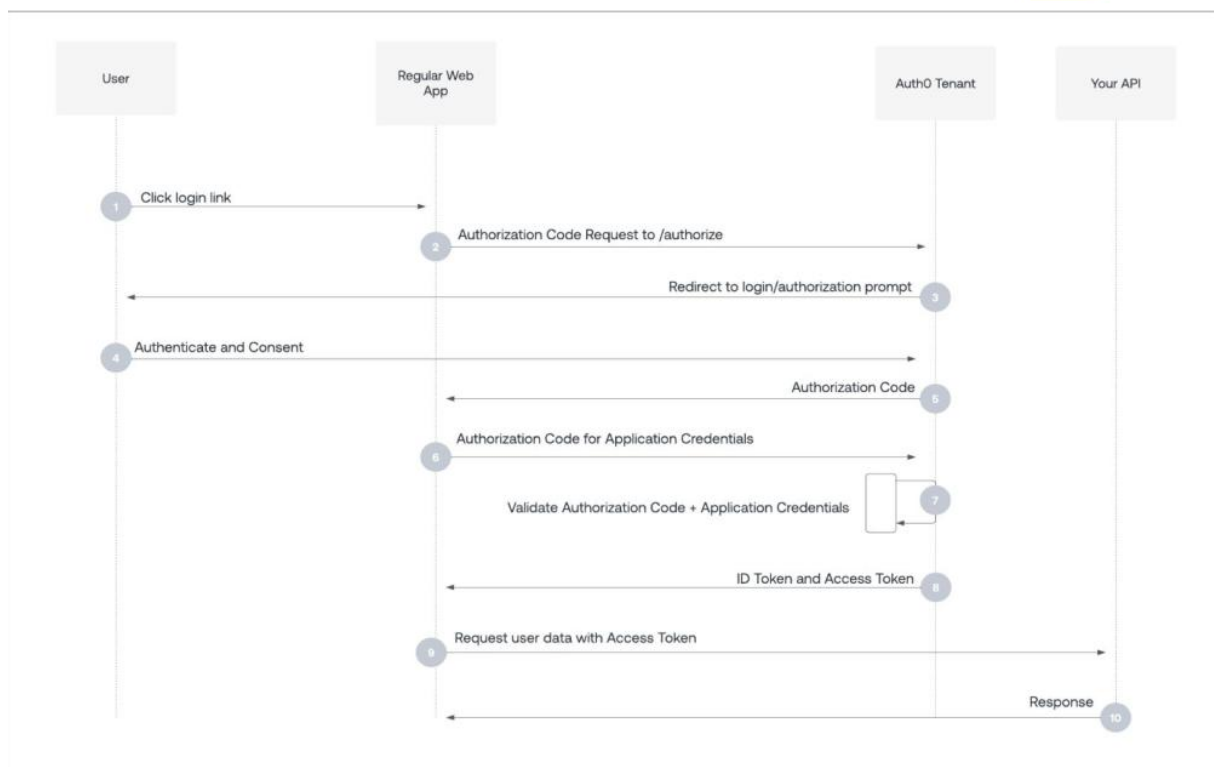
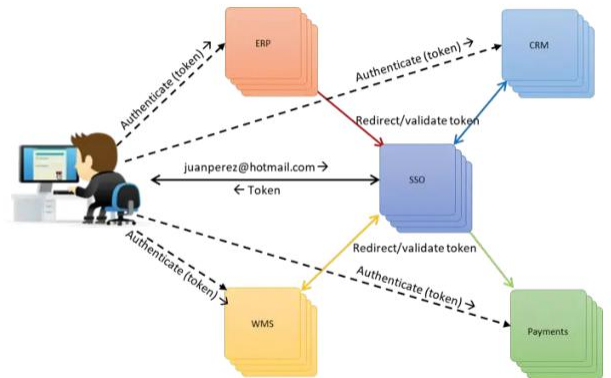
1. Conocimiento - Algo que sabes: contraseña, PIN.
2. Posesión - Algo que tienes: token físico, aplicación de autenticación, tarjeta inteligente.
3. Inherencia - Algo que eres: biometría (huella digital, reconocimiento facial, iris).



# SSO: Single Sign On

Mecanismo de autenticación donde se habilita al usuario para que pueda acceder a múltiples sistemas, recursos o aplicaciones con una sola credencial.

1. Mejora la experiencia de usuario: solo tiene que recordar una clave.
2. Reduce las contraseñas débiles o repetidas.
3. Facilita la implementación y la conexión a nuevas fuentes de datos.
4. Aumenta la seguridad, pudiéndose combinar con autenticación multifactor y políticas centralizadas



## JSON Web Token – JWT

Es un estándar abierto (RFC 7519) que se utiliza para transmitir información de forma segura entre dos partes como un objeto JSON.

*Usos: autenticación y autorización*

1. Un usuario inicia sesión.
2. El servidor valida sus credenciales y genera un JWT.
3. Ese token se envía al cliente (navegador, app, etc.).
4. En las siguientes peticiones, el cliente incluye el JWT para probar que está autenticado.

Dividido en 3 partes, separados por puntos: xxxxx.yyyyy.zzzzz

- Header (encabezado): indica el algoritmo de firma y el tipo de token (alg: “HS256” y typ: “JWT”).
- Payload (carga útil): contiene la información o “claims” (reclamaciones), como:
  - ID de usuario.
  - Rol.
  - Fecha de expiración. (**exp**)
  - Cualquier dato relevante (no sensible).
- Signature (firma): se usa para verificar que el token no fue alterado. Se genera con: HMACSHA256(base64UrlEncode(header) + “.” Base64UrlEncode(payload), clave secreta).

Ej.:

eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJzdWIiOiIxMjM0NTY3ODkwIiwibmFtZSI6IkpvaG4gRG9lIiwiaWF0IjoxNTE2MjM5MDIyfQ.SflKxwRJSMeKKF2QT4fwpMeJf36POk6yJV\_adQssw5c

#### + Ventajas:

- Auto contenible: no requiere consultar la base de datos para validar cada petición.
- Fácil de usar con APIs REST.
- Compatible con la mayoría de los lenguajes.

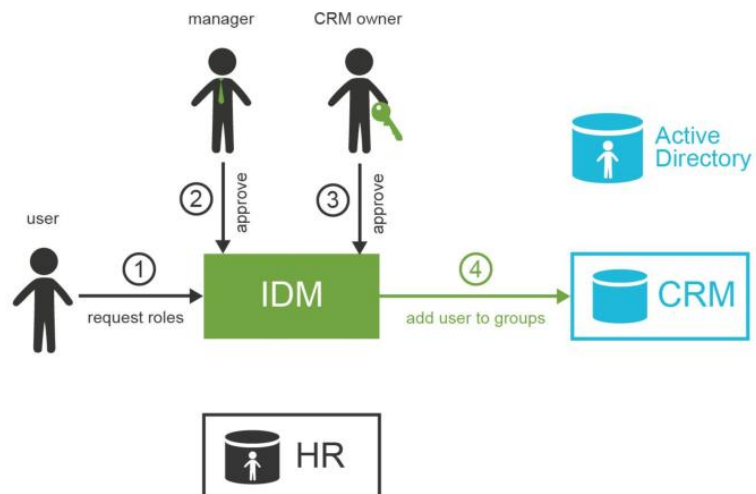
#### Precauciones

- No guardar información sensible (como contraseñas) en el payload: no está cifrado, solo está codificado en base64.
- Usar HTTPS siempre para evitar exposición del token.
- Controlar expiración con exp para limitar el tiempo de uso.

## IDM

El conjunto de políticas y tecnologías que aseguran tanto la gestión de las identidades digitales como el control de su acceso a recursos.

- Reduce el riesgo de accesos no autorizados.
- Controla y limita el acceso según identidad, rol o atributos del usuario.
- Facilita la implementación de autenticación multifactor (MFA) y políticas de contraseñas seguras.



## Modelos de Control de Acceso

Es un aspecto crítico de la seguridad en cualquier sistema. Garantiza que solo los usuarios autorizados puedan acceder a los recursos y realizar acciones.

RBAC (Role Based Access Control): políticas de control de acceso basado en roles. Se asignan roles a los usuarios y son colecciones de permisos sobre recursos específicos.

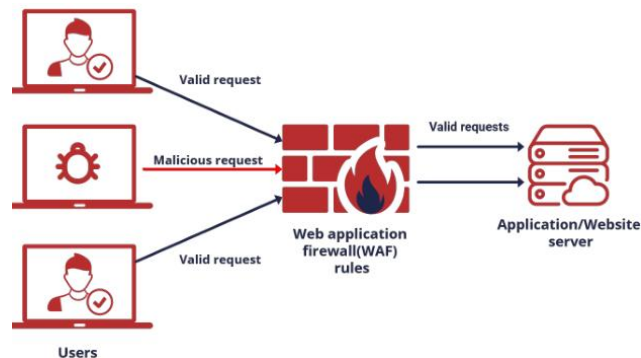
ABAC (Attribute Based Access Control): políticas de control de acceso se basa en los atributos del usuario, recurso, acción y entorno.

| Característica                | RBAC                                                                                                                         | ABAC                                                                           |
|-------------------------------|------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------|
| Política de control de acceso | Basada en roles                                                                                                              | Basada en atributos                                                            |
| Flexibilidad                  | Limitada                                                                                                                     | Altamente flexible                                                             |
| Complejidad                   | Más simple                                                                                                                   | Más complejo                                                                   |
| Impacto en el rendimiento     | Mínimo                                                                                                                       | Puede ser significativo                                                        |
| Gestión de acceso             | Gestión de roles                                                                                                             | Gestión de políticas                                                           |
| Mejor para                    | Estructuras de permisos bien definidas                                                                                       | Control de accesos dinámico y consciente del contexto                          |
| Ejemplos                      | Rol: Gerente, Permiso: Aprobar y ver gastos.<br>Rol: Administrador, Permiso: Acceder y modificar configuraciones del sistema | Política "Puede acceder al documento si pertenece al departamento de sistemas" |

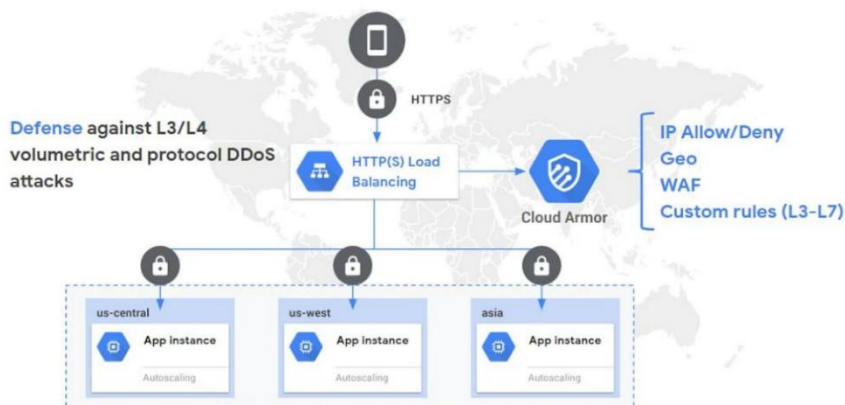
## WAF (Web Application Firewall)

Firewall especializado que protege aplicaciones web filtrando, monitoreando y bloqueando tráfico HTTP/S malicioso entre el usuario y la aplicación.

- Bloquea ataques web comunes, como inyecciones SQL, XSS, CSRF.
- Aumenta la seguridad de la aplicación sin necesidad de modificar el código.
- Facilita el monitoreo y visibilidad de accesos, intentos de ataque y patrones de comportamiento.



## Cloud Armor: DDoS Protection & WAF



## Ejemplos de Componentes

| Categoría                      | Componente                     | Ejemplos Open Source / Comerciales | AWS                                  | Azure                                        | GCP                                |
|--------------------------------|--------------------------------|------------------------------------|--------------------------------------|----------------------------------------------|------------------------------------|
| Balanceo y Alta Disponibilidad | Load Balancer                  | Nginx, HAProxy, Traefik, F5        | Elastic Load Balancer (ALB/NLB)      | Azure Load Balancer / Application Gateway    | Cloud Load Balancing               |
| Rendimiento y Velocidad        | Caché                          | Redis, Memcached, Varnish          | Amazon ElastiCache (Redis/Memcached) | Azure Cache for Redis                        | Memorystore (Redis/Memcached)      |
| Entrega de Contenido           | CDN (Content Delivery Network) | Cloudflare, Akamai, Fastly         | Amazon CloudFront                    | Azure CDN                                    | Cloud CDN                          |
| Autenticación Unificada        | SSO (Single Sign-On)           | Keycloak, Auth0, Okta, OneLogin    | AWS IAM Identity Center (ex AWS SSO) | Azure AD / Entra ID                          | Cloud Identity / Identity Platform |
| Gestión de Identidades         | IDM (Identity Management)      | Keycloak, ForgeRock, Okta          | AWS IAM                              | Azure Active Directory / Entra ID Governance | Google Cloud IAM                   |
| Seguridad en la Capa Web       | WAF (Web Application Firewall) | ModSecurity, Imperva, Barracuda    | AWS WAF                              | Azure Web Application Firewall               | Cloud Armor                        |

## OSWAP – TOP 10

Lista elaborada por la organización Open Worldwide Application Security Project que detalla los 10 principales riesgos de seguridad en aplicaciones web.

1. Pérdida de control de acceso: permitir a usuarios acceder a recursos o funciones fuera de sus permisos.
2. Fallos criptográficos: uso incorrecto o ausencia de mecanismos de cifrado, lo cual termina exponiendo datos sensibles.
3. Inyección: Inyección de código malicioso (como SQL, OS o LDAP) a través de entradas que no contienen validaciones.
4. Diseño inseguro: incluye una serie de vulnerabilidades que se pueden integrar en la arquitectura de una aplicación, como la falta de validación o separación de funciones.
5. Mala configuración de seguridad: como servidores mal configurados o información sensible expuesta.
6. Componentes vulnerables y obsoletos: uso de bibliotecas, frameworks o software con vulnerabilidades conocidas o sin actualizar.
7. Fallos de identificación y autenticación: como contraseñas débiles o sesiones inseguras.
8. Fallos de integridad de software y datos: falta de validación de la integridad de código y datos, permitiendo manipulaciones.
9. Fallos de registro y supervisión de seguridad: ausencia de registros y monitoreo, impidiendo detectar y responder ante ataques.
10. Falsificación de solicitudes del lado del servidor: permite que un atacante haga que el servidor realice solicitudes no autorizadas a otros servicios internos o externos.

# Bounded Context – Contexto Limitado

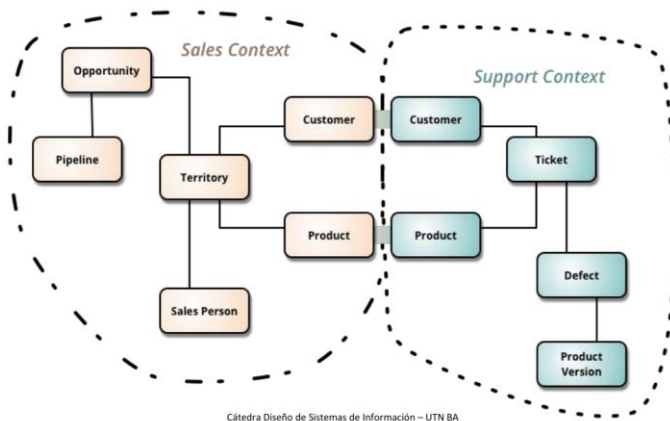
Es el patrón central en el Domain-Driven Design.

Objetivo:

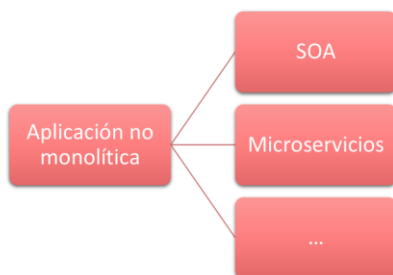
- Dividir sistemas grandes en contextos más pequeños donde cada uno tenga un modelo unificado e interno consistente
- intentar tener un único modelo “unificado” suele generar confusión y contradicciones, porque distintos equipos o partes de la organización usan vocabularios distintos para las mismas ideas

Es importante definir los límites entre contextos y también las relaciones o “mapas de contexto” (cómo se comunican los contextos entre sí, cómo se traducen conceptos entre ellos).

En sistemas legacy, definir un Bounded Context explícito puede ayudar a insertar gradualmente nuevas funcionalidades sin colapsar todo el sistema antiguo.

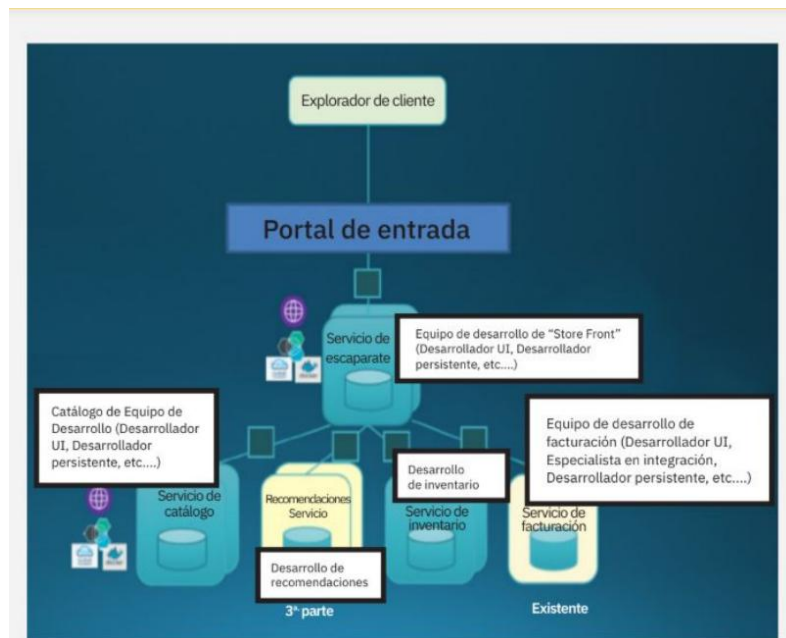
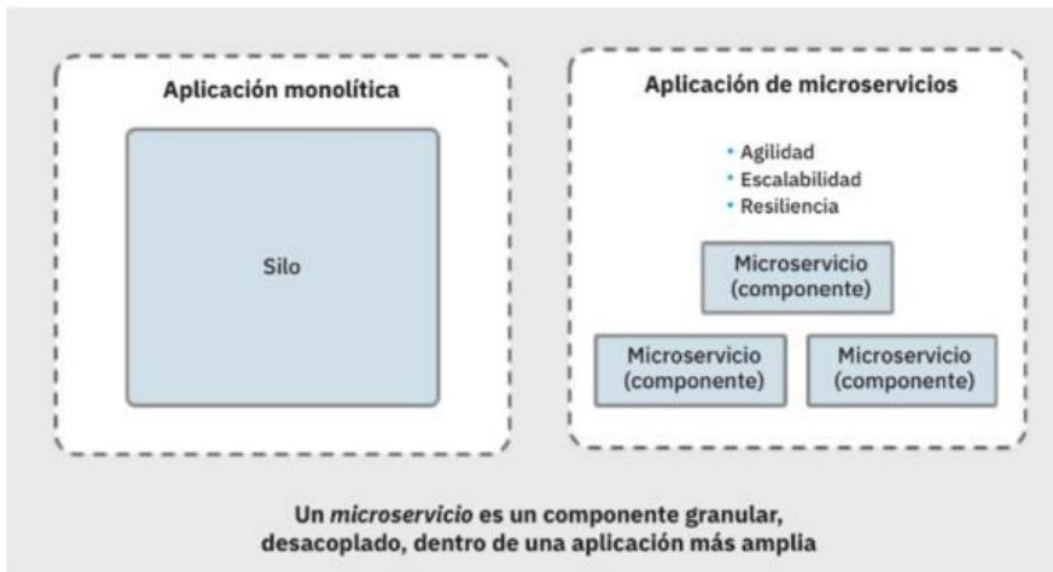


## Aplicación no monolítica





# Microservicios vs Monolítica

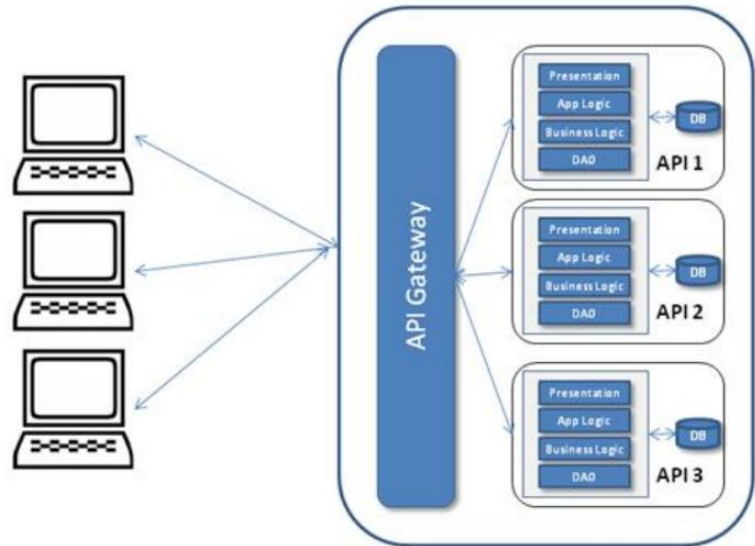


# Microservicios

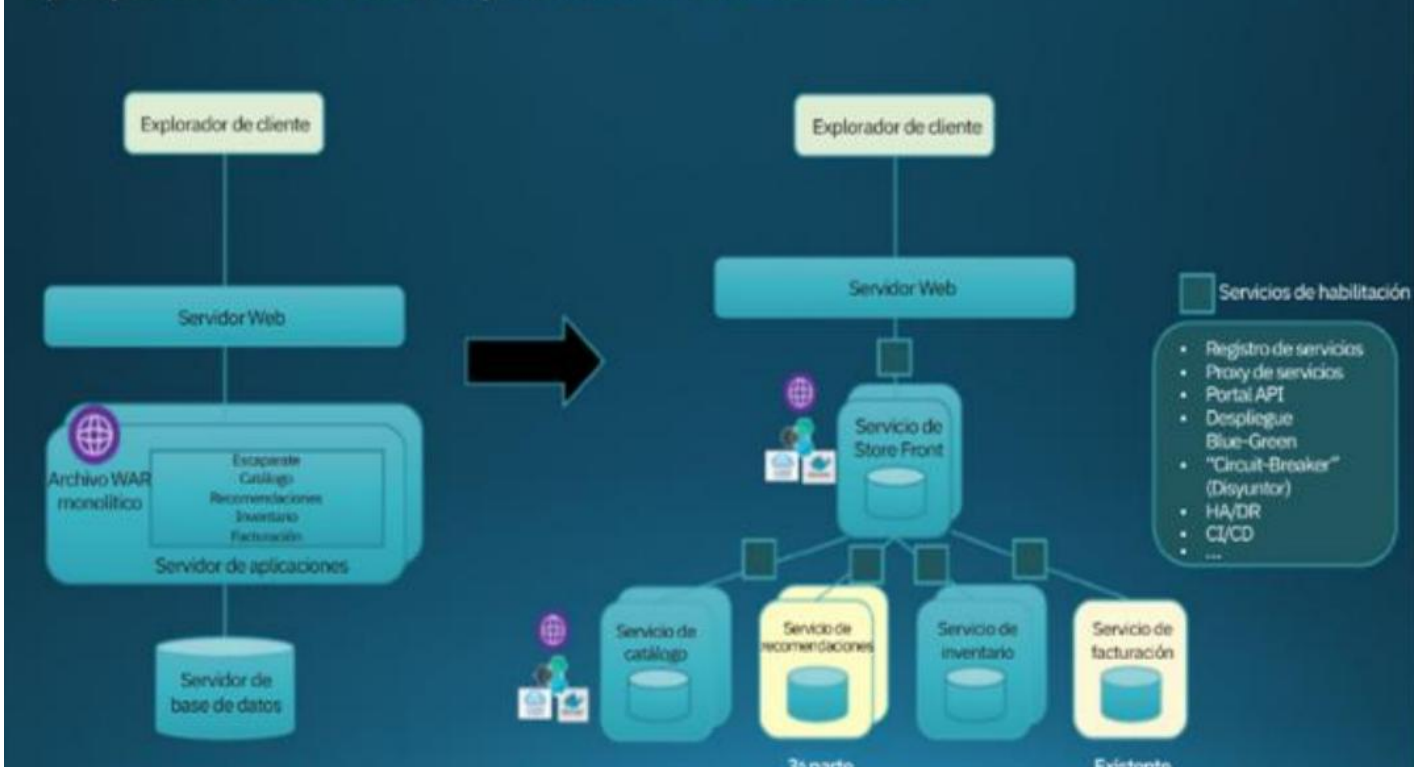
Estilo arquitectónico con enfoque en desarrollar **una única app** como un conjunto de **pequeños servicios**. Con sus propios procesos y comunicación entre ellos.

## + Ventajas:

- Servicios pueden estar integrados con diferentes tecnologías: lenguajes de programación, frameworks y tipos de base de datos.



## Ejemplo: Evolución a una Arquitectura de microservicios



## Características

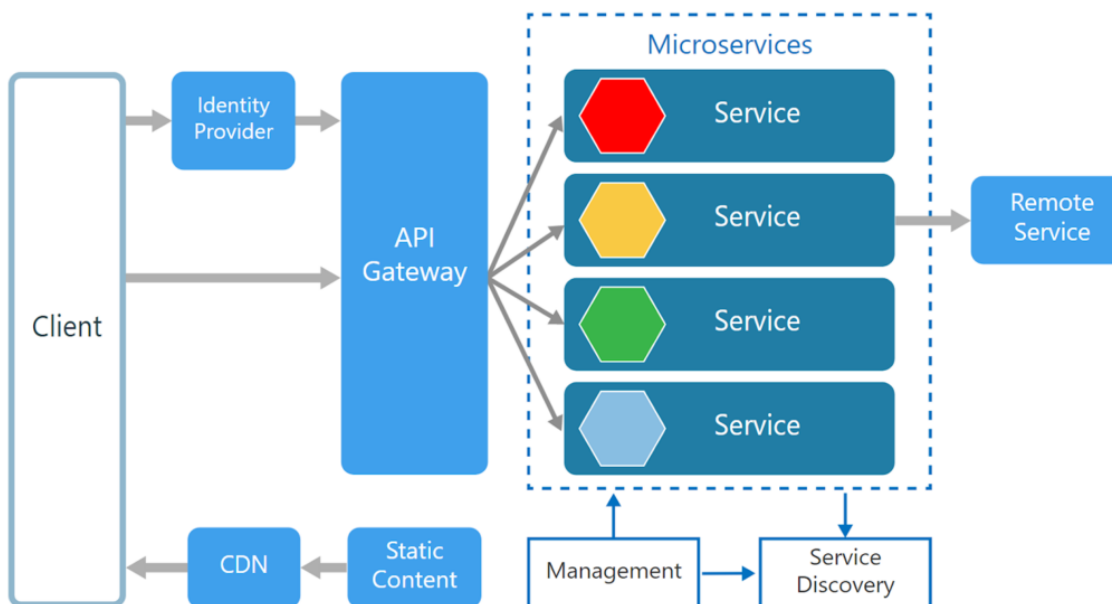
- Servicios chicos e independientes, débilmente acoplados.
- Código base independiente -> posible manejo por un equipo pequeño
- Responsables de conservar sus propios datos o estado externo.
- Comunicación mediante APIs con interfaces bien definidas.

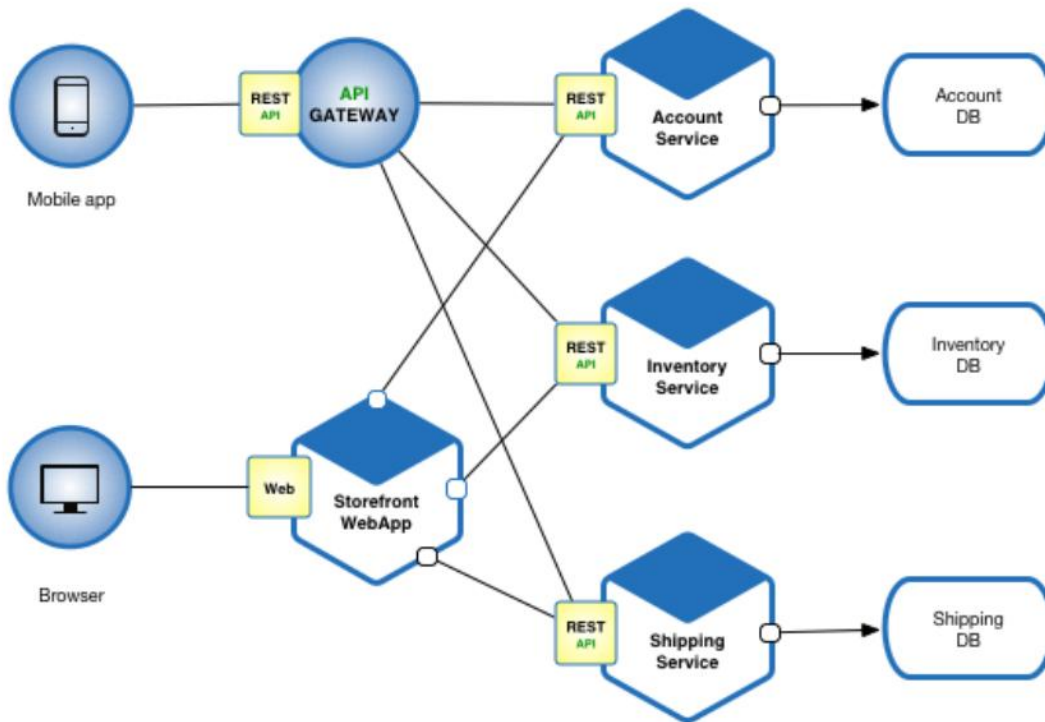
### Consideraciones para el desarrollo:

- Un servicio por contenedor: separar grandes monolitos en muchos servicios pequeños.
- Optimizar los servicios para una función única: única función empresarial por servicio para que sea fácil de escribir y mantener (Principio de responsabilidad única)
- Comunicarse a través de las API REST y los buses de mensajes
- Aplicar CI/CD (Continuous Integration/Continuous Development): los diferentes servicios evolucionan a velocidades diferentes donde cada servicio tiene una única y continua tubería de integración/entrega.
- Aplicar decisiones de alto disponibilidad (HA) / “clustering” por servicio: se puede ampliar recursos por servicio y no al conjunto entero.

### ¿Cuándo utilizarlos?

- Aplicaciones grandes que requieren una alta velocidad de publicación.
- Aplicaciones complejas que necesitan gran escalabilidad.
- Aplicaciones que requieren escalar de forma dinámica y es difícil predecir cuándo sucederá el evento que lo requiere.
- Aplicaciones requieren crecer rápidamente a nivel funcional y estos cambios son sostenidos en el tiempo.
- Aplicaciones con dominios complejos o muchos subdominios.
- Una organización que disponga de pequeños equipos de desarrollo.



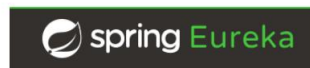


### Componentes Necesarios

- Administración: responsable de la colocación de servicios en los nodos, la identificación de errores, el reequilibrio de servicios entre nodos, etc.
- Detección de servicios: lista de servicios y los nodos en que se encuentran.
- API Gateway: punto de entrada para los clientes para conectarse con la lógica del backend. Se puede encargar del registro, terminación SSL y equilibrio de carga

### Componentes concretos

- Service Registry y Discovery



- Circuit Breaker



HYSTRIX

- API Gateway



- Observabilidad y Gestión

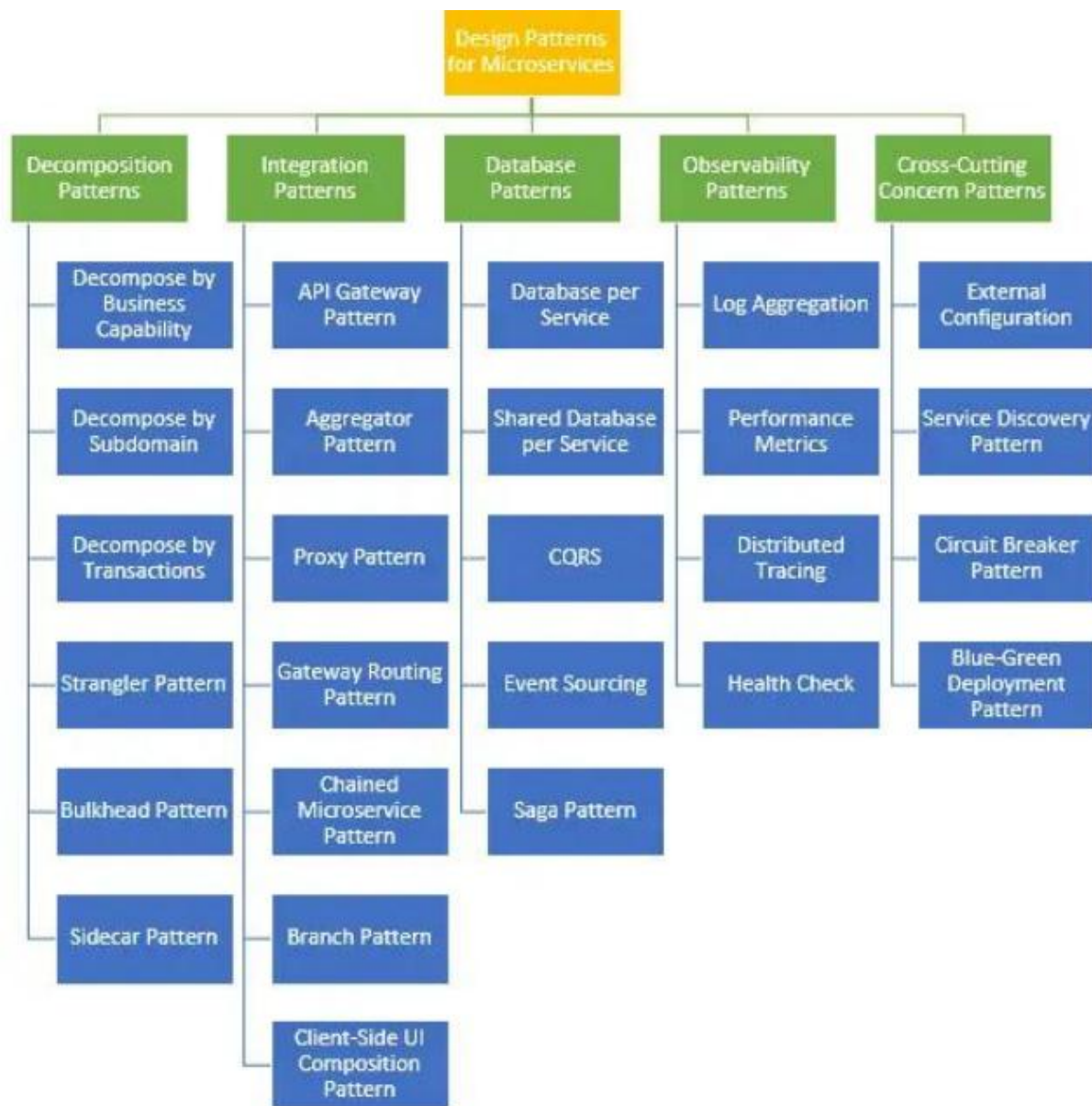


- Orquestador

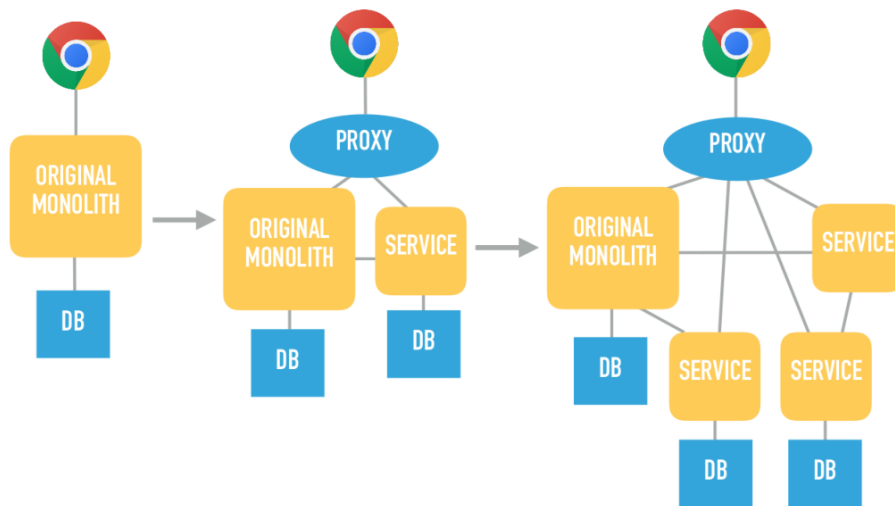


Cátedra Diseño de Sistemas de Información – UTN BA

## Algunos patrones



- El Patrón fachada define una interfaz a través de una API externa específica para un sistema o subsistema.
- Los Patrones de Entidad (Entity) y Agregado (Aggregate) son útiles para identificar conceptos específicos de la empresa que se relacionan directamente con microservicio.
- El Patrón de Servicios ofrece una manera de mapear operaciones que no corresponden a una sola entidad o agregado en el enfoque basado en entidad que se requiere para los microservicios.
- El Patrón Adaptador de Microservicios (Microservicios adaptadores) adapta entre dos API diferentes (conceptualmente). Se adapta una API heredada o servicio tradicional SOAP a una API RESTful o técnicas de mensajería ligeras.
- El Patrón Strangler (estrangulador) aborda el hecho de realizar una reingeniería de una aplicación monolítica y orientarla a microservicios. El patrón brinda un enfoque para gestionar la refactorización y avanzar la migración.

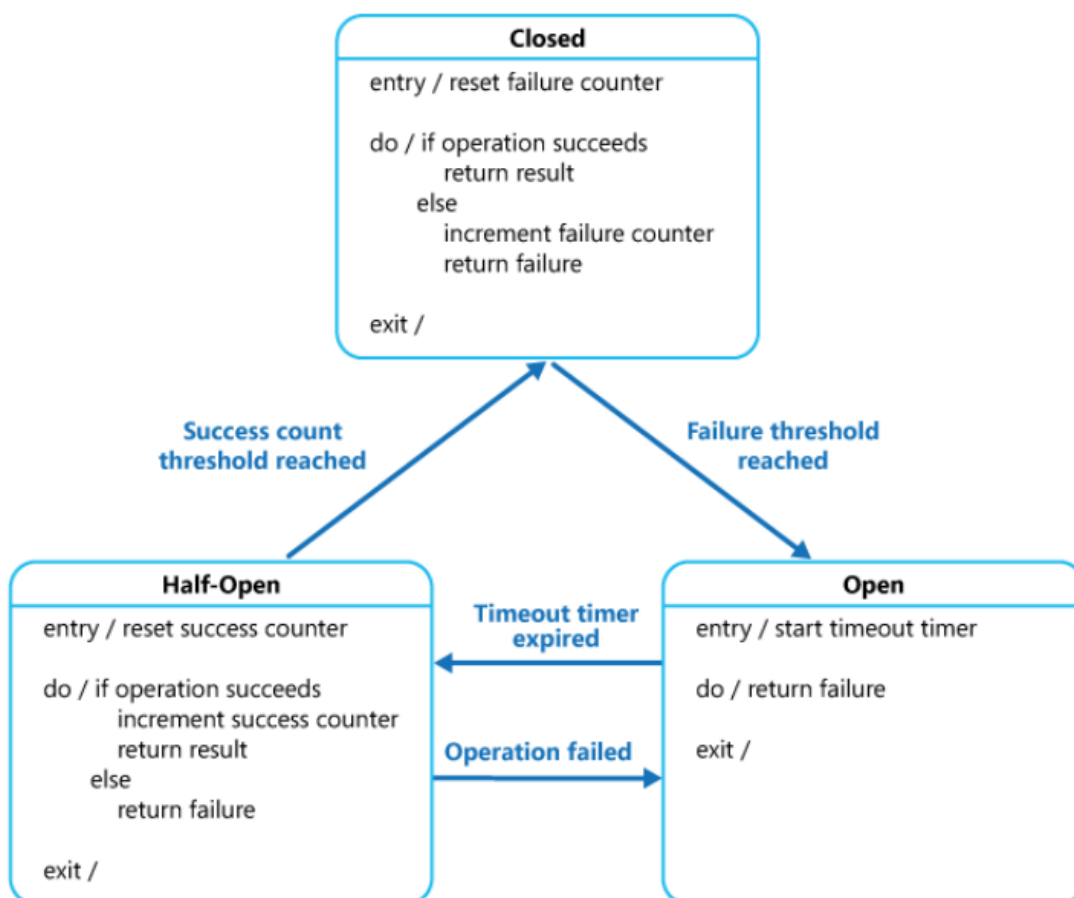


(strangler)

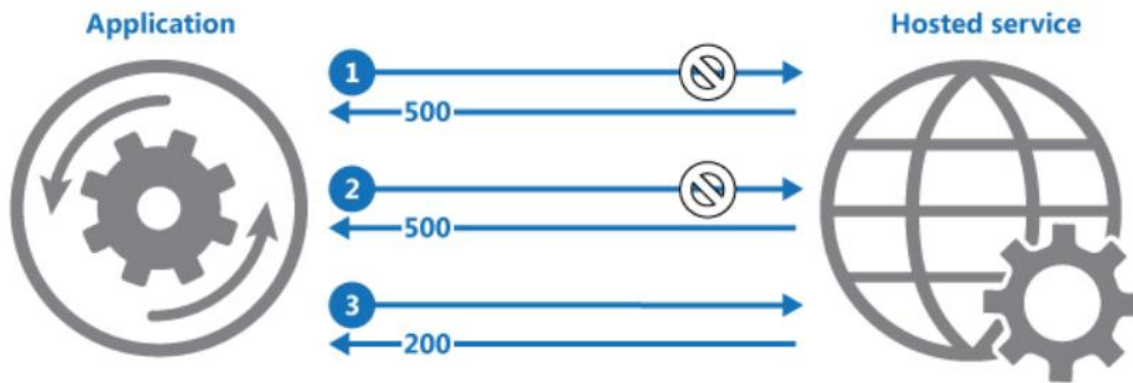
## Patrones de operaciones

- El Patrón “Service Registry” (Registro de servicio) está vinculado a identificar servicios y poder intercambiarlos de manera ágil.
- Los Patrones “Correlation ID” y “Log Aggregator” resuelven problemas de trazabilidad, descubrimiento y seguimiento de errores.
- El Patrón “Circuit Breaker” está vinculado a detectar y cortar el llamado a servicios que están funcionando de manera incorrecta.

## Patrones: Circuit Breaker y Retry.



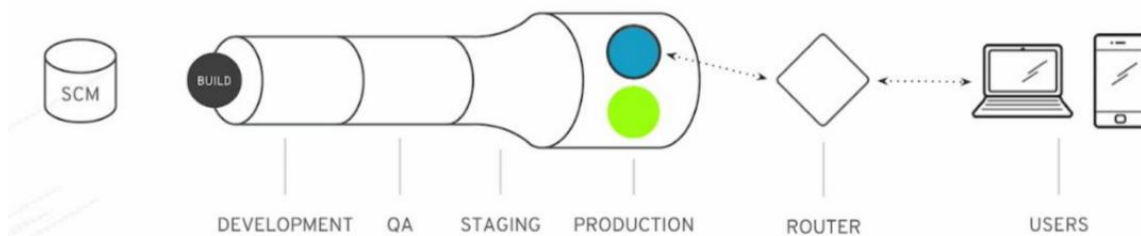




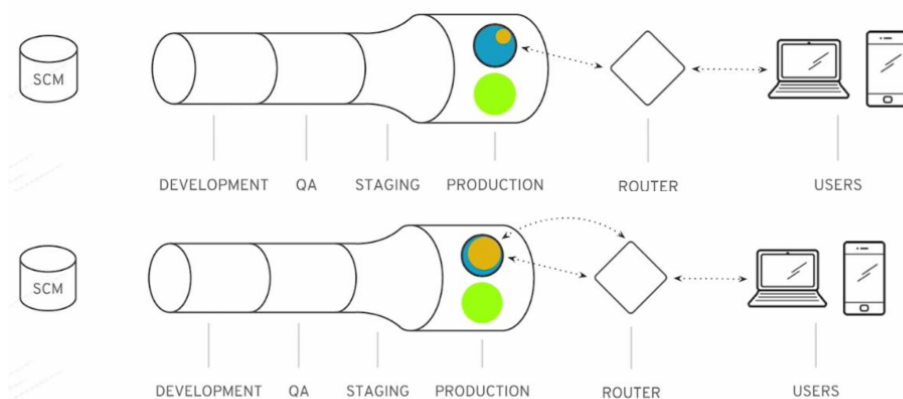
- 1: Application invokes operation on hosted service. The request fails, and the service host responds with HTTP response code 500 (internal server error).
- 2: Application waits for a short interval and tries again. The request still fails with HTTP response code 500.
- 3: Application waits for a longer interval and tries again. The request succeeds with HTTP response code 200 (OK).

## Despliegue de microservicios

# Blue/Green Deployment



# Canary Deployment



## ¿Cómo se comunicación entre sí?

La forma en que se gestiona esta comunicación determina aspectos clave como la escalabilidad, trazabilidad y resiliencia del sistema.

Existen dos patrones principales para esta coordinación:

- Orquestación: un componente central decide quién hace qué y cuándo.
- Coreografía: los servicios actúan en base a eventos sin un controlador central.

## Orquestación de Microservicios

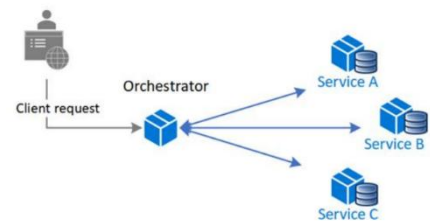
El orquestador administra el flujo de trabajo general donde invoca cada microservicio con una secuencia y control de cada acción. El servicio hace lo que se le pide, sin necesidad de conocer el flujo general.

### + Ventajas:

- Fácil de visualizar el flujo completo.
- Control centralizado permite mayor gobernanza.
- Ideal para flujos de negocio complejos y críticos.

### + Desventajas:

- Introduce un punto único de fallo y mayor acoplamiento.
- El orquestador puede volverse un cuello de botella.
- Menor flexibilidad para cambios dinámicos.



### Ejemplo:

- Un usuario solicita la creación de una cuenta.
- Se envía la solicitud al **orquestador**.
- El orquestador llama al **servicio de validación de identidad**, espera la respuesta.
- Luego, llama al **servicio de creación de usuario**, y finalmente al **servicio de notificación** para enviar un correo de bienvenida.
- El orquestador espera y coordina cada paso antes de finalizar el flujo.

### Tecnologías:

- Camunda, Apache Airflow, BPM engines

## Coreografía de Microservicios

No hay controlador central, los servicios se comunican mediante eventos. Cada servicio sabe cuándo debe actuar y publica eventos cuando cambia el estado. Otros servicios están suscritos a estos eventos y reacciones en consecuencia.

### + Ventajas:

- Alta escalabilidad y resiliencia.
- Menor acoplamiento: los servicios pueden evolucionar de forma independiente.
- Fácil de extender con nuevos servicios sin modificar los existentes.

### - Desventajas:

- Más difícil de entender y depurar el flujo completo.



- Requiere buena gobernanza de eventos y contratos.
- Dificultad para mantener consistencia transaccional (requiere patrones como SAGA).

**Ejemplo:**

- Un usuario realiza una compra desde la aplicación web.
- El **servicio de pedidos** publica el evento "PedidoCreado".
- El **servicio de inventario** escucha ese evento y descuenta stock disponible.
- Luego, el **servicio de facturación** detecta el evento y genera la factura.
- Finalmente, el **servicio de notificación** escucha "FacturaGenerada" y envía un correo al cliente.
- Cada servicio actúa de forma autónoma, reaccionando a eventos, sin coordinación central.

*Tecnologías:*

- Apache Kafka, RabbitMQ, AWS EventBridge

## Resumen de comunicación

| Característica                     | Orquestación                              | Coreografía                                 |
|------------------------------------|-------------------------------------------|---------------------------------------------|
| <b>Control del flujo</b>           | Centralizado (orquestador)                | Distribuido entre servicios                 |
| <b>Tipo de comunicación</b>        | Predominantemente sincrónica              | Predominantemente asincrónica               |
| <b>Acoplamiento</b>                | Mayor acoplamiento                        | Bajo acoplamiento                           |
| <b>Escalabilidad</b>               | Limitada por el orquestador               | Alta escalabilidad                          |
| <b>Trazabilidad del flujo</b>      | Alta (flujo claro y centralizado)         | Baja (flujo distribuido y dinámico)         |
| <b>Facilidad de implementación</b> | Más sencilla en flujos controlados        | Requiere mayor diseño y gobernanza          |
| <b>Manejo de errores</b>           | Centralizado y predecible                 | Distribuido, más complejo                   |
| <b>Casos de uso recomendados</b>   | Flujos de negocio complejos, regulatorios | Sistemas altamente distribuidos y dinámicos |

**Desafíos**

- Complejidad: el todo es complejo
- Desarrollo y pruebas: refactorización en los límites del servicio y dependencias es difícil de especificar.
- Falta de gobernanza: puede llevar a problemas de mantenibilidad.
- Congestión y latencia de red.
- Integridad de datos: cada servicio es responsable de sus propios datos, donde la coherencia puede ser un problema.
- Administración y uso correcto por parte de los DevOps
- Control de versiones: las actualizaciones de un servicio no deben interrumpir sus dependencias.
- Serie de aptitudes: los desarrolladores deben tener los conocimientos y experiencia para diseñar y orquestar los microservicios.

# Documentación de la Arquitectura

¿Por qué documentar una Arquitectura?

- Es el “contenedor” de los Atributos de Calidad.
- Es un buen artefacto para realizar Análisis tempranos.
- Define las Asignaciones de trabajo del Proyecto.
- Ayuda con el mantenimiento Post-Deployment.

## Modelo 4+1



Pilippe (Philippe) Kruchten define: “Una vista es una presentación de un modelo, la cual es una descripción completa de un sistema desde una perspectiva particular”.

### Vista Lógica:

Requisitos funcionales del Sistema y de lo que el Sistema debe hacer, las funciones y servicios que se han definido.

Enfocado a lo definido como el dominio de la aplicación, lo que son las clases y objetos principales que formarán el “core” de la aplicación.

Esta vista se complementa con los Diagramas UML:

- Diagramas de Clases
- Diagrama de Paquetes

### Vista Despliegue:

Mostrar básicamente como está dividido nuestro Sistema de software en componentes, y las dependencias entre los componentes.

Muestra los componentes como: archivos, bibliotecas, módulos, ejecutables o paquetes. Muestra la dependencia y comunicación entre Ellos.

Esta vista se complementa con los Diagramas UML:

- Diagramas de Componentes
- Diagrama de Paquetes

## Vista de Procesos:

Representa los flujos de trabajo de trabajo y de negocio operacionales, paso a paso, de los componentes que conforman el Sistema. También puede involucrar algunos requisitos no funcionales.

Esta vista se complementa con los Diagramas UML:

- Diagrama de Actividad

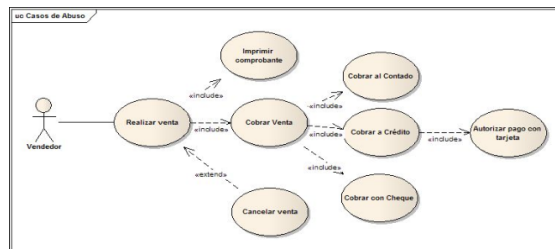
## Vista Física:

Representa cómo están distribuidos los componentes entre los distintos nodos que conforman la solución incluyendo servicios.

Esta vista se complementa con los Diagramas UML:

- Diagrama de Despliegue

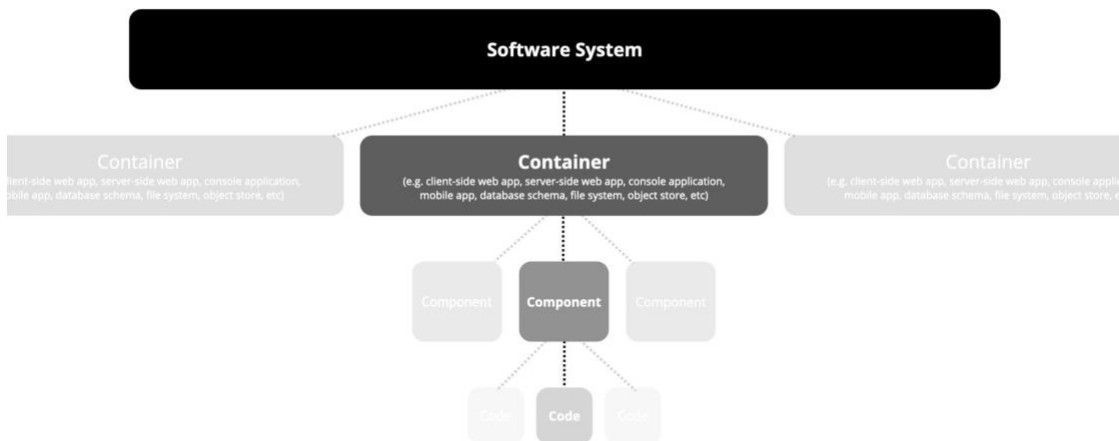
+1:



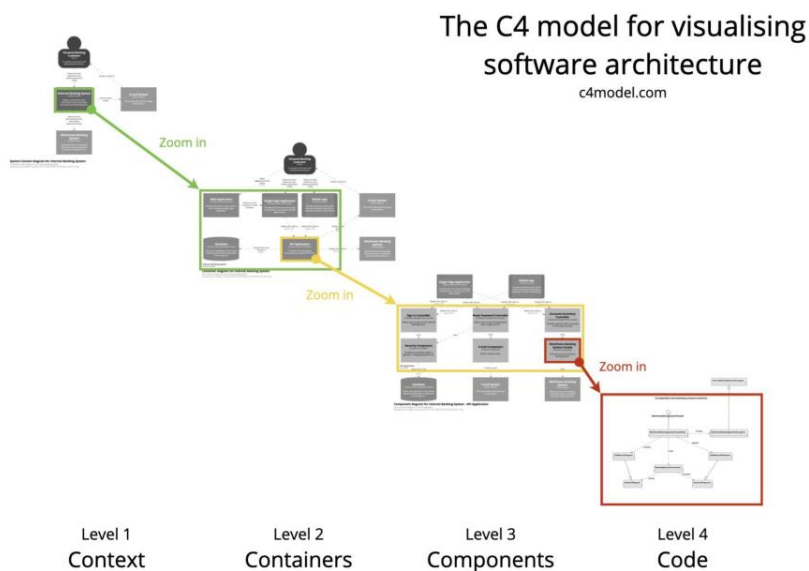
- Esta vista es representada por los casos de uso.
- Tiene la función de unir y relacionar las otras 4 vistas.
- Trazabilidad de componentes, clases, equipos, paquetes, etc., para realizar cada caso de uso.

## Modelo C4

- Busca aliviar la brecha entre modelo y código.
- Permite comunicar la arquitectura de un Sistema en función de niveles de detalle
- Basado en cuatro niveles que describen el Sistema con distintos grados de granularidad
  - Nivel 1: Nivel de Contexto
  - Nivel 2: Nivel de Contenedores
  - Nivel 3: Nivel de Componentes
  - Nivel 4: Nivel del Código

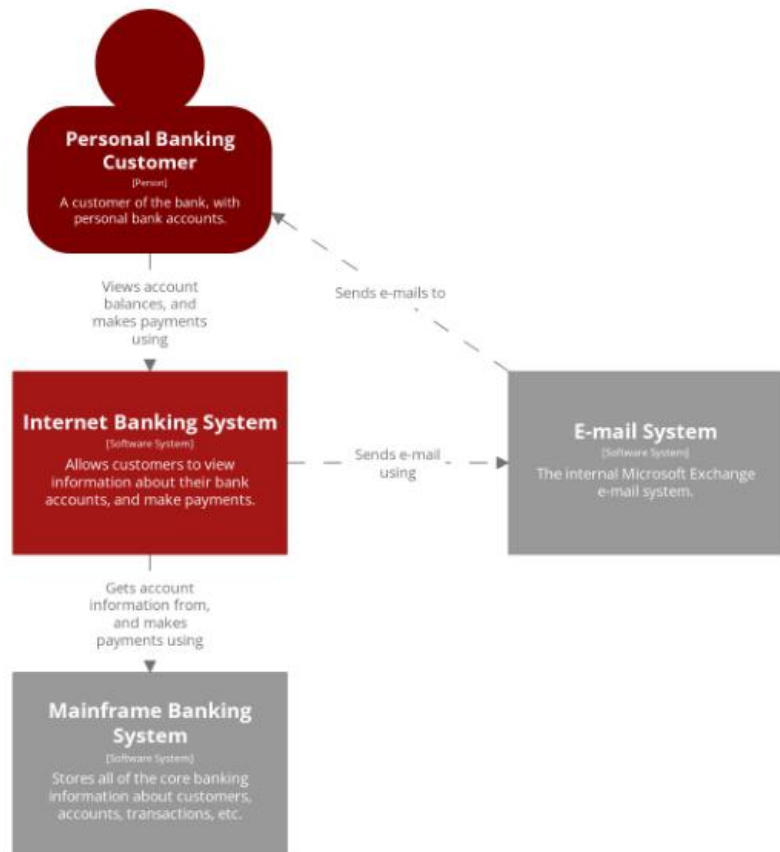


A **software system** is made up of one or more **containers** (applications and data stores), each of which contains one or more **components**, which in turn are implemented by one or more **code** elements (classes, interfaces, objects, functions, etc).



## Nivel 1 – Contexto:

- Permite tener una imagen genérica de nuestro sistema y sus interacciones con el exterior.
- Podemos especificar los Sistemas externos con los que interactúa nuestro propio sistema.
- El Sistema que modelamos se considera una “caja negra”, solo nos interesan sus relaciones externas
- Permite identificar los usuarios finales que harán uso de la funcionalidad de nuestro software.

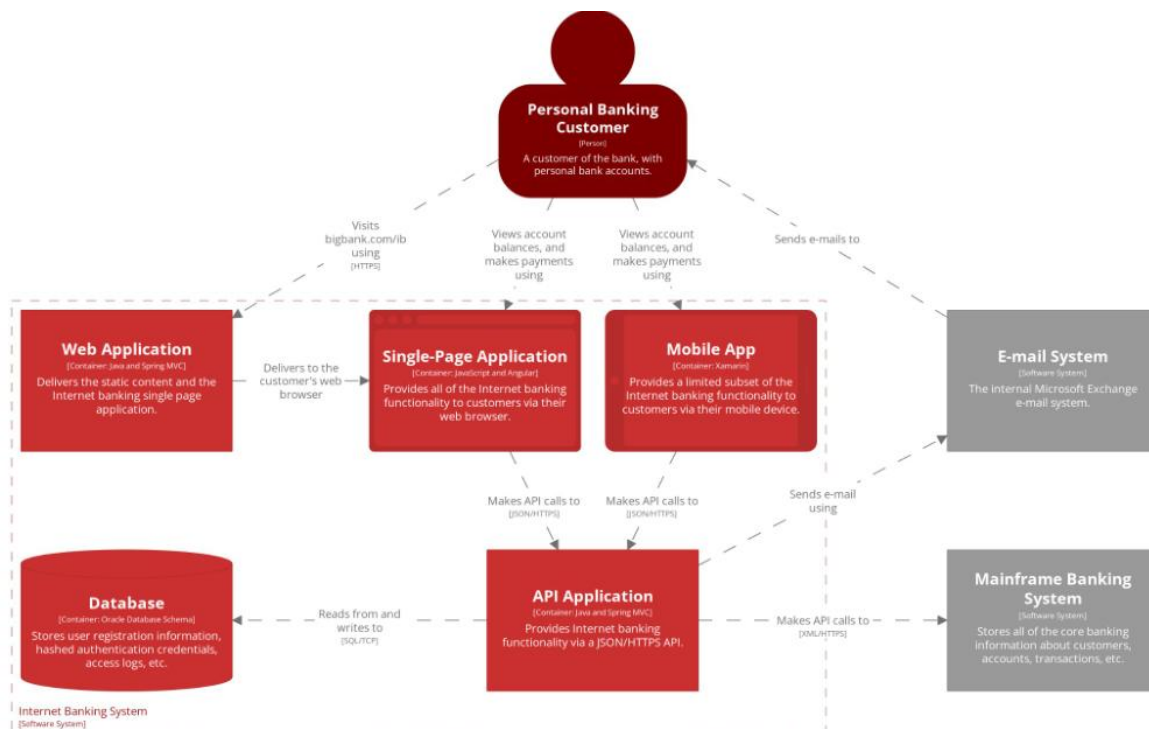


## Nivel 2 – Contenedores

Los contenedores referencian

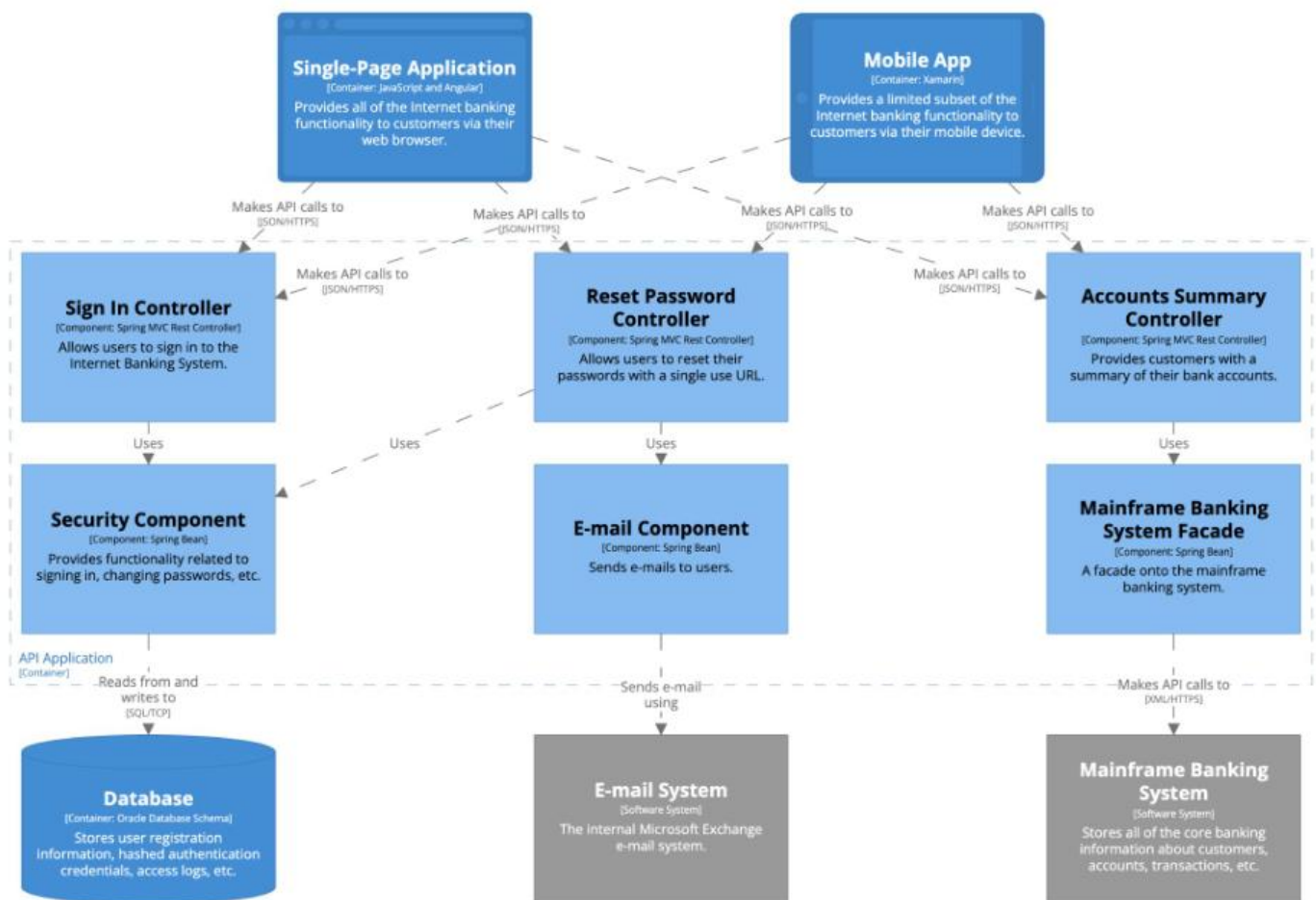
cualquier entidad que ejecuta código o almacena datos. Ejemplos de contenedores:

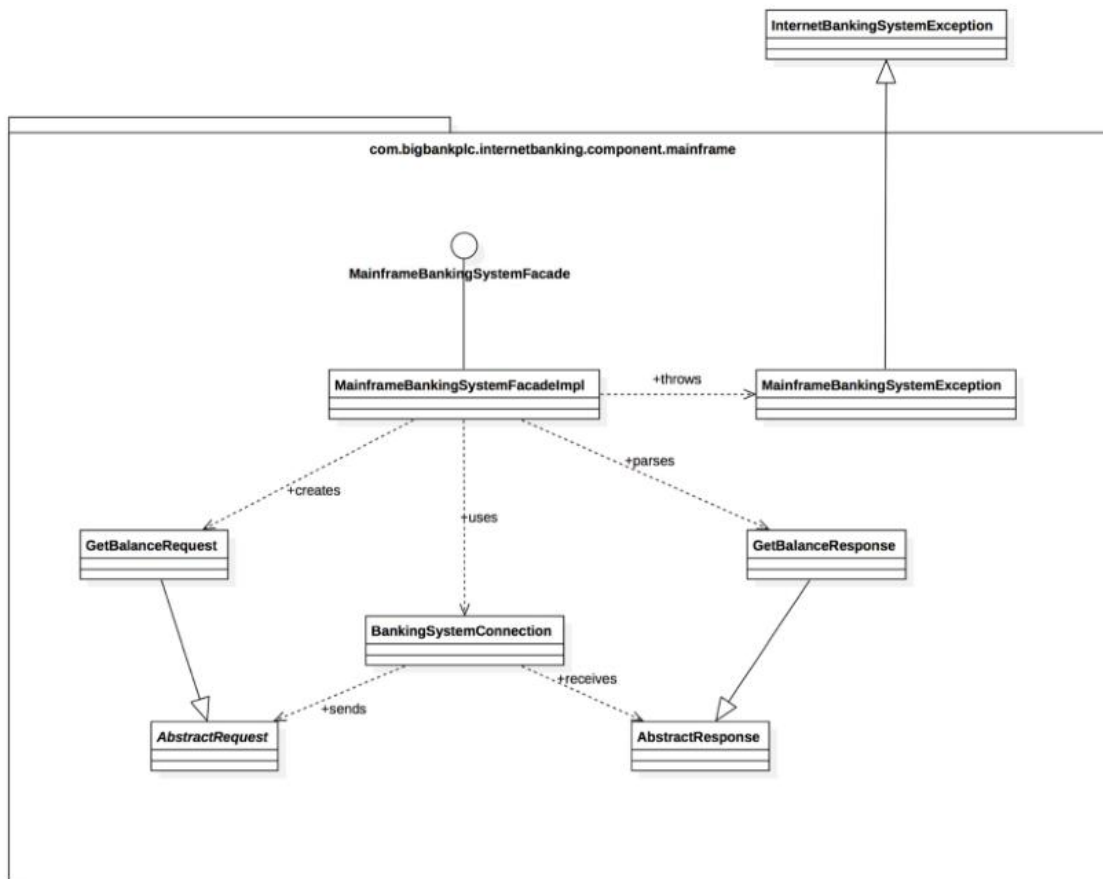
- Aplicaciones web
- Servicios web
- Aplicaciones de escritorio
- Bases de datos
- Sistema de ficheros



## Nivel 3 – Componentes

- Detalla un componente en particular manteniendo el contexto.
- Cada componente posee un grupo de funcionalidades
- Muestra la responsabilidad del componente y detalles de su implementación

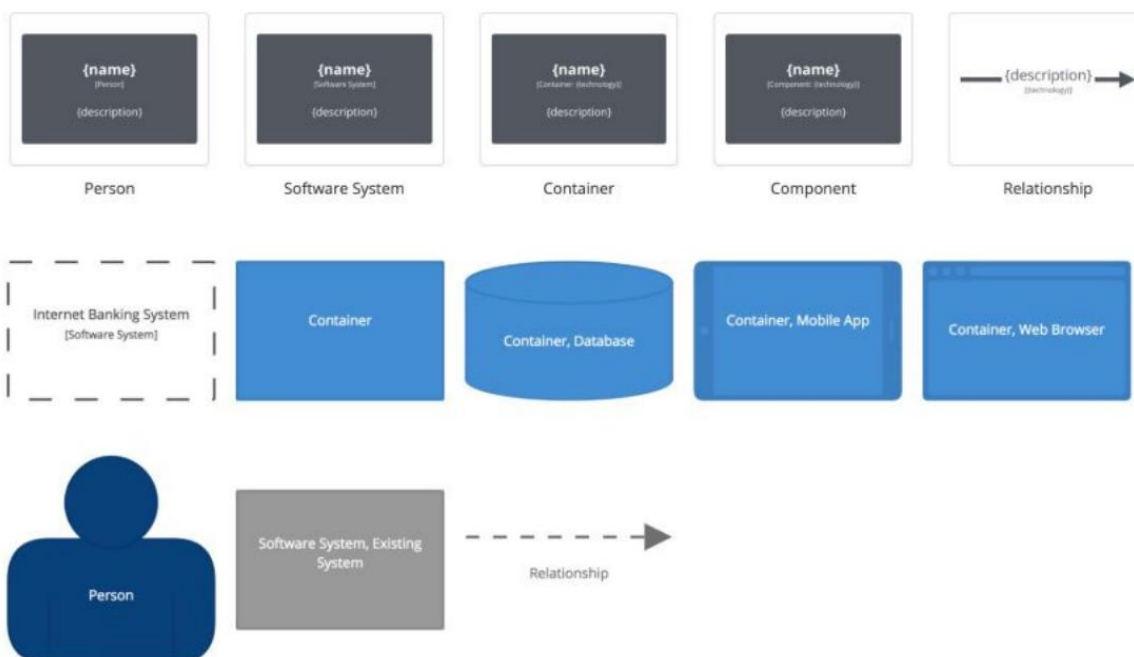




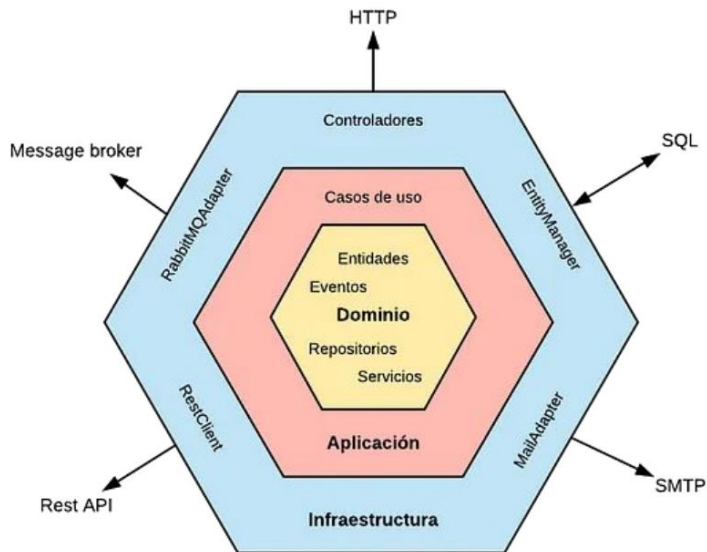
## Nivel 4 – Código

- Se muestran detalles de implementación.
- Diagramas: Clases, Modelo de Datos, Secuencia, etc.

## Notación



# Arquitectura Hexagonal



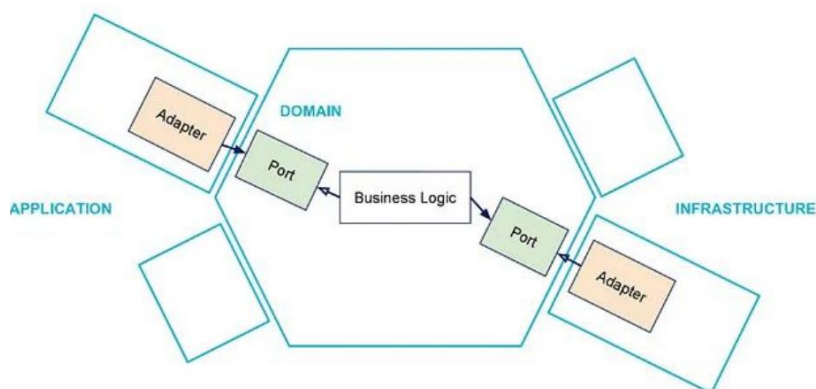
## Objetivo:

- Permitir que la lógica de negocio de una aplicación sea usada de la misma forma por usuarios, otros programas, pruebas automatizadas o scripts.
- Permitir desarrollar y probar de forma aislada de sus eventuales dispositivos y bases de datos en tiempo de ejecución.

## Motivación:

- “Infiltración” de Lógica de Negocio en la Interfaz de Usuario
- Alto Acoplamiento con Medios Persistentes (Base de Datos)
- Dificultad de Pruebas Automatizadas

La Arquitectura Hexagonal propone que nuestro dominio sea el núcleo de las capas y que este no se acople a nada externo. En lugar de hacer uso explícito y mediante el principio de inversión de dependencias nos acoplamos a contratos (interfaces o puertos) y no a implementaciones concretas.



Se propone que nuestro núcleo de dominio sea visto como una API con contratos bien especificados.



Se definen puertos o puntos de entrada e interfaces (adaptadores) para que otros módulos (UI, BBDD, Test) puedan implementarlas y comunicarse con la capa de negocio sin que ésta deba saber el origen de la conexión.

Esto es lo llamado puertos y adaptadores, que podrían ser definidos de la siguiente manera:

- Puerto: definición de una interfaz pública.
- Adapter: especialización de un puerto para un contexto concreto.

## Integración de Sistemas

### ¿Qué es integrar sistemas?

- Integrar sistemas es establecer una comunicación entre ellos.
- Es compartir datos y funcionalidades entre dos o más sistemas.
- Es abrir el universo “cerrado” sobre el que fue pensado y diseñado un sistema.
- Es aumentar la cohesión de un sistema evitando duplicar lógicas existentes.

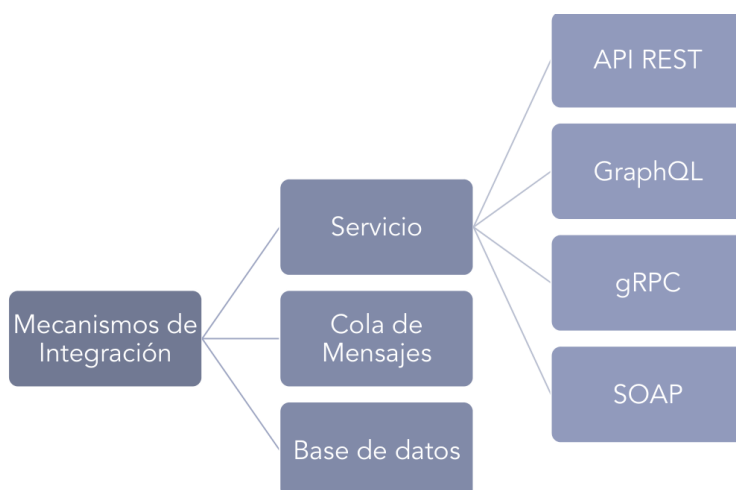
### Necesidades de Integración

- Los sistemas, generalmente, no son aislados y necesitan de otros para subsistir.
- Existen lógicas que ya están creadas que resuelven ciertas tareas.
- Los sistemas deben compartir datos entre sí y no desean “copiarlos” sino “consultarlos”.
- Necesidades vinculadas al negocio.
- Necesidades vinculadas al marco legal.

### ¿Sincrónica o Asincrónica?

Esta decisión dependerá exclusivamente de las necesidades del negocio y/o restricciones técnicas.

### Mecanismos de Integración



## Web Service:

Un servicio web es un término genérico para un componente de software interoperable de máquina a máquina que está alojado en una ubicación direccionable de red.

- Posee una interfaz
- Oculta los detalles de implementación para que se pueda utilizar independientemente de la plataforma y del stack tecnológico.
- Se busca implementaciones de tecnología cruzada, orientadas a componentes y poco acopladas.
- Pueden utilizar solos o con otros servicios web para realizar una agregación compleja o una transacción comercial.

## API:

Una interfaz de programación de aplicaciones (API) es un conjunto de herramientas, definiciones y protocolos que se usa para diseñar e integrar aplicaciones.

Permite que un producto o servicio se comuniquen con otros productos y servicios, sin la necesidad de saber cómo se implementan internamente.

## Tipos de API (Arquitectura)

### •Local

- Proporciona Servicios de Sistema Operativo, Bibliotecas o Middleware.

### •Web

- Proporciona Servicios entre Componentes Distribuidos y en general sobre el protocolo HTTP.

### •Programa

- Proporciona la posibilidad de Ejecución de Código de forma Remota entre Componentes Distribuidos (RPC).

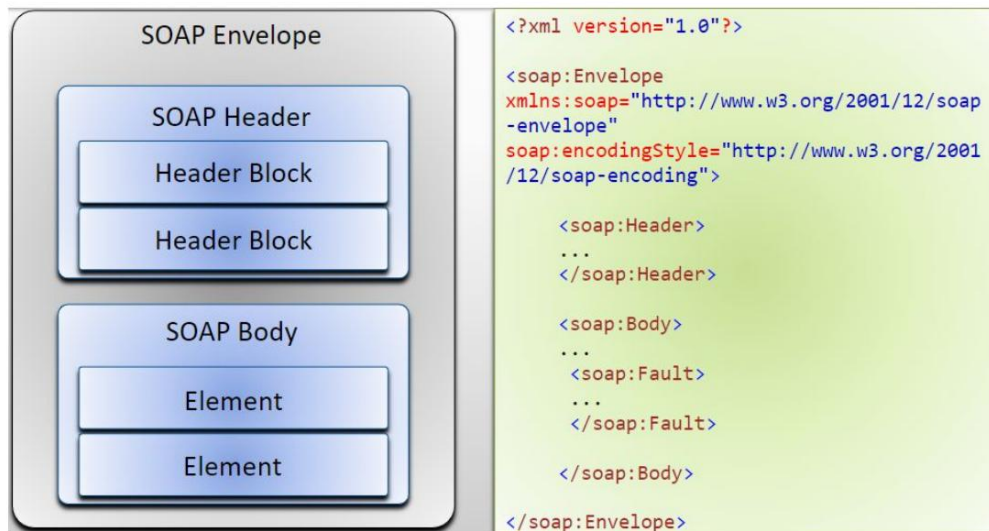
## Tipos de API (alcance)

- Public
- Partner (para terceros)
- Private (interna)

# Tipos de API (Web)

- REST
- SOAP
- gRPC
- GraphQL

## SOAP



### Solicitud

```
<soapenv:Envelope xmlns:soapenv="http://schemas.xmlsoap.org/soap/envelope/">
  <soap:Header xmlns:soap="http://schemas.xmlsoap.org/soap/envelope/">
  </soap:Header>
  <soapenv:Body>
    <aplicarVentaRequest xmlns="http://elements.blogsoa.com/Productos">
      <codigoProducto>1KL979GY</codigoProducto>
      <cantidadProducto>2</cantidadProducto>
      <almacen>Centro</almacen>
      <precioTotal>7600.72</precioTotal>
    </aplicarVentaRequest>
  </soapenv:Body>
</soapenv:Envelope>
```

### Respuesta

```
<soapenv:Envelope xmlns:soapenv="http://schemas.xmlsoap.org/soap/envelope/">
  <soap:Header xmlns:soap="http://schemas.xmlsoap.org/soap/envelope/">
  </soap:Header>
  <soapenv:Body>
    <aplicarVentaResponse xmlns="http://elements.blogsoa.com/Productos">
      <codigo>0</codigo>
      <descripcion>La venta se ha registrado exitosamente</descripcion>
      <numeroOrden>356-MNC298</numeroOrden>
    </aplicarVentaResponse>
  </soapenv:Body>
</soapenv:Envelope>
```

## API REST VS API SOAP

| Característica           | REST                    | SOAP                   |
|--------------------------|-------------------------|------------------------|
| Formato de Intercambio   | XML / JSON              | XML                    |
| Llamada/Respuesta        | Sincrónica              | Asincrónica/Sincrónica |
| Interfaz                 | Fijas y Simples         | Dinámicas y Complejas  |
| Descripción del Servicio | Documentación           | WSDL                   |
| Acoplamiento             | Bajo                    | Alto                   |
| Protocolos               | HTTP                    | HTTP / SMTP / TCP      |
| Seguridad                | HTTPS                   | WS-Security            |
| Manejo de Estados        | Sin Estados (Stateless) | Con Estado             |
| Tipo de Aplicación       | Web - Mobile            | Enterprise             |
| Público                  | General                 | Específico             |

### SOAP – WSDL

En el archivo **wSDL** se describe todo el **servicio web**, las **operaciones disponibles** y las **estructuras de datos** de los **mensajes** de cada **operación**(xsd).

```
1 <?xml version="1.0" encoding="UTF-8"?>
2 <wsi:definitions name="Productos"
3   targetNamespace="http://services.blogsoa.com/Productos"
4   xmlns:client="http://services.blogsoa.com/Productos"
5   xmlns:xsd="http://www.w3.org/2001/XMLSchema"
6   xmlns:prod="http://elements.blogsoa.com/Productos"
7   xmlns:soap="http://schemas.xmlsoap.org/wsdl/soap/"
8   xmlns:wsi="http://schemas.xmlsoap.org/wsdl/"
9
10 <wsi:types>
11   <schema xmlns="http://www.w3.org/2001/XMLSchema"
12     <import namespace="http://elements.blogsoa.com/Productos" schemaLocation="../../XSD/productos.xsd"/>
13   </schema>
14 </wsi:types>
15
16 <!-- Consultar Productos Message -->
17 <wsi:message name="consultarProductosRequest">
18   <wsi:part name="parameters" element="prod:consultarProductosRequest"/>
19 </wsi:message>
20 <wsi:message name="consultarProductosResponse">
21   <wsi:part name="parameters" element="prod:consultarProductosResponse"/>
22 </wsi:message>
23
```

### SOAP – XSD

En el **archivo xsd** se describen las **estructuras de datos** de los **mensajes** de cada **operación**.

```
1 <?xml version="1.0" encoding="UTF-8"?>
2 <xsd:schema
3   elementFormDefault="qualified"
4   targetNamespace="http://elements.blogsoa.com/Productos"
5   xmlns:tipos="http://elements.blogsoa.com/Productos"
6   xmlns:xsd="http://www.w3.org/2001/XMLSchema" >
7
8   <!-- Consultar la lista de Productos disponibles -->
9   <xsd:element name="consultarProductosRequest" type="tipos:ConsultarProductosRequest"/>
10  <xsd:element name="consultarProductosResponse" type="tipos:ConsultarProductosResponse"/>
11
12  <!-- Aplicar Venta de un Producto -->
13  <xsd:element name="aplicarVentaRequest" type="tipos:AplicarVentaRequest"/>
14  <xsd:element name="aplicarVentaResponse" type="tipos:AplicarVentaResponse"/>
15
16
17  <xsd:complexType name="ConsultarProductosRequest">
18    <xsd:sequence>
19      <xsd:element minOccurs="0" maxOccurs="1" name="codigoProducto" type="xsd:string"/>
20      <xsd:element minOccurs="0" maxOccurs="1" name="nombreProducto" type="xsd:string"/>
21      <xsd:element minOccurs="0" maxOccurs="1" name="categoria" type="xsd:string"/>
22      <xsd:element minOccurs="0" maxOccurs="1" name="almacen" type="xsd:string"/>
23    </xsd:sequence>
24  </xsd:complexType>
25
```

# gRPC

Este formato binario permite a las aplicaciones almacenar e intercambiar datos estructurados fácilmente, incluso si los programas están compilados en diferentes lenguajes.

```
1 syntax = "proto3";
2
3 package tutorial;
4
5 option java_package = "com.example.tutorial";
6 option java_outer_classname = "AddressBookProtos";
7
8 message Person {
9   required string name = 1;
10  required int32 id = 2;
11  optional string email = 3;
12
13  enum PhoneType {
14    MOBILE = 0;
15    HOME = 1;
16    WORK = 2;
17  }
18
19  message PhoneNumber {
20    required string number = 1;
21    optional PhoneType type = 2 [default = HOME];
22  }
23
24  repeated PhoneNumber phones = 4;
25 }
26
27 message AddressBook {
28   repeated Person people = 1;
29 }
```

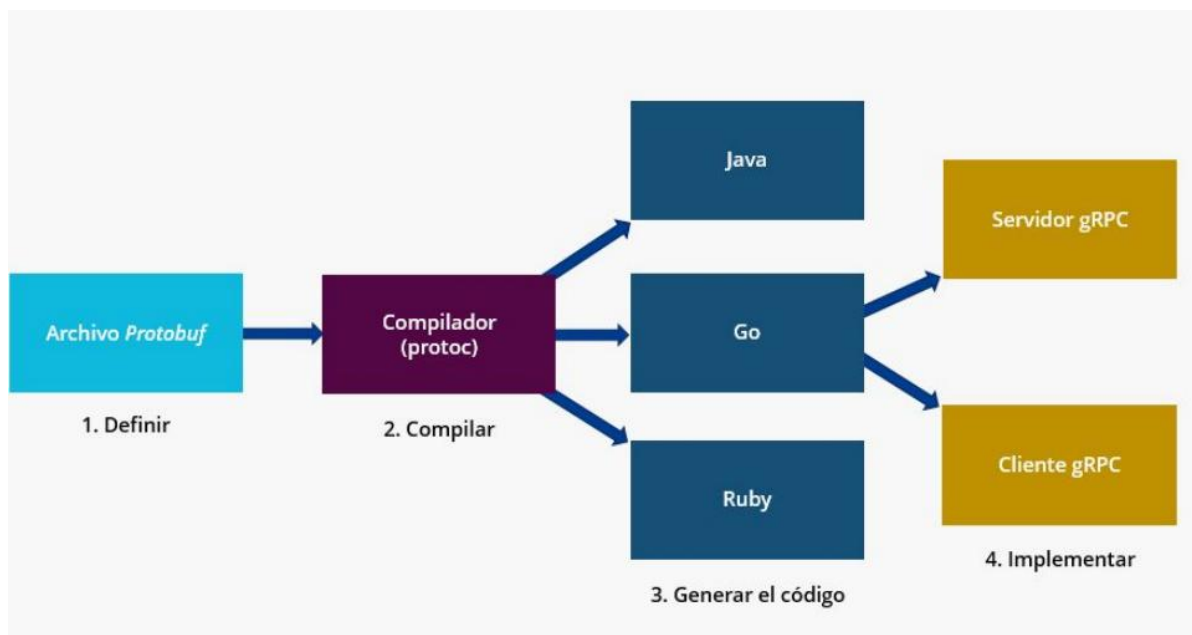
*required*: indica que es obligatorio asignar un valor al campo.

*optional*: en los campos opcionales, es posible indicar un valor, aunque no es necesario.

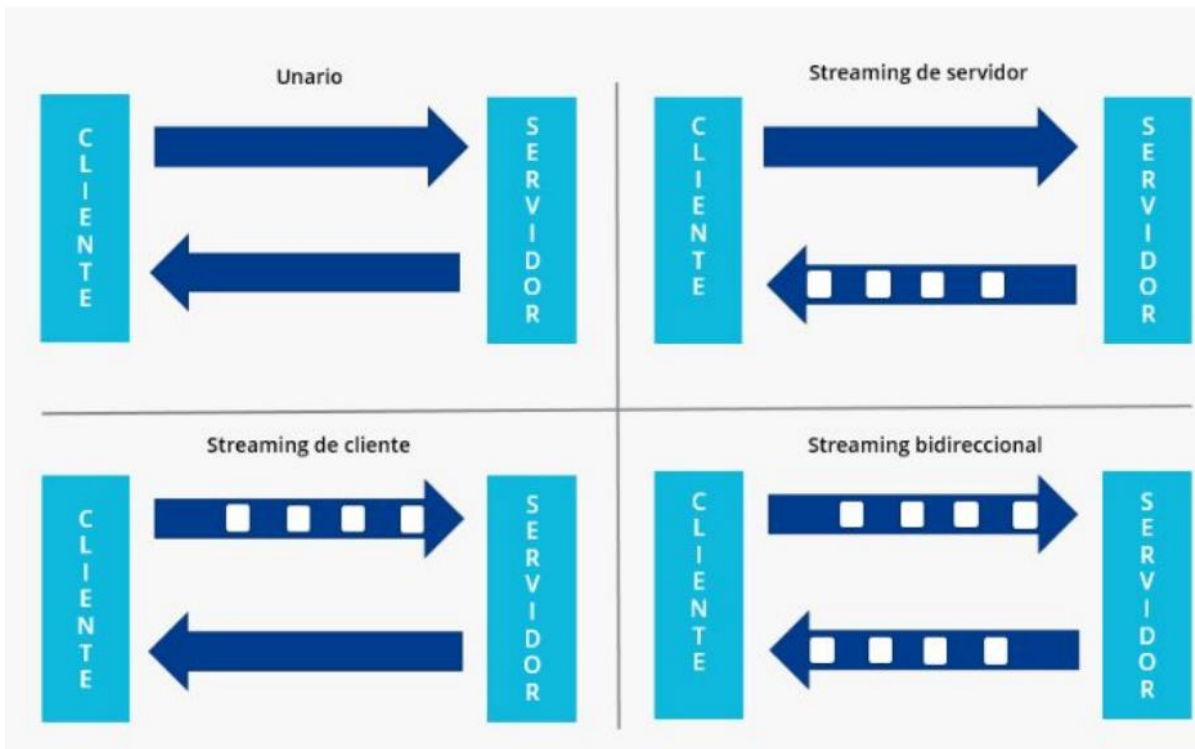
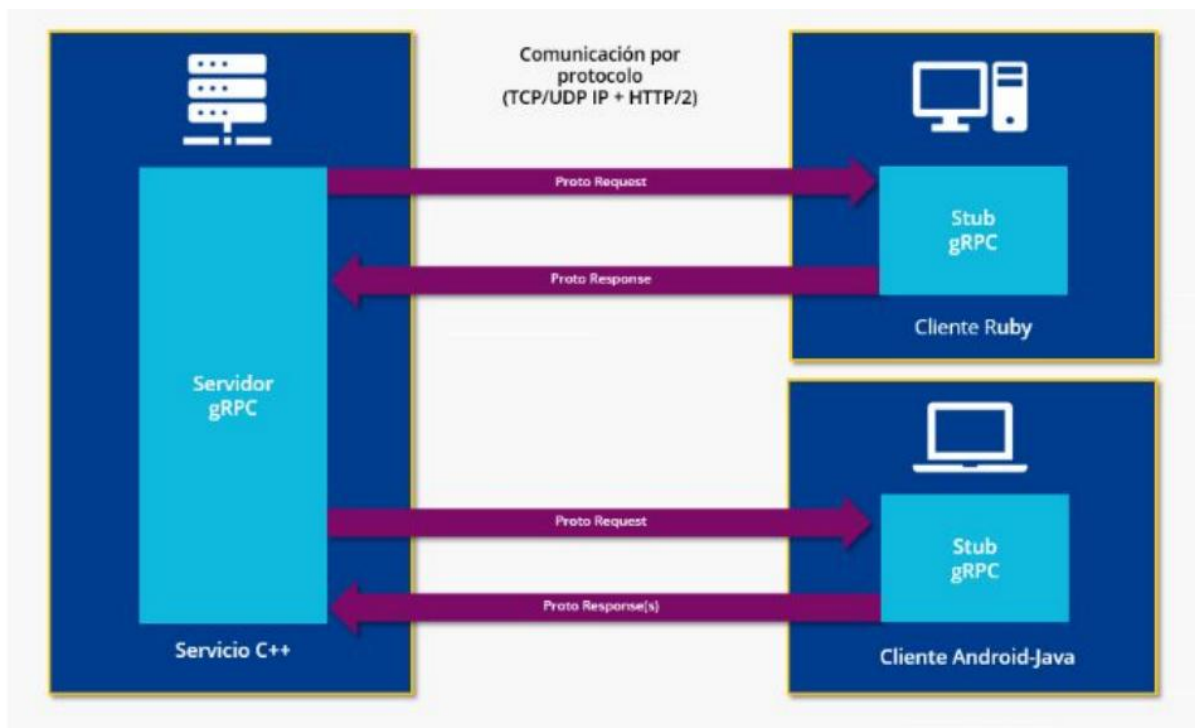
*repeated*: los campos con este modificador pueden repetirse un número indefinido de veces (incluso ninguna).

## Pasos de implementación

1. Definición del contrato de servicio
2. Generación del código gRPC del archivo. proto (en el lenguaje deseado).
3. Implementación del servidor en el lenguaje deseado.
4. Creación del stub del cliente que llama al servicio.



## Arquitectura gRPC



## Conclusiones

- Centrado en la Comunicación entre Servicios/Microservicios
- Soporte Multilenguaje
- Flujo de Intercambio más pequeño
- Streaming Bidireccional



## GraphQL

- Simplifica la consulta
- Se apoya sobre HTTP
- Esquema Tipado

https://swapi.dev/api/people/1

GET

https://swapi.dev/api/people/1

Params

Authorization

Headers (7)

Body

Pre-request Script

Tests

Settings

Query Params

| KEY | VALUE |
|-----|-------|
| Key | Value |

Body

Cookies

Headers (10)

Test Results

Pretty

Raw

Preview

Visualize

JSON

```
1 {
2   "name": "Luke Skywalker",
3   "height": "172",
4   "mass": "77",
5   "hair_color": "blond",
6   "skin_color": "fair",
7   "eye_color": "blue",
8   "birth_year": "1988Y",
9   "gender": "male",
10  "homeworld": "https://swapi.dev/api/planets/1/",
11  "films": [
12    "https://swapi.dev/api/films/1/",
13    "https://swapi.dev/api/films/2/",
14    "https://swapi.dev/api/films/3/",
15    "https://swapi.dev/api/films/6/"
16  ],
17  "species": [],
18  "vehicles": [
19    "https://swapi.dev/api/vehicles/14/",
20    "https://swapi.dev/api/vehicles/30/"
21  ],
```

## Ejemplo Query

https://swapi.apis.guru

GET https://swapi.apis.guru

Params Authorization Headers (9) **Body** Pre-request Script Tests Set

none form-data x-www-form-urlencoded raw binary GraphQL

QUERY

```
1 {
2   person(personID: 1) {
3     name
4     birthYear
5     filmConnection {
6       films {
7         title
8       }
9     }
10  }
11 }
```

Body Cookies Headers (11) Test Results

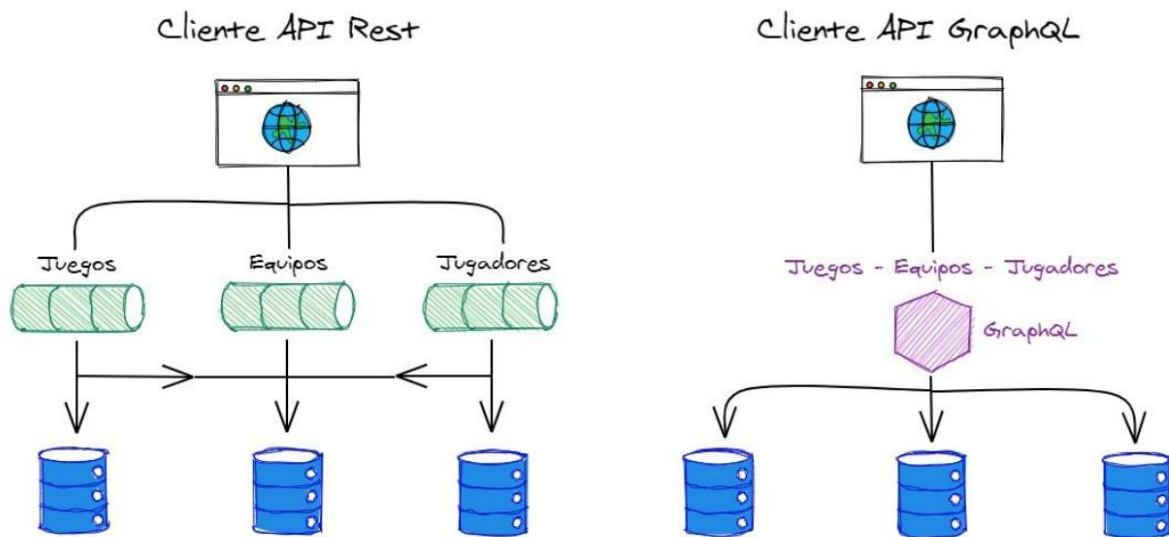
Pretty Raw Preview Visualize JSON

```
1 {
2   "data": {
3     "person": {
4       "name": "Luke Skywalker",
5       "birthYear": "1988Y",
6       "filmConnection": {
7         "films": [
8           {
9             "title": "A New Hope"
10          },
11          {
12            "title": "The Empire Strikes Back"
13          },
14          {
15            "title": "Return of the Jedi"
16          },
17          {
18            "title": "Revenge of the Sith"
19          }
20        ]
21      }
22    }
23  }
24 }
```

## GraphQL vs API REST







## API – Comparativa

| REST                                   | SOAP                                               | GraphQL                                                | gRPC                                          |
|----------------------------------------|----------------------------------------------------|--------------------------------------------------------|-----------------------------------------------|
| Sincrónico HTTP                        | Sincrónico/Asincrónico HTTP, FTP, SMTP             | Sincrónico/Asincrónico HTTP, AMQP, MQTT                | Sincrónico/Asincrónico HTTP/2                 |
| Basado en HTTP (Estados, Métodos, URI) | Basado en intercambio de Mensajes                  | Basado en intercambio de Mensajes                      | Basado en intercambio de Mensajes             |
| Múltiples Estándares (Open API, RAML)  | Estándar de Especificación de Esquema mediante XML | Estándar de Especificación de Esquema mediante IDL/SDL | Estándar de Especificación de Protobuf        |
| Formato de intercambio JSON            | Formato de intercambio XML                         | Formato de intercambio JSON                            | Formato de intercambio Serializado (protobuf) |

# Colas de Menajes

Una cola de mensajes es una forma de comunicación asíncrona de servicio a servicio que se usa generalmente en arquitecturas de microservicios y sin servidor.

Los mensajes se almacenan en la cola hasta que se procesan y eliminan. Cada mensaje se procesa una sola vez, por un solo consumidor.

Las colas de mensajes se pueden usar para desacoplar procesos pesados, para acumular trabajo y para clasificar cargas de trabajo.

Proporcionan la comunicación y la coordinación para aplicaciones distribuidas.

Pueden simplificar de forma significativa la escritura de código para aplicaciones desacopladas.

## Desafíos

- Disponibilidad
- Velocidad de Procesamiento
- Confirmación de Procesamiento
- Desacoplar mediante Mensajería
- Modo de Transmisión: Cola o Topic



### **Mejoran el rendimiento, la fiabilidad y la escalabilidad.**

Permiten a diferentes partes de un sistema comunicarse y procesar las operaciones de forma asíncrona.

Ofrece un búfer ligero que almacena temporalmente los mensajes.

Ofrece puntos de enlace que permiten a los componentes de software conectarse a la cola para enviar y recibir mensajes

## Implementaciones

- Amazon Simple Queue Service
- Azure Service Bus
- RabbitMQ
- ActiveMQ
- ¿Apache Kafka?

# Bases de datos Compartida

## Beneficios:

Base de Datos central para integrar los componentes. Centrado en la integración, y no de uso de funcionalidad.

Permite integración **sincrónica** y **asincrónica**. Siendo simple de integrar, gracias a la flexibilidad de los stacks.

## Problemas:

Puede generar problemas de performance.

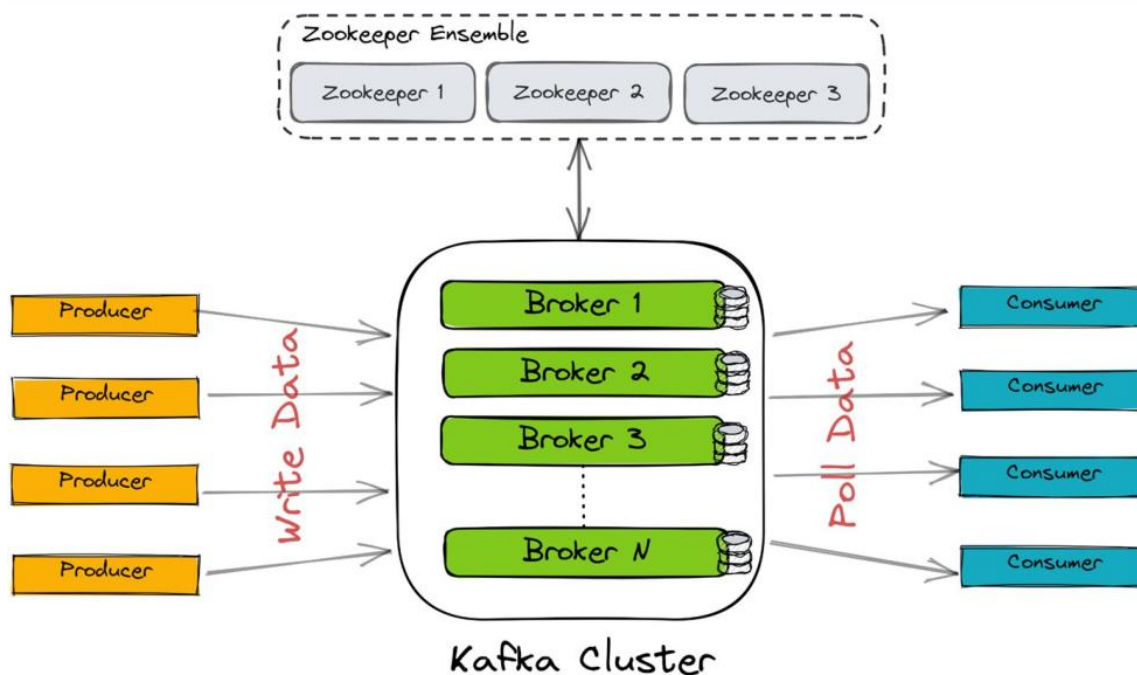
Alto acoplamiento.

Poca flexibilidad al cambio.

Se debe considerar la seguridad (CIA) de los datos.

## Broker: Apache Kafka

- Sistema de mensajería usando patrón Publicación/Subscripción
- Seguimiento de actividades
- Recopilación de métricas y telemetría de diferentes ubicaciones
- Desacoplamiento de dependencias del sistema e intercambio de datos asíncrono entre Microservicios
- Procesamiento de transmisiones con Kafka Stream API o Spark
- Integración con Spark, Hadoop y muchas otras tecnologías de Big data.



### Productores:

- Cualquier programa, sensor, dato o flujo de información que escribe en el clúster de Kafka.

### Consumidores:

- Servicios/Procesos que leen datos almacenados en el clúster de Kafka para producir informes, paneles, conocimientos y procesos comerciales.
- La relación entre productores y consumidores es de muchos a muchos y están totalmente desacopladas.

### Brokers Kafka:

- Un clúster de Kafka consta de uno o más servidores (Brokers de Kafka) que ejecutan procesos de Kafka en ellos.
- Datos centralizados y almacenados en los brokers, cada broker tiene su propio almacenamiento local.
- Los brokers están conectados en red y actúan juntos como un único clúster Kafka.
- El tiempo de retención es el tiempo que se puede establecer a nivel de tema en el que los datos caducarán y saldrán del sistema para dejar espacio para los nuevos datos que lleguen.

Kafka usa Zookeeper para gestionar el consenso sobre algunas partes del estado distribuido del clúster. Por ejemplo, todos los brokers deben ponerse de acuerdo sobre cuál es el broker líder.

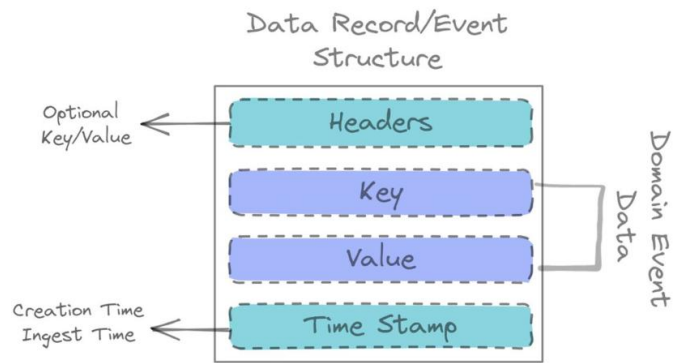
### Principales usos:

- Almacenamiento y gestión de metadatos del clúster
- Elección del broker líder y coordinación distribuida altamente confiable
- Detección y recuperación de fallos
- Almacena y Gestiona Accesos
- Los productores y los consumidores están totalmente desacoplados.
- Los consumidores lentos no impactan a los productores.
- Permite sumar más Consumidores sin impactar a Productores.
- Las fallas de los consumidores no impactan el sistema.

### Topic:

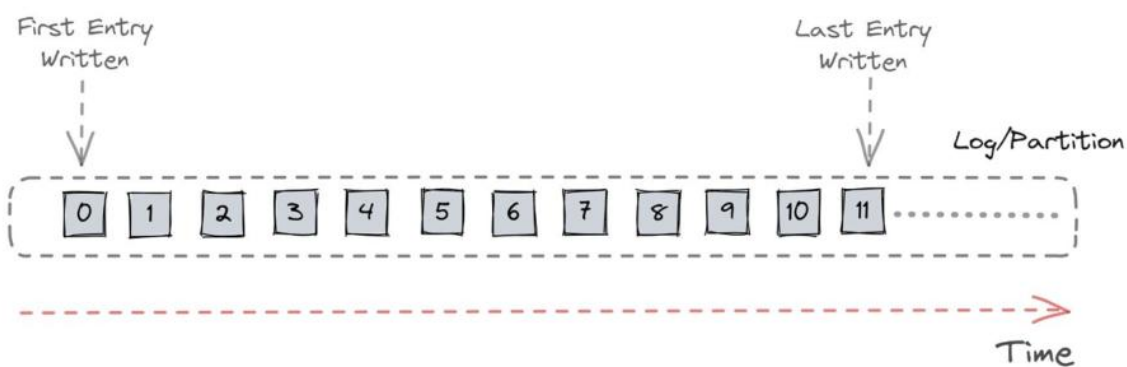
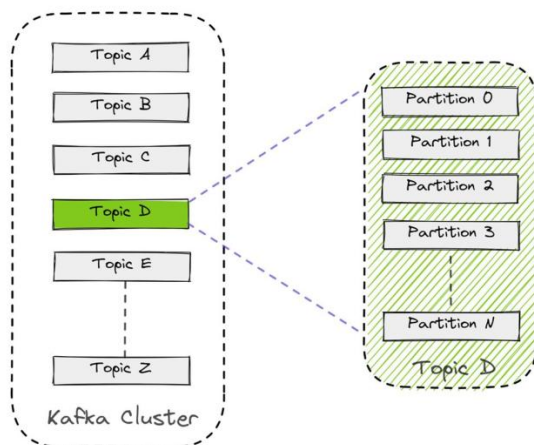
- Son una colección de mensajes y eventos relacionados, puede pensar en un tema como un registro (Secuenciade eventos)
- Es una representación lógica y sus categorías de mensajes en grupos.
- Se podría definir un número ilimitado de topics.
- Productor <---> Topic: Relación N a N
- Los desarrolladores definen topics basados en decisiones de arquitectura.
- Un solo productor puede escribir sobre muchos temas y muchos consumidores pueden leer sobre el mismo tema.

- Encabezados (clave/valor)
- Clave (utilizada para enrutamiento)
- Valor (carga útil serializada)
- Marca de tiempo



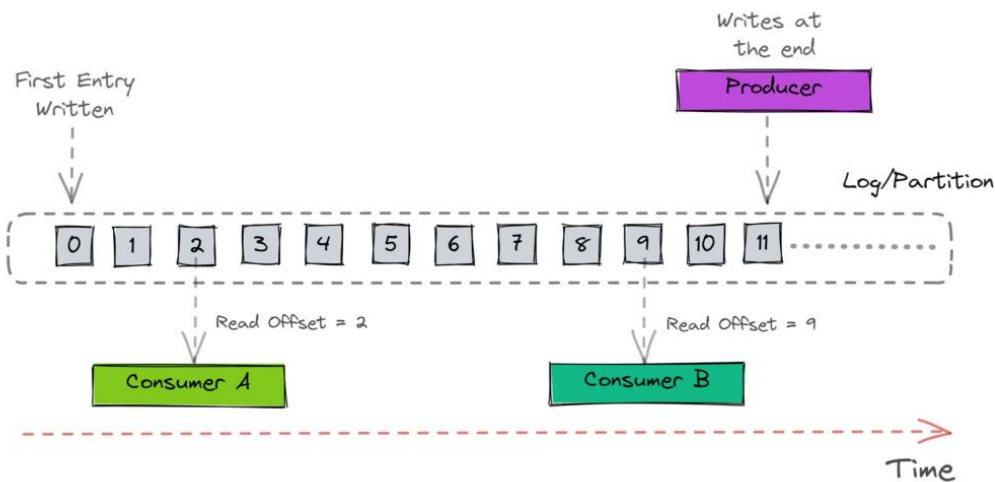
## Partición:

- Es un log persistente para los registros de datos.
- El topic reside en un broker de almacenamiento (servidores).
- Para lograr escalabilidad, es necesario:
  - Dividir topics en partes más pequeñas (particiones)
  - Las particiones deben asignarse a diferentes brokers en el clúster y la misma partición replicarse en diferentes brokers.
  - Cada partición tiene un orden estricto.
  - Los nuevos registros de datos solo se agregan al final de la partición, no se realizan cambios en los registros de datos antiguos (inmutables)



Los mensajes son escritos en los topics de forma inmutable primero/último

El registro es inmutable una vez que se escribe el registro, no hay forma de actualizar este mensaje.



### Partición Escritura/Lectura:

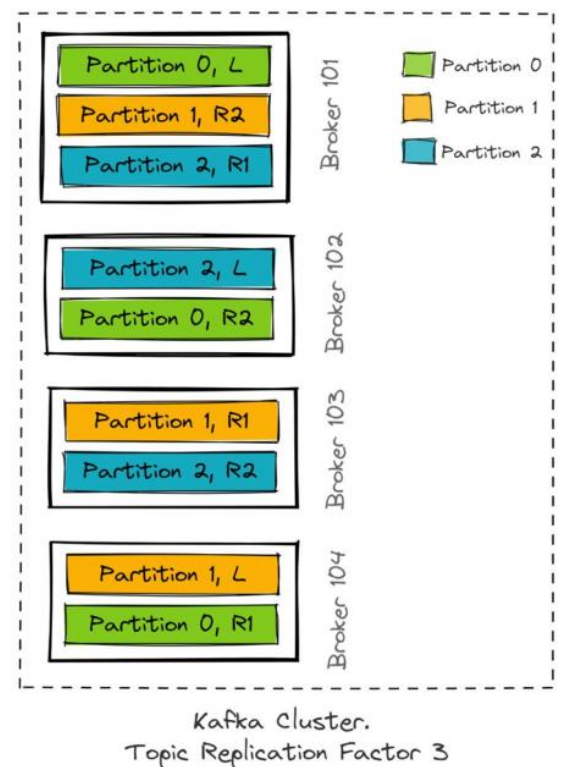
- Mensajes de Topics repartidos/replicados entre particiones
- Particiones repartidas entre Brokers
- Cada Broker maneja muchas Particiones
- Cada Partición es almacenada en el “disco” del Broker
- Cada mensaje en Registro/Partición se identifica mediante un desplazamiento
- El topic tiene una política de retención configurable

### Replicación:

- Cada topic tiene un número configurable de réplicas (factor de replicación). El valor predeterminado es 3.
- Una de esas réplicas se llama Partición Líder.
- Producir datos para un topic son “recibidos” en la partición Líder.
- Los Brokers con particiones de seguidoras contactan al Broker con partición líder para mantenerse actualizados.

### Balanceo:

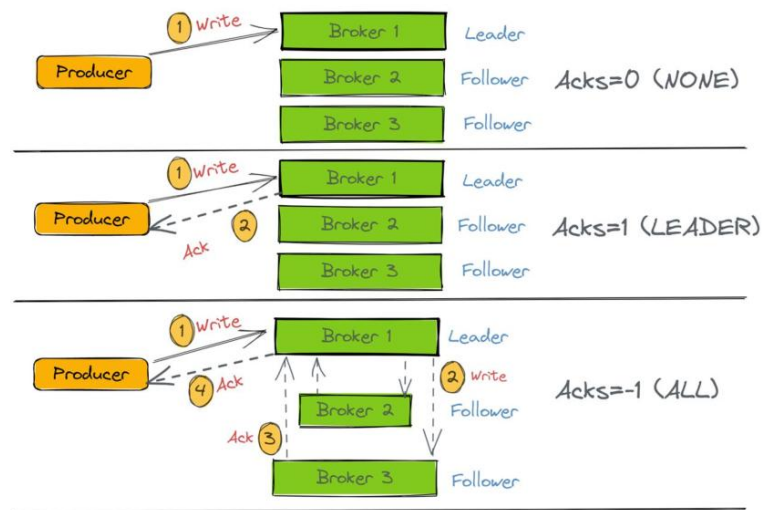
- Los productores utilizan una estrategia para asignar cada mensaje a una partición.
- Es Importante balancear la carga de mensajes en los Brokers.
- Las estrategias de partición disponibles las especifica el productor y son:
  - Sin clave de mensaje Modo Round Robin
  - Clave de mensaje utilizada  $\text{hash}(\text{clave}) \% \text{Números de partición en el tema}$
- Al utilizar una clave, Kafka garantiza un orden estricto de los mensajes en la partición.



- La Clave Puede ser: Identificación del dispositivo, la identificación del camión, la identificación del sensor, etc. Todos los mensajes que vienen con la misma clave terminarán en la misma partición estrictamente ordenada.

### Política de Retención:

- La política de retención decide cuánto tiempo quiero almacenar mis datos/eventos.
- El valor predeterminado es 1 semana.
- Se puede configurar globalmente o por tema.
- Decisión empresarial.
- Analizar Factor de Costo si necesitamos retener por largos periodos.
- Factores de cumplimiento de Regulaciones PII/IIP o GDPR.



“More Safety, more latency”

## Peer to Peer – P2P

Cualquier par (peer) es libre de comunicarse directamente con cualquier otro peer, sin el uso de un servidor central. Para localizarse los pares realizan un intercambio de listas de forma automática entre compañeros conocidos. Son claves los mecanismos de índices para navegarse eficientemente.

### + Ventajas:

- Un par puede realizar consultas o responder. (Cliente – Servidor)
- Descentralización, autonomía, escalabilidad.
- Fiabilidad: los networks P2P son resilientes a fallos en algunos nodos al poder enviar información, aunque haya algunos caídos.

### Tipos:

- Networks P2P puros: networks realmente descentralizados sin una autoridad central o infraestructura dedicada para la comunicación. Todos tienen los mismos privilegios y responsabilidades. BitTorrent.
- Networks P2P híbridos: tienen algunos servidores centrales además de los nodos para tener un balance entre la descentralización y la eficiencia. Skype y eDonkey.



# ¿Qué es la observabilidad?

Es una medición que determina cómo los estados internos pueden ser inferidos a través de las salidas externas.

La observabilidad es la capacidad de comprender el estado o la condición interna de un sistema complejo basándose en su telemetría.

Proporciona:

- Visibilidad profunda para la identificación y resolución de problemas.
- Rapidez y mayor precisión a la hora de identificar problemas sin necesidad de pruebas ni codificación adicional.

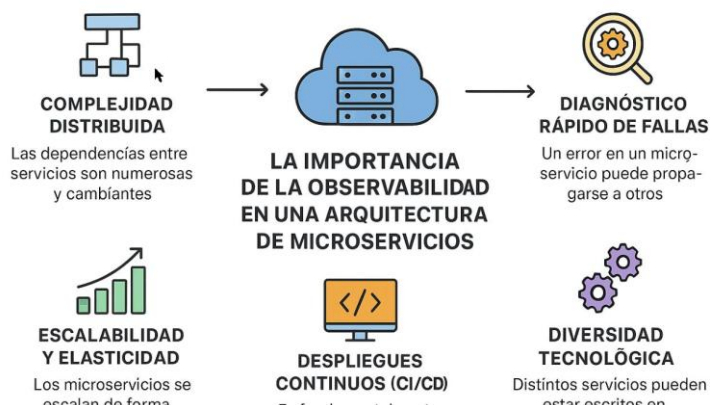
| Observabilidad                                                                                                                                                                                                                                                                     | Monitoreo                                                                                                                                                                                                     |
|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <ul style="list-style-type: none"><li>• Determinar e inferir el estado interno del sistema con las salidas.</li><li>• Usa logs, métricas y trazas para comprender la causa raíz de un problema.</li><li>• Esencial para diagnosticar problemas en sistemas distribuidos.</li></ul> | <ul style="list-style-type: none"><li>• Seguimiento de métricas conocidas para identificar problemas conocidos.</li><li>• Útil para alertar sobre desviaciones.</li><li>• Proporciona los síntomas.</li></ul> |

Para lograr un sistema que sea Observable hay que implementar:

- Métricas: indican lo **QUE** está pasando.
- Logs: indican **POR QUE** está sucediendo.
- Trazas: indican está aconteciendo.
- Perfiles: indica POR QUE y en que partes específicas del código.

## Relevancia en Microservicios

### LA IMPORTANCIA DE LA OBSERVABILIDAD EN UNA ARQUITECTURA DE MICROSERVICIOS



Una vez que nuestro sistema está en producción, ¿cómo sabemos que vamos bien?

- ¿Y si nadie se queja?
- Si nos llega una falla, ¿eso qué implica?
- ¿Qué es lo que se tiene que arreglar y cómo?
- ¿Cuánto tiempo estuvo afectado el servicio?
- ¿Qué impacto tuvo para el negocio?
- ¿Se perdieron usuarios?
- ¿Cuántas operaciones se interrumpieron?

## Logs

Son registros cronológicos de eventos generados por aplicaciones o componentes del sistema que permite reconstruir lo que ocurrió y entender el contexto de cada evento.

- Capturan **QUE**, **CUANDO** y **DONDE** pasó.
- Pueden incluir mensajes de error, advertencias, tiempos de ejecución, entradas de usuario, etc.
- Son detallados, pero verbosos.

Siempre útiles para un diagnóstico profundo.

### Formato ELF (web server)

date, time, c-ip, cs-username(-), s-ip, sport, method, cs-uri-stem, status, csUserAgent

2010-05-02 15:42:15 - 40.89.255.10 34.14.255.10 80 GET /default.htm 200 - HTTP/1.0 Mozilla/4.0  
(compatible: MSIE+5.5+Windows+2000+Server)

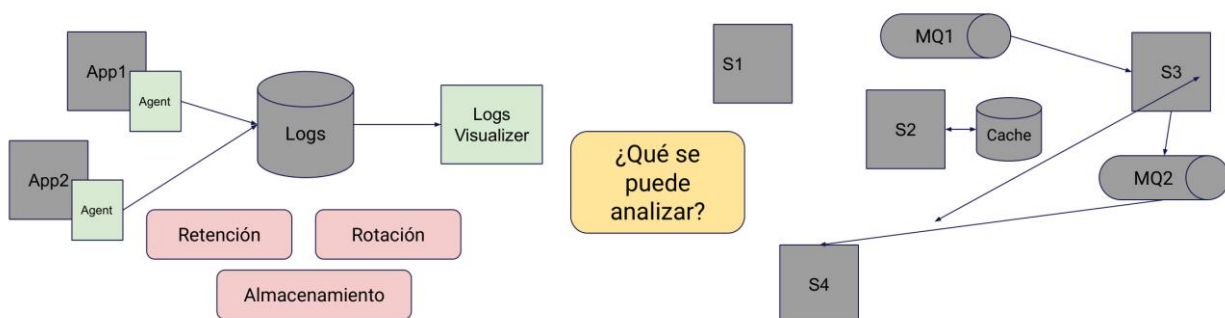
## Características importantes

Mensaje que describe un proceso en ejecución, siempre al menos posee una marca de tiempo

| Timestamp           | Nivel | Mensaje                         | Hilo?      |
|---------------------|-------|---------------------------------|------------|
| 2024-05-18 17:30.66 | DEBUG | Guarda en DB la compra 333      | Sesión?    |
| 2024-05-18 17:31.67 | DEBUG | Inicia compra 347               | Instancia? |
| 2024-05-18 17:31.99 | WARN  | Tarjeta inválida: de compra 282 | Versión?   |
| 2024-05-18 17:32.32 | DEBUG | Inicia compra 346               | ...        |
| 2024-05-18 17:32.14 | DEBUG | Inicia compra 348               |            |
| 2024-05-18 17:35.87 | DEBUG | Guarda en DB compra 347         |            |
| 2024-05-18 17:35.00 | INFO  | Finaliza compra 333             |            |
| 2024-05-18 17:36.07 | ERROR | Falla Compra                    |            |

## Infraestructura

Mensaje que describe un proceso en ejecución, siempre al menos posee una marca de tiempo



# Métricas

Es una medición de un “servicio” capturado en tiempo de ejecución o es una medida numérica asociada con su información de contexto:

- Marca de tiempo / Timestamp \*
- Nombre \*
- Otros: Etiquetas o tags

Útiles para ver que algo falla frecuentemente pero no es tan directo ver por qué. Se almacenan en una base de datos de series de tiempo

## Tipos:

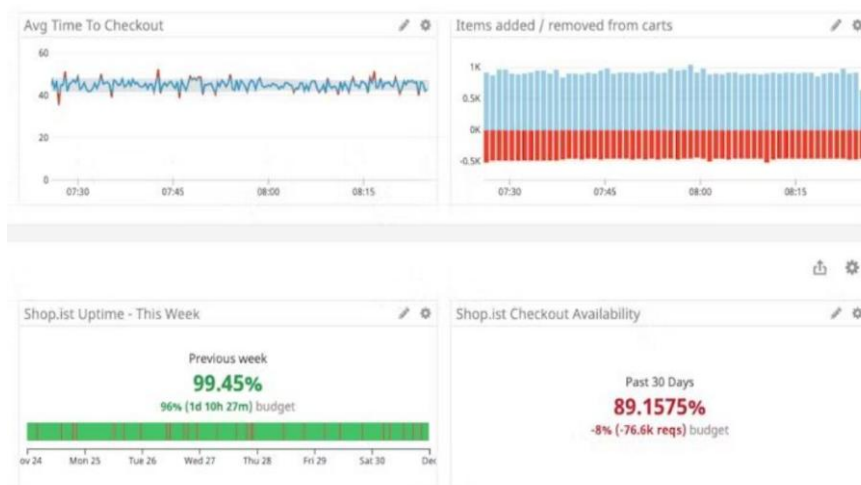
- Counter
- UpDownCounter
- Gauge
- Histogram

Se visualizan en diferentes gráficos, que suelen agruparse en tableros o dashboards, para que la información sea más fácil de interpretar.



## 4 señales de oro de los SRE

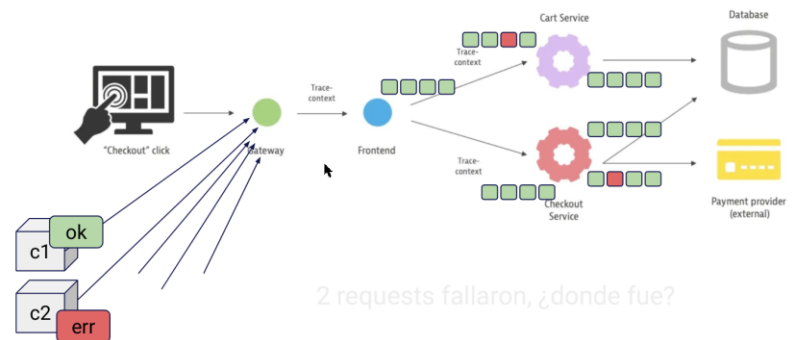
- Tráfico: Carga o demanda que recibe el sistema. Ej
  - Peticiones por segundo (RPS)
  - Mensajes procesados/minuto
- Errores: Tasa de fallos o respuestas incorrectas
  - % de respuestas HTTP 5xx
  - Cantidad de excepciones por unidad de tiempo
- Latencia: Tiempo que tarda el sistema en responder
  - Tiempo promedio de respuesta (p95, p99)



# Trazas

Nos da el recorrido de una operación a lo largo de nuestro sistema.

En la práctica, se relacionan los requests a las distintas apps a través de un identificador.



Home > | Availability >

## End-to-end transaction details

Directory: Unknown directory

Search results

Filtered on: timestamp > timestamp < customDimensions.FullTestResultAvailable == true name == Contoso Retail ...

Suggested

Contoso Retail VM Employee Create  
Availability result: Failed  
Availability location: South Central US  
Duration: 102 ms

All

Sort by @ Relevance

Contoso Retail VM Employee Create  
Availability result: Failed  
Availability location: South Central US  
Duration: 102 ms

### End-to-end transaction

Operation ID: ...

| EVENT                                              | RES. | DURATION |
|----------------------------------------------------|------|----------|
| Contoso Retail VM Employee Create South Central US | ...  | 102 ms   |
| GET Employees/Create                               | 404  | 102 ms   |
| GET Employees/Create                               | 404  | 2.6 ms   |
| EXCEPTION System.Web.HttpException                 | ...  | ...      |
| EXCEPTION System.Web.HttpException                 | ...  | ...      |
| Contoso Retail VM Employee Create South Central US | ...  | 102 ms   |

Traces & events 2 Traces 0 Events

View all

Download web test result Create work item

South Central US

### Troubleshooting Report

- ✓ DNS lookup succeeded. Resolved IP address(es):
- ✓ Connection succeeded with IP address: [Go to Azure Resource]
- ✓ Successfully sent request
- ✓ Received HTTP response (headers)
- ✓ Received HTTP response (body): 3754 bytes
- ✗ Status code validation failed
  - ✗ '404 - Not Found' does not match the expected status '200 - OK'.

### Availability Test Step Properties

Show all

| Event time | Test name                         | Test location    |
|------------|-----------------------------------|------------------|
| ...        | contoso retail vm employee create | South Central US |

# Alarmas

| Tipo de Alarma                  | ¿Qué detecta?                                                            | Ejemplo típico                                                    |
|---------------------------------|--------------------------------------------------------------------------|-------------------------------------------------------------------|
| Por umbral estático             | Se dispara cuando una métrica supera o baja de un valor fijo.            | “CPU > 90% durante 5 min” o “Error rate > 2%”                     |
| Por tendencia / anomalía        | Detecta cambios inusuales respecto al comportamiento histórico.          | “Latencia aumentó un 50% respecto a la media de la última hora”   |
| Basada en SLO/SLA               | Se activa cuando el servicio no cumple su objetivo de nivel de servicio. | “Error budget consumido > 80% este mes”                           |
| De ausencia de datos (silencio) | Detecta la falta de métricas o logs esperados.                           | “No se reciben métricas del servicio X en los últimos 10 minutos” |

**1 Choose the detection method**  
 Threshold Alert | Change Alert | Anomaly Detection  
 An alert is triggered whenever a metric crosses a thresh

**2 Define the metric**  
 Metric: `aws.ecs.service.cpuutilization` from `servicename:demosites-httpbin` excluding (none)

**3 Set alert conditions**  
 Trigger when the metric is `above` the threshold `on average` during the last `5 minutes`  
 Alert threshold: `100` (100 %)  
 Warning threshold: `70` (70 %)  
 Alert recovery threshold: `Alert recovery threshold (opt)`  
 Warning recovery threshold: `Warning recovery threshold`

**Monitors**  
 Latency is high on checkout  
 Metric alert  
 13h ago  
 ALERT

Carlo Francisco testing 7/15  
 Trace-analytics alert  
 13h ago  
 OK

MBS - Error rate on low frequency resource - No rollout or fill  
 Query alert  
 13h ago  
 OK

Service aws.lambda.post-coupon-prod-eu has an abnormal change in throughput  
 Query alert  
 13h ago  
 OK

We have both anomalous latency and high cart abandonment. (Gartner)  
 Composite  
 13h ago  
 /

Service game-night has a high p90  
 13h ago  
 OK

## Base de series temporales (TSDB)

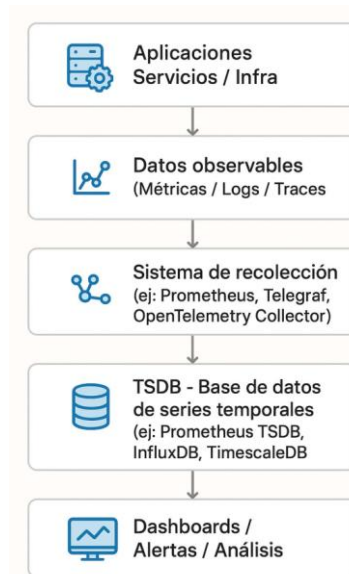
Se especializan en guardar datos temporales:

- Optimización para escrituras frecuentes
- Consultas por rango temporal
- Comparar “ahora” vs. “ayer” o “la semana pasada”.

Las mediciones de las métricas son el caso de uso ideal para estas bases.

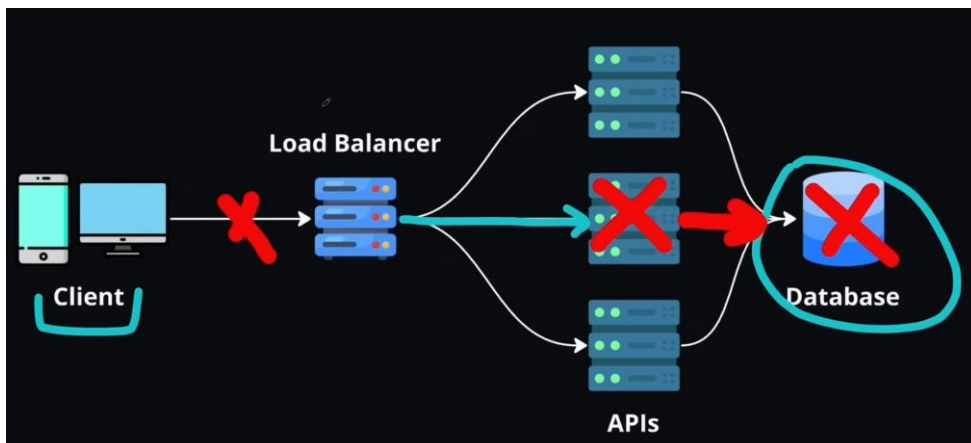
Ejemplos: Prometheus, InfluxDB, TimescaleDB, VictoriaMetrics.

- Suelen ser bases costosas, dependiendo de cuanta información de contexto / tags se almacenen por medición / métrica



# Single Point of Failure (SPOF)

Cualquier componente que podría causar que todo el sistema colapse si deja de funcionar.



La falla de la base de datos hace que colapse todo, lo que puede traer problemas para el negocio al no poder concretar operaciones cruciales.

Es un problema para escalabilidad, donde se incrementa las chances de fallar en el mismo punto si seguimos escalando.

Pueden aparecer problemas de seguridad: si el SPOF es el Load Balancer es posible que ataquen y tiren abajo a todo nuestro sistema.

Evitando SPOF

Load Balancer – Evitando SPOF

- 1) Redundancy: usar múltiples Load Balancers para que la caída de N-1 de N LB no destruya por completo a nuestro sistema. Esto se le suma a que es posible distribuir las cargas.
- 2) Health Checks y Monitoreo: los load balancers ya hacen esta estrategia para nuestras bases de datos. Lo mismo se puede aplicar a nuestros LB.

## API Design

¿Qué son APIs y cuál es su rol en la arquitectura del sistema?

API: contrato que define una interfaz que indica los tipos de pedidos que se harán, como hacerlos y que respuestas esperar.

*Características Claves*

- Mecanismo de Abstracción: esconde la implementación, muestra la funcionalidad.
- Límites del servicio: define interfaces claras entre componentes de sistemas.



## Los 3 estilos de API más comunes

| REST                                                                             | GraphQL                                                                      | gRPC                                                                                              |
|----------------------------------------------------------------------------------|------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------|
| <b>Resource-based:</b> se organiza mediante recursos que llegan de métodos HTTP. | <b>Query Language:</b> clients piden exactamente lo que quieren              | <b>Buffers de protocolo:</b> serialización binaria con un schema definido.                        |
| <b>Stateless:</b> cada request tiene toda la información detallada.              | <b>Single Endpoint:</b> un endpoint para todas las operaciones               | <b>Definición de servicios:</b> métodos definidos como RPCs en archivos .proto                    |
| <b>Métodos estándar:</b> GET, POST, PUT, DELETE, PATCH, etc.                     | <b>Operaciones:</b> Query (read), Mutation (write), Subscription (real-time) | <b>Tipos de comunicación:</b> Unary, Server Streaming, Client Streaming, Bidirectional Streaming. |
| Estilo más común                                                                 | Pocas requests                                                               | Alto performance                                                                                  |
| Web y Apps Móviles                                                               | IU Complejo                                                                  | Microservicios                                                                                    |

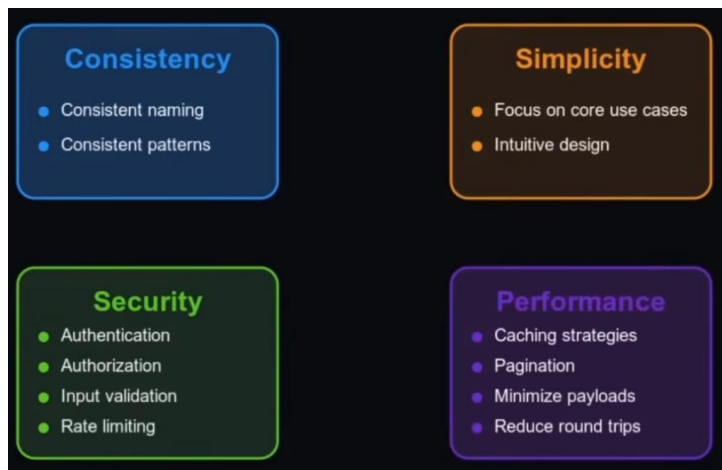
## REST vs GraphQL

| Rest                                                                                    | GraphQL                                                                                          |
|-----------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------|
| Endpoints con recursos específicos                                                      | Endpoint único                                                                                   |
| Múltiples requests para obtener data                                                    | Una request para datos precisos                                                                  |
| Métodos HTTP definen operaciones                                                        | Lenguaje Query define operaciones                                                                |
| Estructuras de respuesta fija                                                           | Cliente específica la estructura de la respuesta dada.                                           |
| Versionado explícito (/v1/, /v2/)                                                       | Evolución de schema sin versionado                                                               |
| Built-in HTTP caching                                                                   | Caching a nivel de la aplicación                                                                 |
| GET /api/v1/users/123<br>GET /api/v1/users/123/posts<br>GET /api/v1/users/123/followers | Query {<br>User(id: "123") {<br>name<br>posts { title, content }<br>followers { name }<br>}<br>} |



## 4 principios claves de diseño con APIs

“La mejor API es la que los desarrolladores pueden usar sin leer la documentación.”



## Como los protocolos de aplicación influyen las decisiones de diseño de APIs

| Protocolos                                                                                                              | Diseño de APIs                                                                                                                            |
|-------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------|
| <ul style="list-style-type: none"><li>• HTTP</li><li>• Códigos de estatus</li><li>• WebSockets</li><li>• gRPC</li></ul> | <ul style="list-style-type: none"><li>• RESTful APIs</li><li>• Operaciones CRUD</li><li>• GraphQL</li><li>• APIs en tiempo real</li></ul> |

### Consideraciones:

- La elección de protocolo afecta la estructurad de la APIs, performance y capacidades del sistema.
- El diseño de la API debe acomodarse a las limitaciones del protocolo y tomar ventaja de las fortalezas del mismo.

## El proceso del diseño de una API

- Identificar casos de uso core e historias de usuario.
- Definir scope y límites.
- Determinar requerimientos de performance
- Considerar restricciones de seguridad.

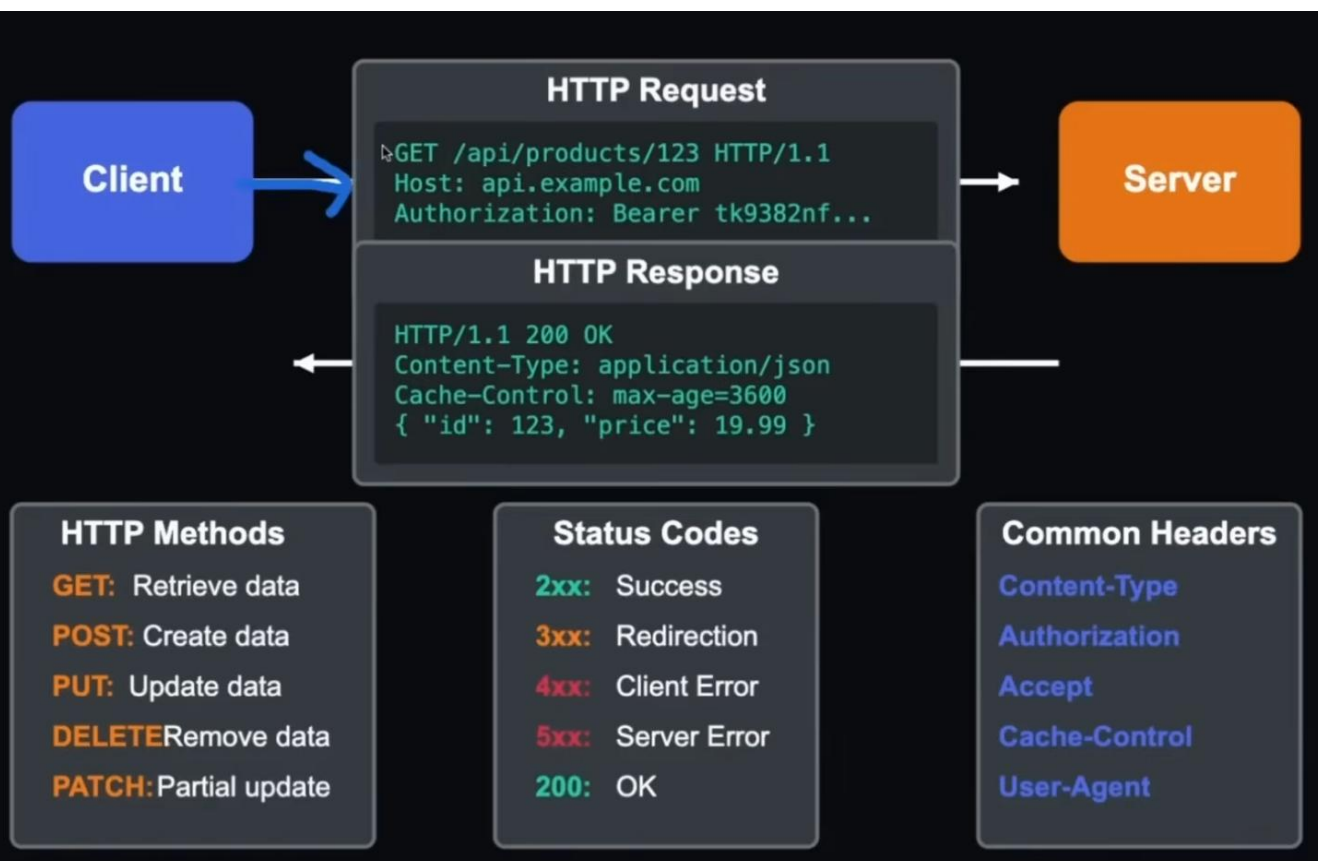
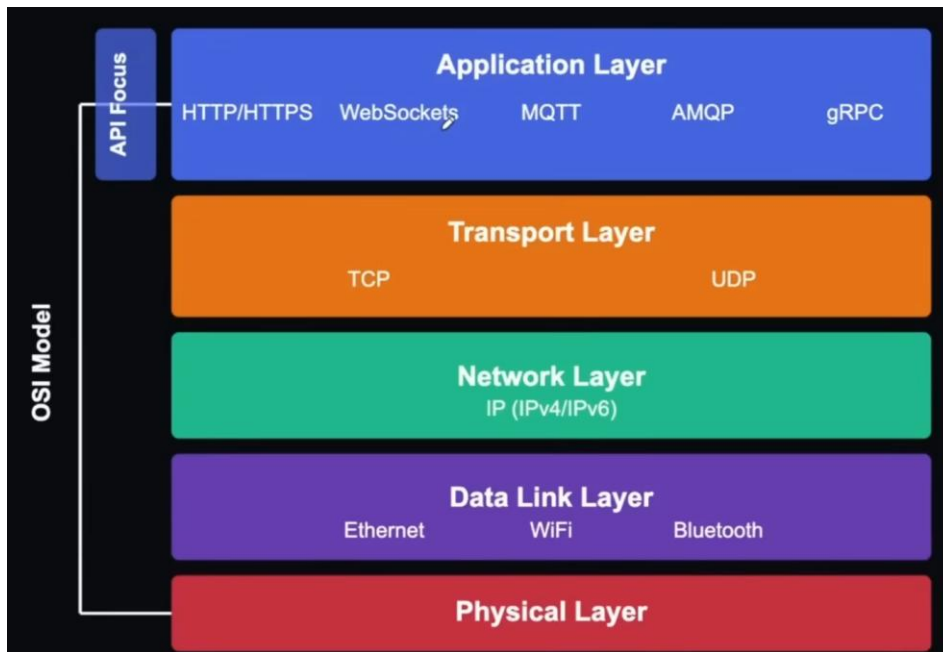
## ¿Cómo encarar el diseño?

- Top-down: empezar con los requerimientos de alto nivel y workflows (más usado en entrevistas y parciales)
- Bottom-up: usar con modelos de datos existentes y las capacidades requeridas (más común en el mercado real)
- Contract-first: definir el contrato de la API antes de la implementación (más usado en entrevistas)

## Manejo del ciclo de vida de una API

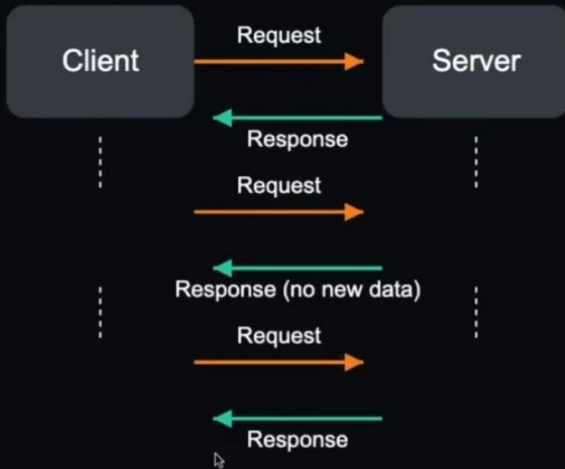
Diseño => Desarrollo => Deployment y monitoreo => Mantenimiento => Retiro

## Protocolos de APIs



# WebSockets

## HTTP Polling



### Disadvantages:

- Increased latency
- Wasted bandwidth (empty responses)
- Server resource overhead

## WebSocket

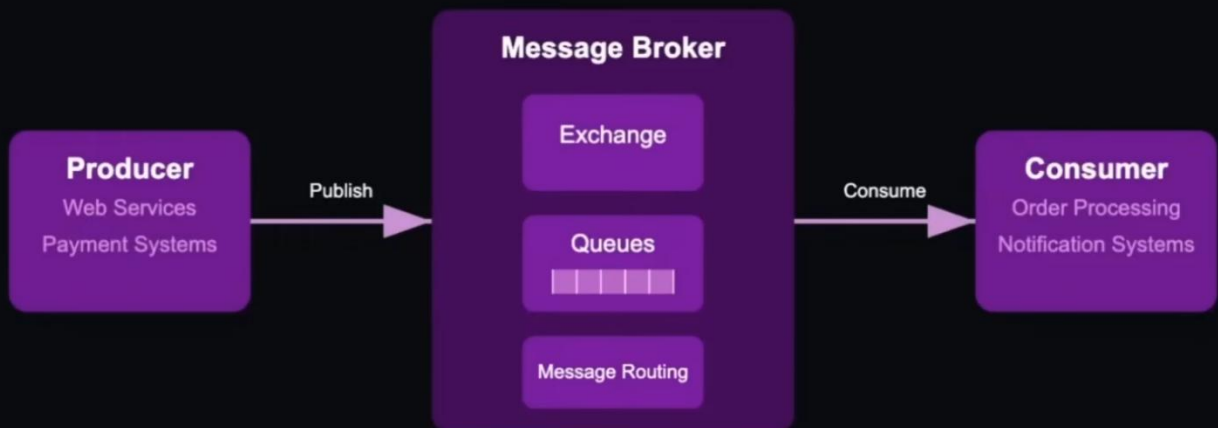


### Advantages:

- Real-time data (minimal latency)
- Reduced bandwidth usage
- Bidirectional communication

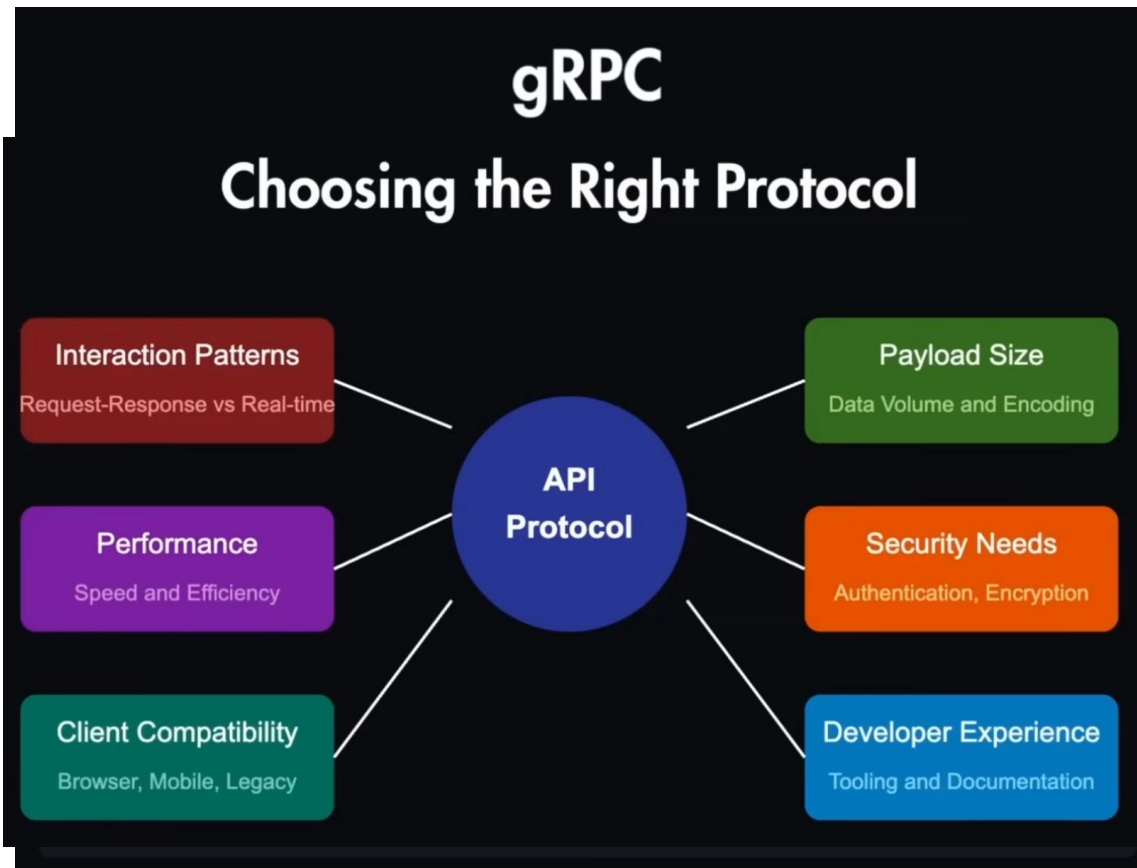
# AMQP

## Advanced Message Queuing Protocol



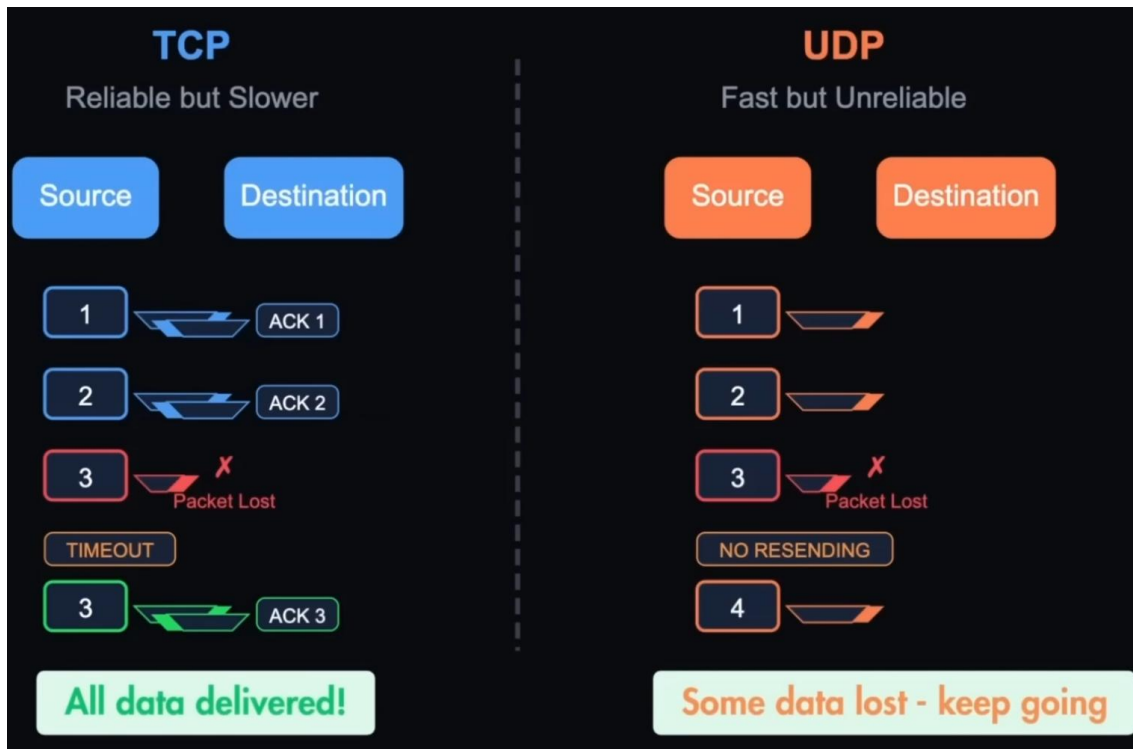
### Exchange Types





## TCP VS UDP

| <b>TCP: transmisión control protocol</b> | <b>UDP: user datagram protocol</b>                |
|------------------------------------------|---------------------------------------------------|
| Guaranteed Delivery                      | No delivery guarantee                             |
| Connection-based (3-way handshake)       | Connectionless (no handshake)                     |
| Ordered Packets (adds overhead)          | Faster transmittion (not ordered, not all arrive) |
| Error Checking                           | Lees Overhead                                     |
| 1. SYN 2. SYN-ACK 3.ACK                  | -                                                 |
| Used in banking, emails, payments, etc.  | Video, streaming, games, etc.                     |



## API

### Filtering

- `GET /products?category=books&inStock=true`

### Sorting

- `GET /products?sort=price_asc`

### Pagination

- `GET /products?page=2&limit=3`

### Benefits

- Save bandwidth.
- Improves performance.
- Gives frontend more flexibility.

# CRUD Operations

| HTTP Methods |                  |                                                        |
|--------------|------------------|--------------------------------------------------------|
| Method       | CRUD             | Example                                                |
| GET          | Read             | GET /api/v1/products<br>Safe, Idempotent               |
| POST         | Create           | POST /api/v1/products<br>Not Safe, Not Idempotent      |
| PUT          | Update (Replace) | PUT /api/v1/products/123<br>Not Safe, Idempotent       |
| PATCH        | Update (Partial) | PATCH /api/v1/products/123<br>Not Safe, Not Idempotent |
| DELETE       | Delete           | DELETE /api/v1/products/123<br>Not Safe, Idempotent    |

**Safe:** Does not change server state

**Idempotent:** Same effect if repeated

## Status Codes & Error Handling

**2xx: Success**

- 200 OK
- 201 Created
- 204 No Content

**3xx: Redirection**

**4xx: Client errors**

- 400 Bad Request
- 401 Unauthorized
- 404 Not Found

**5xx: Server errors**

```
{  "error": {    "code": "VALIDATION_ERROR",    "message": "Invalid product ID format",    "details": [...]  }}
```

**Common Status Codes & When to Use Them**

**200 OK**  
*GET /products/123 - Successfully retrieved a product*

**201 Created**  
*POST /products - Successfully created a new product*

**400 Bad Request**  
*Invalid parameters or malformed JSON*

**404 Not Found**  
*GET /products/999 - Product doesn't exist*

**500 Internal Server Error**  
*Unexpected server-side failures*

## RESTful API Best Practices

- Use plural nouns for resources.
- Keep URLs consistent and hierarchical.
- Support filtering, sorting, and pagination.
- Version your APIs

## GraphQL

### Schema Design and Type System

```
type User {
  id: ID!
  name: String!
  Posts: [Post!]!
}

type Query {
  user(id: ID!): User
}

Type Mutation {
  createUser(name: String!): User
}
```

### Queries & Mutations

```
query {
  user(id: "123"){
    name
    posts {
      title
    }
  }
}

mutation{
  createPost (
    title: "New Post"
    body: "Hello"
  ){
    Id
    title
  }
}
```

### Poor vs. Good REST API Design

#### Poor Design

```
GET /getUser/123
```

#### Good Design

```
GET /users/123
```

*Use nouns, not verbs in URLs*

#### Poor Design

```
POST /users/123/delete
```

#### Good Design

```
DELETE /users/123
```

*Use proper HTTP methods*

#### Poor Design

```
GET /products?page=3
```

#### Good Design

```
GET /products?
page=3&limit=25&sort=price:desc
```

*Support filtering, sorting and pagination*



```
}
```

All requests always return 200 OK, so we have to return an error message in the response.

```
{  
  "data": { "user": null },  
  "errors": [  
    {  
      "statusCode": 404,  
      "message": "User not found",  
      "path": [ "user" ]  
    }  
  ]  
}
```

## Best practices



## What is Authentication?

Authentication = Who are you?

Login Request => User Identity Confirmed => Authorization

Before you can access data or perform actions the system needs to know who you are...

## Basic Authentication

Structure: username + password

Basic base64 (username + password)

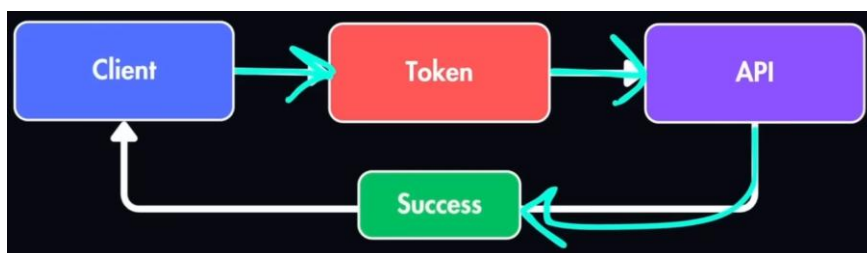
Login Request => base64 (username + password)

Base64 encoded: simple encoding that is easily reversible. So, it is insecure if not used with HTTPS.

Rarely used today outside internal tools.

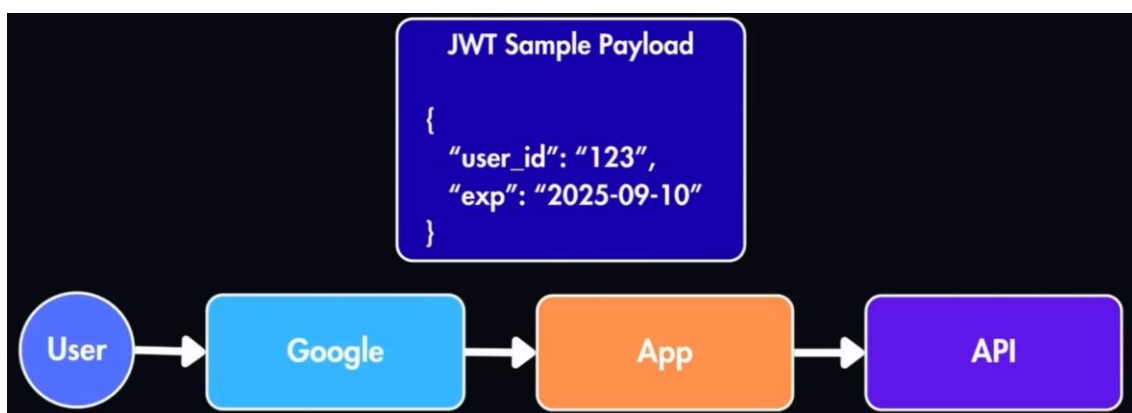
## Bearer Token Authentication

Bearer <access\_token>



Standard approach in API design today. As it is Fast and Stateless

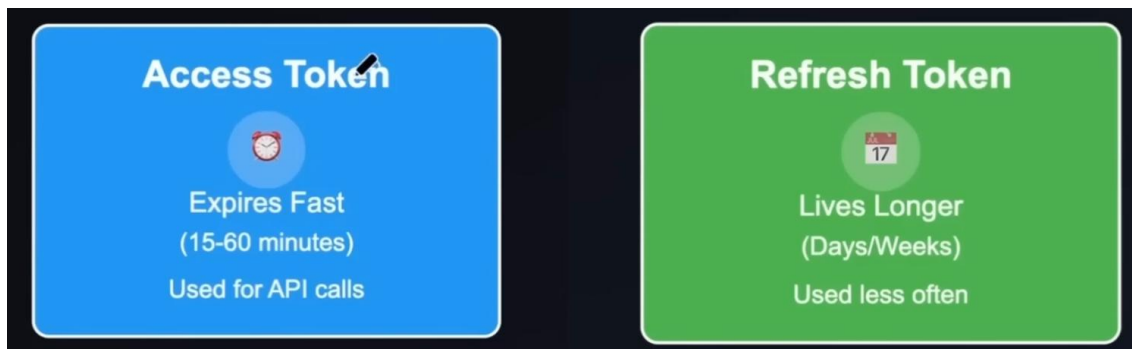
## OAuth2 + JWTs



Used to login with Google, Facebook, GitHub.

JWT are stateless => you do not need to store sessions between requests and can be executed separately.

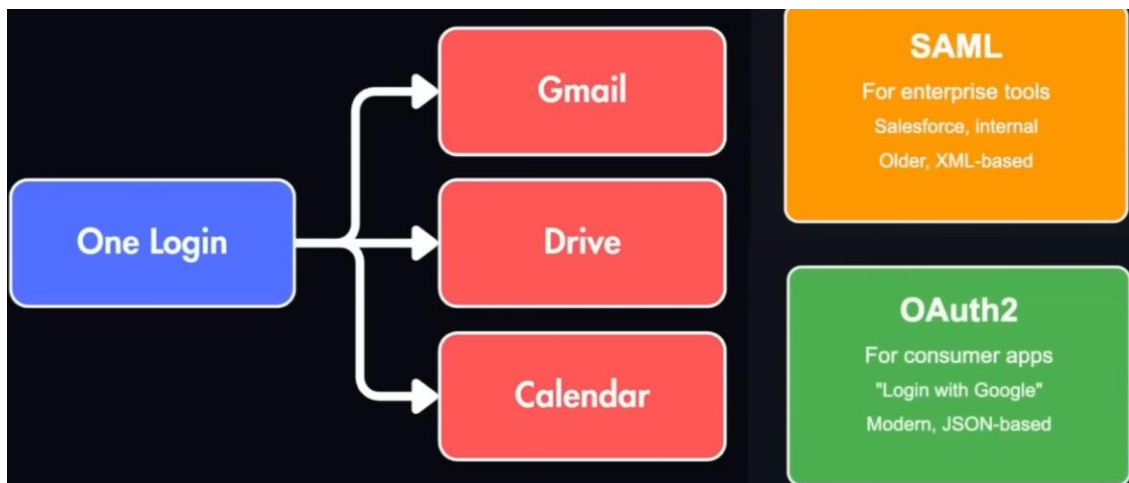
## Access vs Refresh Tokens



When access token expires, refresh token gets a new one behind the scenes.

Users do not have to log in again, and your system stays secure.

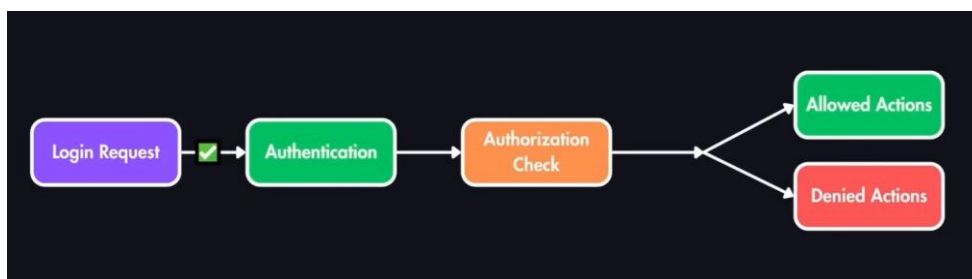
## SSO + Identity Protocols



SAML and OAuth2 are identity protocols => define how apps securely exchange user login information between each other.

## Authorization

Determines what the authenticated user can do. Controls security and permissions in a system.



How applications manage permissions

1. RBAC (Role-Based Access Control)
2. ABAC (Attribute-Based Access Control)
3. ACL (Access Control List)

+ How OAuth2 and JWTs help enforce those rules in practice.

| Model                                                                                                                  | Description                                                            |
|------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------|
|  <b>RBAC</b><br>Role-Based          | Assign roles like admin or editor<br>Most common approach              |
|  <b>ABAC</b><br>Attribute-Based     | Based on user/resource attributes<br>More flexible, more complex       |
|  <b>ACL</b><br>Access Control Lists | Each resource has its own permission list<br>Think Google Docs sharing |

**Each has trade-offs**

Real systems often combine multiple models

# RBAC

## Role-Based Access Control

**Admin**

All Access  
Create, Read, Update, Delete, Manage Users

**Editor**

Edit Content  
Create, Read, Update content

**Viewer**

Read-Only  
Read content only

**Used in most apps you use daily**

Stripe dashboards, CMS tools, team management

**Layer with scopes**

"Admin of Team A" or "Read-only for Billing"

# ACL

## Access Control Lists

**Resource**  
doc123.json  
Each resource has its own permission list

**Permission List**

|       |            |
|-------|------------|
| Alice | Read       |
| Bob   | Read/Write |
| Carol | No Access  |

**Real Example: Google Drive**

Every file has its own sharing settings

Each document maintains individual permission lists

✓ Granular control

⚠ Hard to scale

# ABAC

## Attribute-Based Access Control

**Example Policy**

Only allow access if:

```
user.department === 'HR' && time < 6PM  
&& resource.confidentiality === 'internal'
```

**User Attributes**

- Department
- Age
- Clearance Level

**Resource Attributes**

- Confidentiality
- Owner
- Classification

**Environment**

- Time of day
- Location
- Device type

**More flexible than RBAC**

But requires good policy management

Risk of policy conflicts and complexity

# OAuth2: Delegated Authorization

When one service accesses another on behalf of a user



## Access Token (Not Password!)

Represents permissions you approved  
"Read repos" but not "Delete repos"

## GitHub never shares your password

OAuth2 defines the secure flow for token issuance

# Token-Based Authorization

JWT, Bearer Tokens, and Permission Logic



## JWT Token Contents

|                 |                         |                 |                  |
|-----------------|-------------------------|-----------------|------------------|
| <b>User ID:</b> | user_123                | <b>Expires:</b> | 24h              |
| <b>Roles:</b>   | admin, editor           | <b>Issuer:</b>  | auth.example.com |
| <b>Scopes:</b>  | read:posts, write:posts |                 |                  |

## Key Distinction

Token carries identity and claims

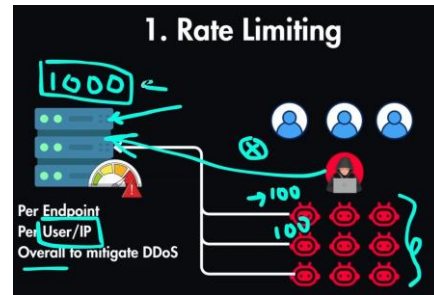
Authorization model (RBAC/ABAC/ACL) defines what's allowed

**Tokens are just the mechanism. They don't make decisions - your backend does.**

# 7 techniques to protect your APIs.

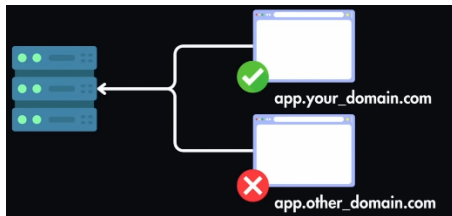
## 1. Rate Limiting

- a. Per Endpoint
- b. Per user/IP
- c. Overall limit on API to mitigate DDoS.



## 2. CORS (Cross-Origin Resource Sharing)

- a. No se permite que las requests de otras URLs o Domains usen o pidan los datos de tu API



## 3. SQL & NoSQL Injection

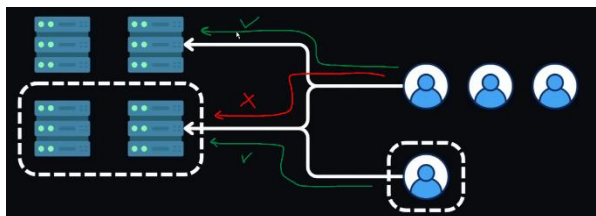
- a. ORM safeguards para denegar consultas SQL a través del frontend.

## 4. Firewalls

- a. GateKeeper filtering the malicious traffic from regular traffic. Blocks requests with unknown/suspicious requests to your API.

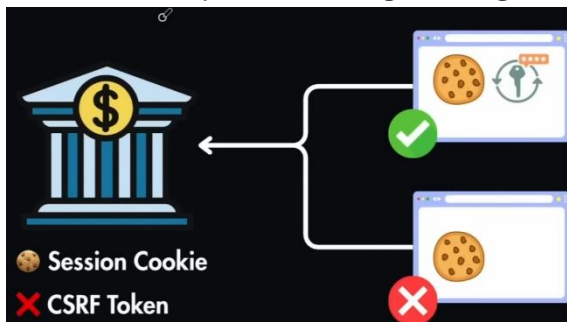
## 5. VPNs

- a. Private APIs can only be accessed by someone in that same network.



## 6. CSRF (Cross-Site Request Forgery)

- a. Used in combination with cookies in this example. Where even if the attacker has your cookie, the request will not go through the API as they would also need the CSRF Token





## 7. XSS (Cross-Site Scripting)

- a. Se inyecta un código de la base de datos del sistema. Cuando un usuario abra la sección donde esté ubicada el script malicioso se podría ejecutar en el dispositivo de un usuario.

